

Università degli studi di Padova

DIPARTIMENTO DI MATEMATICA “TULLIO LEVI-CIVITA”

CORSO DI LAUREA IN INFORMATICA



**Valutazione di pgvector come
database unificato per RAG**

Tesi di laurea triennale

Relatore

Marco Zanella

Laureando

Davide Lorenzon

Matricola 2101075

I hate perfection. To be perfect is to be unable to improve any further.

— Kurotsuchi Mayuri

Ringraziamenti

Innanzitutto, vorrei esprimere la mia gratitudine al Prof. Marco Zanella, relatore della mia tesi, per l'aiuto e il continuo sostegno fornitomi durante la stesura del lavoro.

Desidero ringraziare con affetto i miei genitori e i miei parenti per il sostegno e per essermi stati vicini durante gli anni di studio.

Ho desiderio di ringraziare poi i miei amici per tutti i bellissimi anni passati insieme.

Ci tengo infine a ringraziare i colleghi di Pat SRL e il tutor aziendale Davide Bastianetto per avermi dato l'opportunità di lavorare a questo progetto.

Padova, Settembre 2026

Davide Lorenzon

Sommario

Il presente documento descrive il lavoro svolto durante il periodo di stage curricolare, della durata di circa trecentosedici ore, dal laureando Davide Lorenzon presso l'azienda Pat SRL. Lo stage è stato condotto sotto la supervisione del tutor aziendale Davide Bastianetto, mentre il Prof. Marco Zanella ha ricoperto il ruolo di tutor accademico.

Al centro di questo elaborato vi è la progettazione e lo sviluppo del modulo di information retrieval basato su ricerca semantica, ricerca full-text e ricerca ibrida. Tale modulo è concettualmente destinato a essere utilizzato in un contesto RAG.

Lo scopo principale del progetto è valutare l'adeguatezza, la fattibilità tecnica e le performance dell'estensione pgvector per Postgres, impiegandola come database unificato. Nello specifico, l'obiettivo è verificare se tale tecnologia possa supportare efficacemente l'indicizzazione dei documenti, la ricerca ibrida (combinazione di ricerca full-text e semantica), l'applicazione di strategie di ranking avanzate e la correlazione relazionale con entità strutturate. In particolare, si valuterà una modalità di ricerca detta linked, che permette di cercare un'informazione all'interno dell'intero database e di ricostruirne il contesto.

Per convalidare questa ipotesi e fornire una misura della qualità del sistema, l'infrastruttura progettata verrà sottoposta a test di valutazione ed eventualmente test comparativi contro un sistema basato su Elasticsearch, tecnologia attualmente adottata all'interno dell'impresa.

I due sistemi potrebbero non adottare approcci equivalenti qualora pgvector e Postgres permettano di implementare funzionalità utili non attualmente utilizzate sul sistema basato su Elasticsearch.

Tale progetto è da intendersi come proof of concept con finalità puramente esplorativa e mira a verificare il reale rapporto costi e benefici di un cambiamento dello stack tecnologico.

L'utilità e il valore aggiunto che questa ricerca vuole verificare stanno nella potenziale semplificazione dell'infrastruttura IT aziendale. Gestire dati relazionali, testuali e vettoriali all'interno di un unico ecosistema permetterebbe di eliminare i workaround che implementano concetti re-

lazionali e la necessità di due sistemi di persistenza dei dati che utilizzano linguaggi diversi. Questo approccio promette di abbattere l'overhead di sincronizzazione dei dati, ridurre i costi di manutenzione sistemistica e garantire transazioni più sicure.

Organizzazione del testo

Il primo capitolo introduce l'azienda, il progetto e le motivazioni che mi hanno portato a sceglierlo;

Il secondo capitolo descrive il progetto e l'organizzazione del lavoro, definendo gli obiettivi e analizzando i rischi;

Il terzo capitolo descrive l'analisi dei requisiti del progetto, indicando un'analisi degli utenti, i casi d'uso e il tracciamento dei requisiti;

Il quarto capitolo descrive le tecnologie usate per risolvere i problemi indicando gli aspetti teorici alla base, gli strumenti che sono stati scelti e con quali criteri;

Il quinto capitolo descrive le fasi iniziali di ricerca, le informazioni così ricavate e come queste hanno influenzato lo sviluppo;

Il sesto capitolo descrive i punti più significativi dell'implementazione concreta del sistema di ricerca e le problematiche a esso collegate;

Il settimo capitolo descrive l'implementazione del sistema di test utilizzato per la valutazione, illustrandone le scelte progettuali e le difficoltà incontrate;

L'ottavo capitolo raggruppa le conclusioni tratte dallo svolgimento del progetto.

Convenzioni tipografiche

Durante la stesura del testo ho scelto di adottare le seguenti convenzioni tipografiche:

- Gli acronimi, le abbreviazioni e i termini di uso non comune menzionati vengono definiti nel **glossario**, situato alla fine del documento (**p. 133**);
- Per la prima occorrenza dei termini riportati nel glossario viene utilizzata la seguente nomenclatura: *Termine*;
- I nomi di funzioni o variabili appartenenti a un linguaggio di programmazione vengono scritti con un carattere **monospaziato**;
- Le citazioni ad un libro o ad una risorsa presente nella **bibliografia** (**p. 139**) saranno affiancate dal rispettivo numero identificativo, es. [1];
- I link a fonti esterne al documento sono contraddistinti dal [colore blu e da una sottolineatura](#);
- I link a sezioni, tabelle, figure, codici o altro testo interni al documento sono contraddistinti dal **peso semibold e da una sottolineatura**;
- I blocchi di codice sono rappresentati nel seguente modo:

```
1  float Q_rsqr( float number ){
2      long i;
3      float x2, y;
4      const float threehalfs = 1.5F;
5      x2 = number * 0.5F;
6      y = number;
7      i = * (long * ) &y;
8      i = 0x5f3759df - (i>>1);
9      y = * (float * ) &i;
10     y = y * ( threehalfs - ( x2 * y * y ) );
11     //y = y * ( threehalfs - ( x2 * y * y ) );
12     return y;
13 }
```

Codice 1: Codice d'esempio

Indice

1	Introduzione	1
1.1	L'azienda	1
1.2	Il progetto	1
1.3	Scelta del progetto	3
2	Descrizione stage	5
2.1	Competenze da apprendere	5
2.2	Vincoli	5
2.3	Pianificazione	6
2.4	Analisi dei rischi	6
3	Analisi dei requisiti	15
3.1	Analisi degli utenti	15
3.1.1	Attori primari	15
3.1.2	Attori secondari	16
3.2	Casi d'uso	16
3.2.1	Struttura della scheda descrittiva	16
3.2.2	Lista dei casi d'uso	18
3.3	Tracciamento dei requisiti	49
4	Introduzione Teorica	61
4.1	Contesto e problema	61
4.1.1	Caratteristiche di Elasticsearch	62
4.1.2	Caratteristiche di Postgres	63
4.1.3	Motivi del confronto	63
4.1.4	Limiti del sistema attuale	64
4.1.5	Limiti di Postgres e pgvector	64
4.2	Basi teoriche	65
4.3	Architettura del progetto	66
4.3.1	Sistema di ricerca	67
4.3.2	Sistema di test	67
4.3.3	Dashboard Grafana	67
4.3.4	Relazione tra i sistemi	67
4.4	Criteri di scelta delle tecnologie	68
4.4.1	Sistema di ricerca	68
4.4.2	Sistema di test	68
4.5	Tecnologie	68
4.5.1	Sistema di ricerca	70

4.5.1.1 Backend	70
4.5.1.2 Database	72
4.5.1.3 Deployment	73
4.5.1.4 Strumenti di supporto	73
4.5.2 Sistema di test	73
4.5.2.1 Backend	73
4.5.2.2 Database	74
4.5.2.3 Deployment	74
4.5.2.4 Strumenti di supporto	74
5 Principi e caratteristiche del sistema	75
5.1 Studio iniziale	75
5.2 Principi del sistema di ricerca	76
5.2.1 Definizione modello dati	76
5.2.2 Caratteristiche del database	78
5.2.2.1 Gestione della staging area	78
5.2.3 Tipologie di ricerca e requisiti implementativi	79
5.2.3.1 Ricerca semantica	80
5.2.3.2 Ricerca full-text	81
5.2.3.2.1 Ricerca con soglia di corrispondenza ..	82
5.2.3.2.2 Gestione dei tsvector	83
5.2.3.3 Ricerca ibrida	84
5.2.3.4 Ricerca linked	85
5.2.3.5 Pipeline di esecuzione delle ricerche	86
5.2.4 Caratteristiche del backend	88
5.3 Principi del sistema di test	89
5.3.1 Definizione modello dati	89
5.3.2 Metriche e il loro significato	90
5.3.3 Caratteristiche del backend	90
5.3.4 Caratteristiche del database	91
6 Implementazione del sistema di ricerca	93
6.1 Architettura del codice	93
6.1.1 Classi di dominio condivise	93
6.1.1.1 Vincoli sullo schema	95
6.1.1.2 Ricerca linked e struttura a grafo	96
6.1.1.3 Radice dell'aggregato	97
6.2 Sistema di persistenza	98
6.3 Ingestion	101
6.3.1 Elaborazione dei batch e uso dello stream	102
6.3.2 Validazione, arricchimento e scrittura	102
6.3.3 Promozione da staging a tabelle reali	103
6.4 Ricerca	105
6.4.1 Filtering	106
6.4.2 Query e specializzazioni	107

6.4.3 Ricerca semantica	109
6.4.4 Ricerca full-text	111
6.4.5 Ricerca ibrida	115
6.4.6 Ricerca linked	117
7 Implementazione del sistema di test	121
7.1 Perimetro di test	121
7.2 Architettura del codice	122
7.3 Sistema di persistenza	123
7.4 Start test	124
7.5 Visualizzazione dei risultati	124
7.6 Limitazioni imposte da elementi esterni	126
8 Conclusioni	129
8.1 Consuntivo finale	129
8.2 Requisiti soddisfatti	130
8.3 Rischi occorsi e mitigati	130
8.4 Valutazione complessiva sulle tecnologie	131
8.5 Valutazione personale	132
Glossario	133
Bibliografia	139

Elenco delle Figure

Figura 1	Logo Pat SRL	1
Figura 2	Diagramma del caso d'uso: Ingestion di documenti	18
Figura 3	Diagramma del caso d'uso: Ingestion liste entità	19
Figura 4	Diagramma del caso d'uso: Ingestion lista entità	20
Figura 5	Diagramma del caso d'uso: Caricamento blocco entità ...	21
Figura 6	Diagramma del caso d'uso: Termina ingestion	23
Figura 7	Diagramma del caso d'uso: Ricerca su singola entità	25
Figura 8	Diagramma del caso d'uso: Inserimento query su singola entità	26
Figura 9	Diagramma del caso d'uso: Ricerca semantica	27
Figura 10	Diagramma del caso d'uso: Ricerca ibrida	28
Figura 11	Diagramma del caso d'uso: Ricerca ibrida con modello di reranking	30
Figura 12	Diagramma del caso d'uso: Ricerca linked	32
Figura 13	Diagramma del caso d'uso: Inserimento query linked ...	33
Figura 14	Diagramma del caso d'uso: Ricerca linked semantica ...	34
Figura 15	Diagramma del caso d'uso: Ricerca linked ibrida	35
Figura 16	Diagramma del caso d'uso: Ricerca linked ibrida con modello di reranking	37
Figura 17	Diagramma del caso d'uso: Avvia test	39
Figura 18	Diagramma del caso d'uso: Visualizza dashboard performance	40
Figura 19	Diagramma del caso d'uso: Visualizza metriche di performance	42
Figura 20	Diagramma del caso d'uso: Visualizzazione retrieval latency	44
Figura 21	Logo Python	69
Figura 22	Logo Postgres	69
Figura 23	Logo Grafana	70
Figura 24	Logo FastAPI	70
Figura 25	Diagramma ER del core del database	99
Figura 26	Diagramma ER delle tabelle di supporto allo staging ...	100
Figura 27	Diagramma ER delle tabelle di supporto al tracking ...	100

Elenco delle Tabelle

Tabella 1	Campi dei casi d'uso	16
Tabella 2	Requisiti funzionali	49
Tabella 3	Requisiti di qualità	57
Tabella 4	Requisiti di vincolo	57
Tabella 5	Riepilogo dei requisiti	60
Tabella 6	Consuntivo orario finale	129
Tabella 7	Riepilogo dei requisiti soddisfatti	130
Tabella 8	Rischi occorsi con la loro mitigazione	130

Elenco dei Codici Sorgente

Codice 1	Codice d'esempio	ix
Codice 2	Entity - dataclass e vincolo searchable/chunked_text ...	95
Codice 3	SchemaConstraint e RelationalSchemaConstraint	96
Codice 4	LinkedSearchConfiguration e TraversalPath	97
Codice 5	SchemaConfiguration - dataclass	98
Codice 6	SchemaConfigurationBuilder - firma di build()	98
Codice 7	Firma porta di ingestion dei dati	103
Codice 8	Staging promotion - selezione degli elementi candidati .	103
Codice 9	Staging promotion - gestione dei duplicati	104
Codice 10	Staging promotion - estrazione dei valori	104
Codice 11	Staging promotion - inserimento nella tabella reale	105
Codice 12	Staging promotion - conteggio degli elementi processati .	105
Codice 13	Ricerca - interfaccia FilterCondition e implementazioni concrete	106
Codice 14	Ricerca - type alias per filtro su singola entità e post-join	107
Codice 15	Ricerca - Query e specializzazioni	107
Codice 16	Ricerca - Query result e specializzazioni	108
Codice 17	Ricerca semantica - selezione dei candidati da una singola partizione	109
Codice 18	Ricerca semantica - combinazione dei candidati tra partizioni	110
Codice 19	Ricerca semantica - join con la tabella principale	110
Codice 20	Ricerca full-text - estrazione dei lessemi e calcolo della soglia di corrispondenza	112
Codice 21	Ricerca full-text - selezione dei candidati per corrispondenza e ranking combinato	113
Codice 22	Ricerca full-text - filtro finale sulla soglia di corrispondenza	114
Codice 23	Ricerca full-text - ordinamento dei lessemi per lunghezza	115
Codice 24	Ricerca full-text - ordinamento dei lessemi per frequenza	115

Codice 25	Ricerca full-text - Materialized view per le frequenze ...	115
Codice 26	Ricerca ibrida - fusione RRF pesata tramite FULL OUTER JOIN	116
Codice 27	Ricerca linked - un ramo di risalita lungo un TraversalPath	117
Codice 28	Ricerca linked - combinazione di tutti i rami e ordinamento finale	118
Codice 29	Sistema di test - EvaluationService	123
Codice 30	Sistema di test - Vista metriche	125
Codice 31	Sistema di test - Variabile Grafana	126
Codice 32	Sistema di test - Time series	126
Codice 33	Sistema di test - Gauge	126

Capitolo 1

Introduzione

In questo capitolo descrivo l'azienda, introduco il progetto e spiego le motivazioni che mi hanno portato a sceglierlo.

1.1 L'azienda

Pat SRL è un'azienda parte del Gruppo Zucchetti, che vanta un'esperienza ultratrentennale nello sviluppo di soluzioni software destinate sia ad aziende private che a istituzioni pubbliche. L'azienda si posiziona come partner tecnologico specializzato nella progettazione di piattaforme per la gestione e l'automazione dei processi aziendali e dei servizi di assistenza e supporto.



Figura 1: Logo Pat SRL

1.2 Il progetto

Il progetto ha come obiettivo principale la realizzazione di un *proof of concept (PoC)_G* valutativo della riprogettazione e della sostituzione dell'attuale architettura dati utilizzata per la persistenza e il recupero delle informazioni all'interno dei prodotti aziendali.

Attualmente, l'impresa adotta una soluzione basata sul paradigma della *persistenza poliglotta_G*.

1 Introduzione

Tale architettura prevede l'utilizzo congiunto di due sistemi separati: Postgres per la gestione dei dati strettamente relazionali ed *Elasticsearch_G* per l'indicizzazione dei documenti, la *ricerca full-text_G*, la gestione degli *embedding_G* vettoriali e le operazioni di filtraggio avanzato.

Sebbene questo approccio ibrido sia funzionale e ampiamente utilizzato, la divisione dei carichi di lavoro su motori di database differenti comporta diverse limitazioni architettoniche e operative:

- denormalizzazione dei dati, la natura orientata ai documenti e non relazionale di Elasticsearch obbliga a replicare e denormalizzare i dati relazionali per consentire un filtraggio efficiente, aumentando la ridondanza e forzando a implementare in modo non ottimale funzioni come i join;
- overhead di sincronizzazione, il mantenimento della coerenza tra il database primario Postgres e il motore di ricerca Elasticsearch richiede complesse pipeline di allineamento, esponendo il sistema di ricerca a ritardi di sincronizzazione o disallineamenti;
- mancanza di garanzie *ACID_G* globali, operando su sistemi separati, risulta complesso garantire l'atomicità e la consistenza transazionale durante l'inserimento o l'aggiornamento simultaneo di dati strutturati e vettoriali.

Per superare queste criticità, il progetto esplora la transizione verso un paradigma a database unificato. L'obiettivo è accentrare l'intero carico di lavoro su Postgres sfruttando *pgvector_G*, un'estensione open-source che introduce il supporto nativo alla persistenza dei vettori di embedding e alle operazioni di algebra lineare direttamente all'interno dell'ecosistema relazionale.

Nello specifico, il nuovo sistema di ricerca dovrà soddisfare i seguenti requisiti implementativi:

- *ingestion_G*, sviluppo di un modulo dedicato all'inserimento simultaneo di dati relazionali e documenti;
- ottimizzazione degli indici, sfruttamento delle capacità di indicizzazione full-text native di Postgres e creazione di indici vettoriali dedicati tramite *pgvector* per garantire l'efficienza scalabile della *ricerca semantica_G*;
- integrazione relazionale, utilizzo di costrutti *SQL_G* per correlare dinamicamente i documenti e i vettori tra le varie entità del sistema di ricerca;
- *ricerca ibrida_G*, implementazione di una logica di recupero che combini la precisione lessicale della ricerca testuale con la profondità concettuale della ricerca semantica, fondendo i risultati tramite l'algoritmo di *RRF_G*;

1 Introduzione

- *ricerca linked_G*, modalità che permette di analizzare automaticamente ogni entità del sistema di ricerca e di ricostruire un quadro complessivo dell'informazione tramite join.

Al fine di validare rigorosamente l'efficacia di questa nuova architettura unificata, il sistema di ricerca verrà sottoposto a una fase di benchmarking contro la soluzione attualmente in uso.

I test comparativi si concentreranno sulle seguenti metriche chiave:

- latenza di interrogazione: misurazione dei tempi di risposta durante la ricerca testuale, semantica e ibrida;
- velocità di indicizzazione: tempi necessari per l'elaborazione e l'inserimento nel database di nuovi record complessi;
- impatto sullo storage: analisi dello spazio su disco occupato dai dati e dai relativi indici;
- complessità della pipeline: valutazione qualitativa della semplificazione architetturale.

1.3 Scelta del progetto

Ho scelto questo progetto per tre ragioni principali:

1. rilevanza dell'argomento, alla base dei moderni sistemi di intelligenza artificiale, come la *RAG_G*, che utilizzano l'*information retrieval (IR)_G* per fornire contesto agli *LLM_G*;
2. evoluzione di un sistema di ricerca reale, permette di partecipare all'evoluzione di un software, sfida che durante il percorso universitario non ho affrontato;
3. stack tecnologico, offre l'opportunità di operare con strumenti e framework moderni.

1 Introduzione

Capitolo 2

Descrizione stage

In questo capitolo approfondisco l'organizzazione dello stage, il rapporto con l'azienda e svolgo l'analisi dei rischi.

2.1 Competenze da apprendere

Il tirocinio è strutturato per favorire lo sviluppo di un ventaglio di competenze trasversali, che spaziano dalla progettazione architettuale di alto livello fino all'implementazione tecnica.

Dal punto di vista metodologico, lo stage mira a consolidare le capacità di analisi dei requisiti, astrazione dei problemi complessi e progettazione di soluzioni algoritmiche generali e scalabili, promuovendo inoltre la collaborazione con il tutor.

Sotto il profilo tecnico, il percorso formativo permetterà di acquisire e approfondire le seguenti tematiche:

- database avanzati, padronanza nell'utilizzo di Postgres non solo come database relazionale, ma come motore di information retrieval vettoriale tramite l'estensione pgvector;
- progettazione della ricerca ibrida, studio e valutazione delle tecnologie di indicizzazione;
- testing delle performance, acquisizione di metodologie per la conduzione di benchmark, definendo metriche di valutazione per misurare le performance e l'efficienza dei sistemi sviluppati.

Per garantire un confronto equo e rigoroso con Elasticsearch, oltre alla ricerca full-text nativa di Postgres, verrà esplorata l'integrazione di *ParadeDB_C*, una soluzione basata su Postgres che promette capacità di ricerca lessicale più avanzate.

2.2 Vincoli

Il progetto è soggetto a specifici vincoli architettureali volti a garantire la coerenza con gli obiettivi della ricerca.

I vincoli principali sono i seguenti:

2 Descrizione stage

- utilizzo di pgvector per la ricerca semantica, obiettivo principale del progetto;
- utilizzo della ricerca full-text nativa di Postgres, per semplicità di licensing.

Estensioni più evolute come ParadeDB, che si propongono come sostituto diretto della componente full-text di Elasticsearch su Postgres, sono state valutate ma escluse dall'implementazione finale per rispettare il vincolo di licensing; il loro studio è stato comunque utile per orientare l'implementazione delle funzionalità di ricerca full-text nativa.

2.3 Pianificazione

Lo stage si articola in 316 ore distribuite su sette settimane da 40 ore e una da 36.

La pianificazione, derivata dal piano di lavoro, è la seguente:

1. Prima settimana - Studio e analisi iniziale del nuovo e del vecchio stack tecnologico e setup: studio delle tecnologie, setup dell'ambiente di lavoro locale e prove generiche;
2. Seconda settimana - Analisi comparativa dettagliata di Elasticsearch e Postgres con tentativo di modellazione di un sottoinsieme limitato del problema;
3. Terza settimana - Studio del dominio, definizione dei requisiti e dei casi d'uso e definizione dello schema relazionale;
4. Quarta settimana - Studio del dominio, definizione dei requisiti e dei casi d'uso e definizione dello schema relazionale e ottimizzazione tramite indici;
5. Quinta settimana - Progettazione di alto livello del sistema di ricerca e inizio della codifica del modulo di ingestion;
6. Sesta settimana - Completamento della codifica del modulo di ingestion e dei moduli di ricerca, completa delle relative interfacce;
7. Settima settimana - Progettazione e implementazione del sistema di test partendo da dei dati locali fittizi;
8. Ottava settimana - Benchmarking, realizzazione della dashboard e deployment.

2.4 Analisi dei rischi

I rischi identificati per questo progetto sono classificati con un codice progressivo della forma **RN**, dove **N** è un numero intero incrementale che parte da 01 e decorati con una probabilità di occorrenza, un impatto e una strategia di mitigazione.

2 Descrizione stage

Ogni rischio è stato analizzato tenendo conto della complessità di comprendere i casi d'uso del sistema di ricerca di paragone Elasticsearch, delle sue scelte implementative dovute allo stack tecnologico e alla comprensione degli interessi sperimentativi dell'impresa.

Codice: R01

Incompletezza o ambiguità dei requisiti

- **Descrizione:**

I requisiti descritti nel piano di lavoro possono risultare incompleti o ambigui, portando a implementazioni non coerenti con le aspettative aziendali.

- **Mitigazione:**

I requisiti vengono analizzati e chiariti prima delle attività di codifica. Viene data priorità ad attività di studio teorico dello stack tecnologico e alla realizzazione di prototipi a diversi livelli di complessità. Sono previsti incontri iterativi con il tutor per definire chiaramente i confini del progetto e le interfacce di comunicazione.

- **Probabilità:** Medio-Alta

- **Impatto:** Medio

Codice: R02

Sovradimensionamento del progetto rispetto alle tempistiche

- **Descrizione:**

Le attività previste potrebbero rivelarsi eccessive rispetto al monte ore disponibile e alle tempistiche di apprendimento, con il rischio di non completare tutti gli obiettivi prefissati.

- **Mitigazione:**

Le attività sono suddivise per priorità (requisiti obbligatori, desiderabili, opzionali). In caso di ritardi o imprevisti strutturali, solo le attività a priorità inferiore e non vincolanti per lo scopo principale del progetto subiranno ridimensionamenti.

- **Probabilità:** Media

- **Impatto:** Alto

2 Descrizione stage

Codice: R03

Difficoltà nel coordinamento interno

- **Descrizione:**
La comunicazione con il tutor aziendale o con gli stakeholder potrebbe risultare discontinua, rallentando il processo decisionale tecnico e causando incomprensioni.
- **Mitigazione:**
Vengono pianificati incontri periodici di allineamento, accompagnati da aggiornamenti asincroni frequenti sullo stato di avanzamento. Eventuali blocchi tecnici vengono segnalati tempestivamente senza attendere la riunione successiva.
- **Probabilità:** Media
- **Impatto:** Alto

Codice: R04

Curva di apprendimento delle tecnologie

- **Descrizione:**
L'assimilazione delle tecnologie necessarie potrebbe richiedere un tempo di studio superiore alle stime iniziali.
- **Mitigazione:**
Le prime settimane dello stage includono ore dedicate esplicitamente allo studio della documentazione ufficiale, alla schematizzazione delle architetture e alla realizzazione di proof of concept isolati.
- **Probabilità:** Bassa
- **Impatto:** Medio

Codice: R05

Discostamento dal sistema legacy

- **Descrizione:**

Il modulo realizzato dovrà essere paragonabile a quello di produzione; potrebbero emergere attriti o altre differenze significative sia a livello di comportamento interno che a livello di interazione con l'esterno.

- **Mitigazione:**

Confronto continuo con il team di sviluppo per comprendere a fondo le funzionalità da implementare. Adozione del design pattern *Adapter* fin dalle prime fasi di progettazione per disaccoppiare la forma dei dati accettata dal sistema di ricerca dalla forma dei dati che deve essere accettata dall'esterno.

- **Probabilità:** Media

- **Impatto:** Medio

Codice: R06

Saturazione delle risorse durante i benchmark

- **Descrizione:**

L'esecuzione dei test comparativi su volumi di dati enterprise (fino a 500.000 record) potrebbe causare la saturazione delle risorse hardware dell'ambiente di test, falsando le metriche o causando crash di sistema.

- **Mitigazione:**

I test verranno eseguiti in modo incrementale su dataset di dimensioni crescenti.

Verrà implementato un monitoraggio attivo delle risorse tramite il framework di observability per individuare eventuali memory leak o configurazioni sub-ottimali degli indici vettoriali prima dei test massivi.

Inoltre è previsto il deployment del sistema di ricerca su server remoto.

- **Probabilità:** Media

- **Impatto:** Alto

Codice: R07

Rappresentatività dei dati di test e accesso limitato alla produzione

- **Descrizione:**

L'utilizzo di dati generati appositamente per le fasi di test locali, reso necessario dai vincoli di privacy aziendale, potrebbe non riflettere appieno la reale complessità e varianza del dominio. Questa discrepanza rischia di alterare la valutazione dell'accuratezza della ricerca e di non far emergere casistiche limite verosimili, portando a possibili divergenze di performance tra l'ambiente di sviluppo e le aspettative finali.

- **Mitigazione:**

La validazione seguirà un approccio a due fasi: l'architettura dovrà innanzitutto superare i benchmark di riferimento in ambiente locale utilizzando dei dati di test.

Ulteriori misurazioni con dati reali saranno effettuate successivamente; le misurazioni definitive non verranno trattate in questo progetto ma verranno effettuate dall'azienda sui dati reali operando esclusivamente all'interno dei server proprietari aziendali, nel pieno rispetto delle direttive di sicurezza e privacy, utilizzando i risultati del sistema di ricerca attuale come punto di riferimento.

Si è comunque consapevoli che il contributo principale all'accuratezza deriva da parametri esterni o configurabili, come i modelli di embedding utilizzati, i tipi di vettore e distanza utilizzati per il calcolo, le configurazioni testuali applicate e la struttura della funzione di *ranking*.

- **Probabilità:** Alta

- **Impatto:** Basso

Codice: R08

Difficoltà nella valutazione oggettiva dei risultati di retrieval

- **Descrizione:**

L'assenza di metriche standardizzate o un'errata interpretazione dei risultati potrebbe portare a conclusioni soggettive, rendendo impossibile valutare se il nuovo sistema di ricerca eguaglia o meno la soluzione precedente.

- **Mitigazione:**

Le metriche di valutazione verranno definite esplicitamente nella fase di analisi dei requisiti.

Queste corrispondono alle metriche attualmente usate per la valutazione del sistema di ricerca attuale.

È inoltre previsto un caso d'uso specifico per la produzione di una dashboard di monitoraggio, con criteri di accettazione e soglie minime di qualità chiaramente prestabiliti.

- **Probabilità:** Media

- **Impatto:** Alto

Codice: R09

Sbilanciamento metodologico nel confronto

- **Descrizione:**

Essendo Postgres nato come sistema per la gestione di database relazionali ed Elasticsearch come motore di ricerca con analizzatori linguistici avanzati, testare scenari non calibrati rischia di favorire asimmetricamente una delle due tecnologie, invalidando l'equità scientifica del benchmark.

- **Mitigazione:**

Il set di query e il benchmark verranno definiti e congelati a priori, mirando a rispecchiare l'effettivo utilizzo ed evitando scenari costruiti ad hoc per avvantaggiare una specifica piattaforma.

- **Probabilità:** Alta

- **Impatto:** Alto

Codice: R10

Bias tecnologico e deriva dei requisiti

- **Descrizione:**

Esiste il rischio di adattare inconsciamente l'implementazione per favorire le peculiarità di Postgres, introducendo logiche non necessarie o deviazioni dai requisiti originali solo per giustificare l'adozione della nuova tecnologia.

- **Mitigazione:**

Adozione dell'*architettura esagonale*. Isolando la logica di dominio dall'infrastruttura di persistenza, si garantisce che i dettagli implementativi di Postgres non inquinino il comportamento atteso del sistema di ricerca.

- **Probabilità:** Media

- **Impatto:** Alto

Codice: R11

Dispersione del perimetro di test su tecnologie alternative

- **Descrizione:**

L'evoluzione rapida dell'information retrieval solleva l'interrogativo su alternative tecnologiche, come Open-Search.

La loro valutazione va fuori dagli interessi dell'impresa per questo specifico tirocinio e rischia di aggiungere attività di studio non utili alla realizzazione del progetto.

- **Mitigazione:**

I confini del tirocinio sono rigidamente circoscritti al confronto diretto tra la soluzione in uso, basata su Elasticsearch e le tecnologie che l'impresa vuole valutare, Postgres e pgvector.

Eventuali tecnologie alternative verranno affrontate esclusivamente a livello teorico.

- **Probabilità:** Bassa

- **Impatto:** Medio

Codice: R12

Sovrapposizione tra le performance di retrieval e generazione

- **Descrizione:**

L'architettura RAG si compone di due fasi: la fase di retrieval, che si occupa del recupero delle informazioni e la fase di generazione della risposta. Nel sistema di ricerca attuale solo la parte di retrieval e ingestion di documenti è collegata strettamente a Elasticsearch. Le altre parti del sistema non sono sottoposte a valutazione nell'ambito di questo progetto.

- **Mitigazione:**

Vengono posti dei rigidi confini sul sistema da implementare. L'implementazione, l'analisi e il testing si concentreranno unicamente sulla componente di retrieval e sul relativo modulo di ingestion. La valutazione ignorerà la qualità dell'output generativo dell'LLM, misurando esclusivamente la pertinenza dei documenti e la velocità di recupero.

- **Probabilità:** Bassa

- **Impatto:** Alto

2 Descrizione stage

Capitolo 3

Analisi dei requisiti

In questo capitolo effettuo l'analisi degli utenti, sviluppo gli use case e compongo la lista dei requisiti dividendoli per tipologia e necessità.

3.1 Analisi degli utenti

Al fine di modellare correttamente le interazioni e definire i confini del sistema oggetto del tirocinio, è necessario individuare gli attori coinvolti. In accordo con le metodologie dell'ingegneria del software, gli attori comprendono sia gli utenti umani sia i sistemi software esterni che interagiscono direttamente con l'architettura.

Gli attori sono classificati in due categorie:

- Attori primari: entità che avviano i casi d'uso e richiedono un servizio al sistema.
- Attori secondari: servizi o sistemi esterni invocati dal sistema per completare una specifica elaborazione.

3.1.1 Attori primari

- **Companion**

Con il termine **Companion** si identifica l'applicativo aziendale di intelligenza artificiale di livello superiore. Questo attore software è il client principale: è il software applicativo reale che utilizza il modulo di information retrieval.

Companion interagisce con il sistema di ricerca tramite chiamate

APIG.

Companion interagisce con il sistema di ricerca per due scopi fondamentali: delegare l'ingestion dei dati documentali e interrogare la base di conoscenza per ottenere le informazioni necessarie alla generazione delle risposte.

3 Analisi dei requisiti

- **Supervisore**

Rappresenta l'utente tecnico qualificato. Il suo ruolo è quello di interfacciarsi con il sistema di test a scopo di analisi, validazione e benchmarking. Il Supervisore avvia i test prestazionali, monitora l'infrastruttura e raccoglie le metriche necessarie per valutare e comparare le prestazioni del nuovo database unificato rispetto alla soluzione attualmente usata in produzione.

3.1.2 Attori secondari

- **Modello di embedding**

Si tratta di un attore software esterno che fornisce il servizio di calcolo degli embedding. Il sistema di ricerca oggetto dello stage invoca questo attore delegandogli il compito computazionale di trasformare i *chunk di testo* grezzo in vettori numerici, che verranno poi usati per popolare l'indice vettoriale e per l'esecuzione della ricerca semantica.

- **Modello di reranking**

Si tratta di un attore software esterno che realizza la funzione di *reranking* dei risultati della ricerca semantica e della ricerca full-text.

3.2 Casi d'uso

In questa sezione vengono definiti i casi d'uso del sistema. Ogni caso d'uso è univocamente identificato da una sigla progressiva (es. **UCXX**) ed è corredato da una scheda descrittiva analitica. Gli attori menzionati fanno diretto riferimento alle definizioni riportate nella **Sezione 3.1**.

3.2.1 Struttura della scheda descrittiva

Ciascun caso d'uso è documentato attraverso le informazioni elencate nella tabella seguente. Al fine di mantenere la documentazione concisa, i campi non rilevanti o non applicabili a uno specifico scenario verranno omessi.

Campo	Descrizione
Grafico UML	Rappresentazione visiva dello scenario tramite diagramma UML dei casi d'uso.
Attore principale	Entità esterna che avvia l'interazione con il sistema per raggiungere un obiettivo specifico.

3 Analisi dei requisiti

Campo	Descrizione
Attore secondario	Sistema o servizio esterno invocato dal sistema per completare una determinata funzionalità.
Scenario principale	La sequenza nominale di operazioni necessaria per portare a compimento il caso d'uso senza interruzioni.
Precondizioni	Stato del sistema o requisiti che devono essere necessariamente soddisfatti prima dell'avvio del caso d'uso.
Postcondizioni	Stato del sistema o modifiche persistenti che si verificano al termine della corretta esecuzione dello scenario principale.
Scenario alternativo	Flussi di esecuzione secondari che deviano dallo scenario base, tipicamente per gestire anomalie, errori o percorsi non standard.
Inclusioni	Relazione verso un altro caso d'uso, il cui comportamento è obbligatoriamente e incondizionatamente incorporato nello scenario corrente.
Estensioni	Relazione che introduce un comportamento opzionale o alternativo, attivato unicamente al verificarsi di specifiche condizioni.
Specializzazioni	Varianti strutturali del caso d'uso generale. I casi d'uso specializzati sono tra loro mutualmente esclusivi ed ereditano i comportamenti dello scenario base.
Trigger	L'evento, l'azione o la condizione scatenante che innesca l'esecuzione del caso d'uso.

Tabella 1: Campi dei casi d'uso

3 Analisi dei requisiti

3.2.2 Lista dei casi d'uso

UC01: Ingestion di documenti

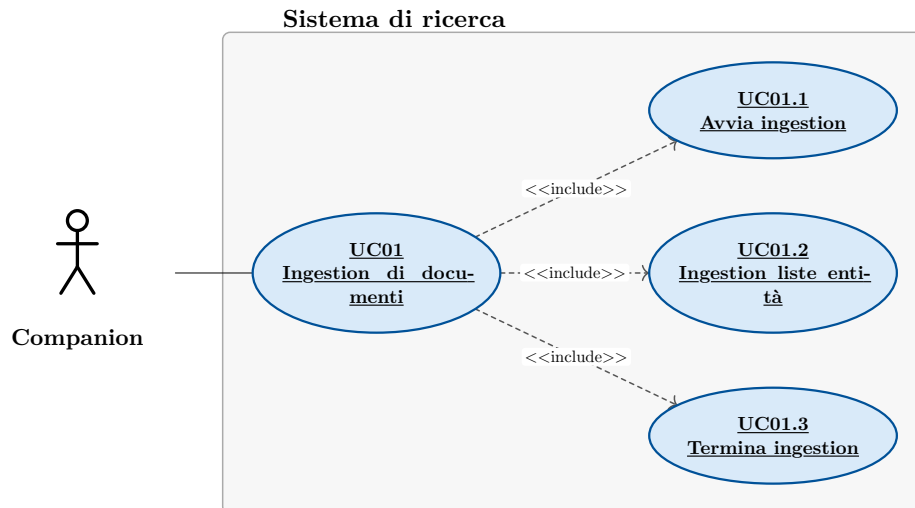


Figura 2: Diagramma del caso d'uso: Ingestion di documenti

- **Attore principale:** Companion
- **Precondizioni:**
 - Il sistema di ricerca è attivo
 - Nel sistema di ricerca non ci sono sessioni di ingestion di documenti attive
- **Postcondizioni:**
 - Il sistema di ricerca ha salvato le informazioni ricevute
 - Il sistema di ricerca ha salvato gli embedding relativi ai dati
 - Il sistema di ricerca ha salvato le informazioni di lingua relative ai dati
 - Il sistema di ricerca ha reso le informazioni disponibili per la ricerca
- **Scenario principale:**
 1. Companion avvia il processo di ingestion dei dati → **UC01.1**
 2. Companion carica le liste di entità → **UC01.2**
 3. Companion termina il processo di ingestion → **UC01.3**
 4. Companion viene notificato del corretto salvataggio dei dati
- **Inclusioni:**
 - **UC01.1**
 - **UC01.2**
 - **UC01.3**
- **Trigger:** Companion vuole caricare dei documenti sul sistema di ricerca

3 Analisi dei requisiti

UC01.1: Avvia ingestion

- **Attore principale:** Companion
- **Precondizioni:**
 - Il sistema di ricerca è attivo
 - Nel sistema di ricerca non ci sono sessioni di ingestion di documenti attive
- **Postcondizioni:**
 - Nel sistema di ricerca è attiva una sessione di ingestion di documenti
- **Scenario principale:**
 1. Companion chiede di avviare una sessione di ingestion
 2. Il sistema di ricerca avvia la sessione di ingestion
 3. Companion viene notificato della corretta apertura della sessione di ingestion

UC01.2: Ingestion liste entità

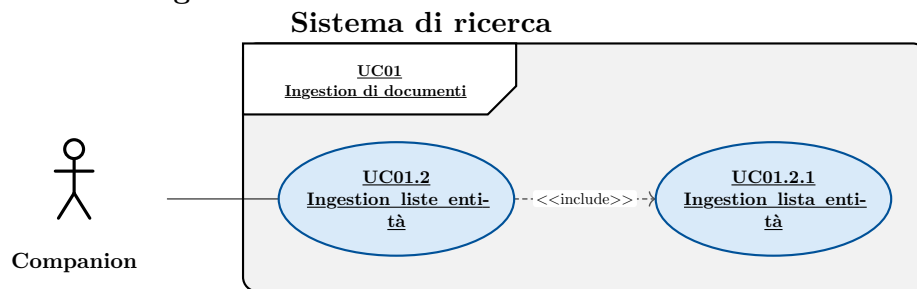


Figura 3: Diagramma del caso d'uso: Ingestion liste entità

- **Attore principale:** Companion
- **Precondizioni:**
 - Nel sistema di ricerca è attiva una sessione di ingestion
- **Postcondizioni:**
 - Il sistema di ricerca ha salvato le informazioni relative alle liste di entità
 - Il sistema di ricerca ha salvato gli embedding relativi alle liste di entità
 - Il sistema di ricerca ha salvato le informazioni di lingua relative alle liste di entità
- **Scenario principale:**
 1. Companion carica le liste di entità
 - 1.1. Companion carica una lista di entità → UC01.2.1
- **Inclusioni:**
 - UC01.2.1

3 Analisi dei requisiti

UC01.2.1: Ingestion lista entità

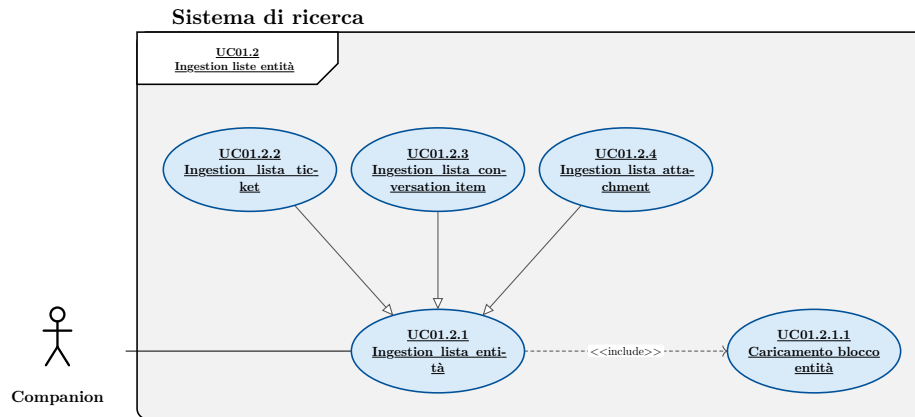


Figura 4: Diagramma del caso d'uso: Ingestion lista entità

- **Attore principale:** Companion
- **Precondizioni:**
 - Nel sistema di ricerca è attiva una sessione di ingestion
- **Postcondizioni:**
 - Il sistema di ricerca ha salvato le informazioni relative alla lista di entità
 - Il sistema di ricerca ha salvato gli embedding relativi alla lista di entità
 - Il sistema di ricerca ha salvato le informazioni di lingua relative alla lista di entità
- **Scenario principale:**
 1. Companion carica la lista di entità
 - 1.1. Companion carica un blocco di entità → UC01.2.1.1
- **Inclusioni:**
 - UC01.2.1.1
- **Specializzazioni:**
 - UC01.2.2
 - UC01.2.3
 - UC01.2.4

UC01.2.1.1: Caricamento blocco entità

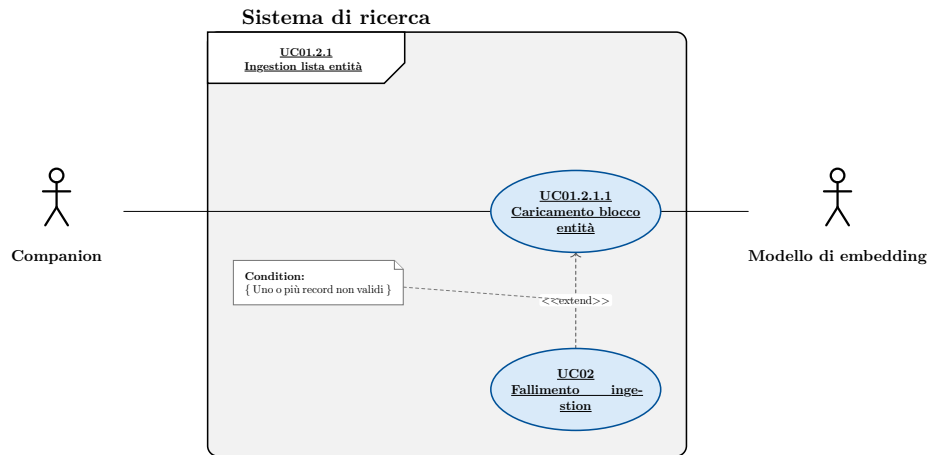


Figura 5: Diagramma del caso d'uso: Caricamento blocco entità

- **Attore principale:** Companion
- **Attore secondario:** Modello di embedding
- **Precondizioni:**
 - Il processo di caricamento lista di entità è in corso
- **Postcondizioni:**
 - Il sistema di ricerca ha salvato le informazioni relative al blocco di entità
 - Il sistema di ricerca ha salvato gli embedding relativi al blocco di entità
 - Il sistema di ricerca ha salvato le informazioni di lingua relative al blocco di entità
- **Scenario principale:**
 1. Companion carica un blocco di entità
 2. Il sistema di ricerca salva i dati delle entità
 3. Il sistema di ricerca calcola l'embedding da associare ai chunk di testo usando il modello di embedding
 4. Il sistema di ricerca rileva le informazioni di lingua da applicare ai chunk
 - 4.1. Il sistema di ricerca può usare l'informazione linguistica presente nei dati caricati
 - 4.2. Il sistema di ricerca può rilevare la lingua del testo
 5. Il sistema di ricerca associa gli embedding al relativo chunk di testo
 6. Il sistema di ricerca associa la lingua al relativo chunk di testo
 7. Il sistema di ricerca salva il blocco di entità
- **Scenari alternativi:**
 - Uno o più record del blocco non sono validi → **UC02**
- **Estensioni:**
 - **UC02**

3 Analisi dei requisiti

UC01.2.2: Ingestion lista ticket

- **Attore principale:** Companion
- **Precondizioni:**
 - Nel sistema di ricerca è attiva una sessione di ingestion
- **Postcondizioni:**
 - Il sistema di ricerca ha salvato le informazioni relative alla lista di ticket
 - Il sistema di ricerca ha salvato gli embedding relativi alla lista di ticket
 - Il sistema di ricerca ha salvato le informazioni di lingua relative alla lista di ticket
- **Scenario principale:**
 1. Companion carica la lista dei ticket

UC01.2.3: Ingestion lista conversation item

- **Attore principale:** Companion
- **Precondizioni:**
 - Nel sistema di ricerca è attiva una sessione di ingestion
- **Postcondizioni:**
 - Il sistema di ricerca ha salvato le informazioni relative alla lista di conversation item
 - Il sistema di ricerca ha salvato gli embedding relativi alla lista di conversation item
 - Il sistema di ricerca ha salvato le informazioni di lingua relative alla lista di conversation item
- **Scenario principale:**
 1. Companion carica la lista dei conversation item

UC01.2.4: Ingestion lista attachment

- **Attore principale:** Companion
- **Precondizioni:**
 - Nel sistema di ricerca è attiva una sessione di ingestion
- **Postcondizioni:**
 - Il sistema di ricerca ha salvato le informazioni relative alla lista di attachment
 - Il sistema di ricerca ha salvato gli embedding relativi alla lista di attachment
 - Il sistema di ricerca ha salvato le informazioni di lingua relative alla lista di attachment
- **Scenario principale:**
 1. Companion carica la lista degli attachment

3 Analisi dei requisiti

UC01.3: Termina ingestion

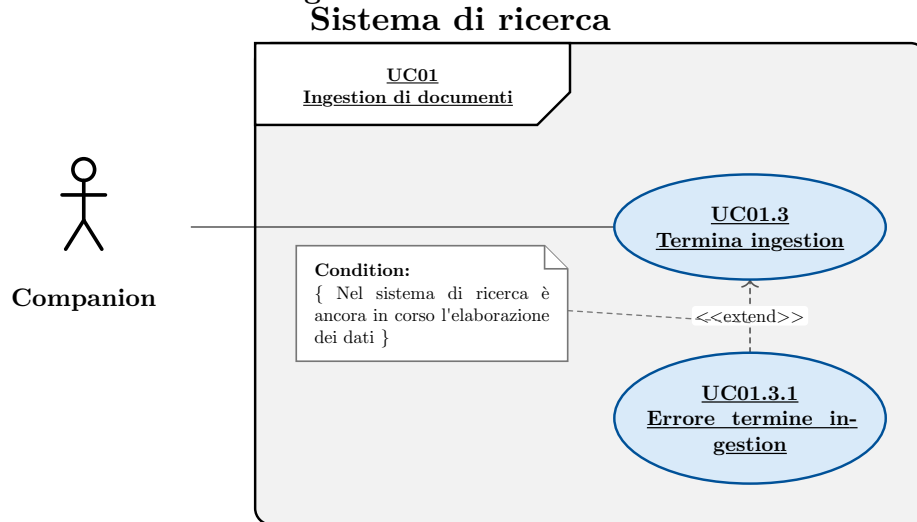


Figura 6: Diagramma del caso d'uso: Termina ingestion

- **Attore principale:** Companion
- **Precondizioni:**
 - Nel sistema di ricerca è attiva una sessione di ingestion
 - Nel sistema di ricerca i dati inseriti sono stati resi disponibili per la ricerca
- **Postcondizioni:**
 - Nel sistema di ricerca viene chiusa la sessione di ingestion
- **Scenario principale:**
 1. Companion chiede il termine della sessione di ingestion
 2. Il sistema di ricerca rende i dati disponibili per la ricerca
 3. Il sistema di ricerca chiude la sessione di ingestion
- **Scenari alternativi:**
 - Nel sistema di ricerca è ancora in corso l'elaborazione dei dati
→ UC01.3.1
- **Estensioni:**
 - UC01.3.1

3 Analisi dei requisiti

UC01.3.1: Errore termine ingestion

- **Attore principale:** Companion
- **Precondizioni:**
 - Nel sistema di ricerca è attiva una sessione di ingestion
 - Nel sistema di ricerca i dati inseriti sono stati resi disponibili per la ricerca
- **Postcondizioni:**
 - Nel sistema di ricerca rimane attiva la sessione di ingestion
 - Companion viene notificato dell'errore
- **Scenario principale:**
 1. Companion chiede il termine della sessione di ingestion
 2. Il sistema di ricerca rileva che è ancora in corso l'elaborazione dei dati
 3. Companion riceve una notifica dell'errore

UC02: Fallimento ingestion

- **Attore principale:** Companion
- **Precondizioni:**
 - Il processo di caricamento lista di entità è in corso
- **Postcondizioni:**
 - Il sistema di ricerca non ha salvato i record non validi
 - Companion riceve un messaggio di errore esplicativo
- **Scenario principale:**
 1. Companion carica un blocco di entità
 2. Il sistema di ricerca rileva degli errori relativi a uno o più record del blocco
 3. Companion riceve un errore esplicativo relativo ai record che hanno generato errori

3 Analisi dei requisiti

UC03: Ricerca su singola entità

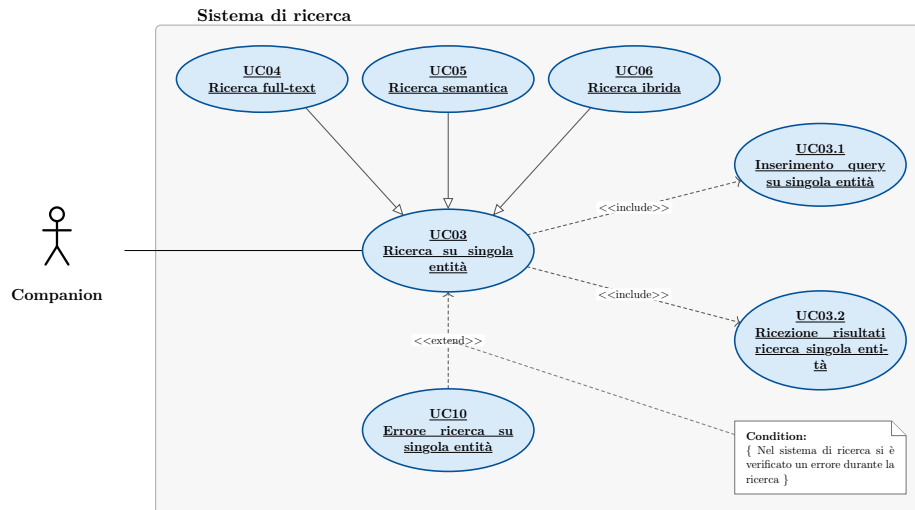


Figura 7: Diagramma del caso d'uso: Ricerca su singola entità

- **Attore principale:** Companion
- **Precondizioni:**
 - Il sistema di ricerca è attivo
- **Postcondizioni:**
 - Companion ha ricevuto i risultati della ricerca
- **Scenario principale:**
 1. Companion inserisce una query → UC03.1
 2. Il sistema di ricerca valida la query
 3. Il sistema di ricerca esegue la query
 4. Companion riceve i risultati → UC03.2
- **Scenari alternativi:**
 - Errore durante la ricerca → UC10
- **Inclusioni:**
 - UC03.1
 - UC03.2
- **Estensioni:**
 - UC10
- **Specializzazioni:**
 - UC04
 - UC05
 - UC06
- **Trigger:** Companion vuole eseguire una ricerca su una singola entità

3 Analisi dei requisiti

UC03.1: Inserimento query su singola entità Sistema di ricerca

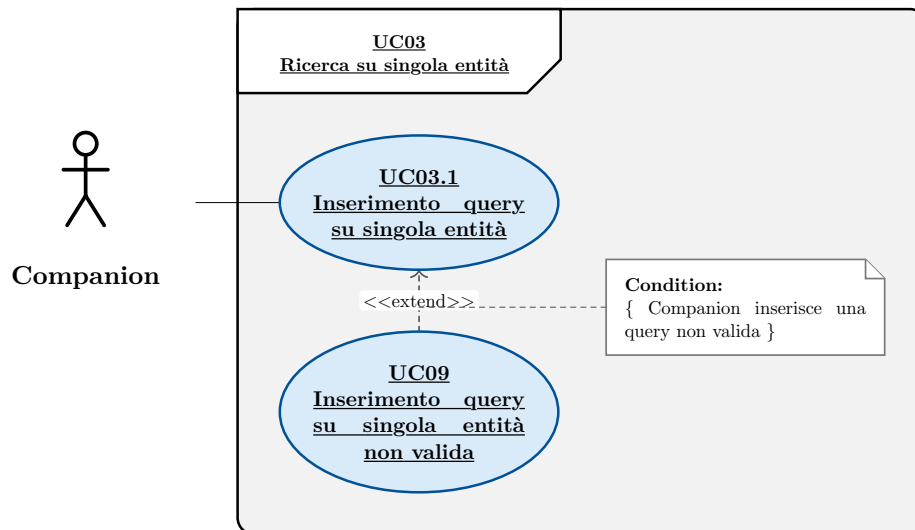


Figura 8: Diagramma del caso d'uso: Inserimento query su singola entità

- **Attore principale:** Companion
- **Precondizioni:**
 - Nel sistema di ricerca è in corso una ricerca su una singola entità
- **Postcondizioni:**
 - Il sistema di ricerca ha elaborato la query
- **Scenario principale:**
 1. Companion inserisce la query di ricerca
- **Scenari alternativi:**
 - Errore query non valida → **UC09**
- **Estensioni:**
 - **UC09**

UC03.2: Ricezione risultati ricerca singola entità

- **Attore principale:** Companion
- **Precondizioni:**
 - Nel sistema di ricerca è in corso una ricerca su singola entità
 - Il sistema di ricerca ha eseguito la query
- **Postcondizioni:**
 - Companion ha ricevuto i risultati della ricerca
- **Scenario principale:**
 1. Il sistema di ricerca ritorna la lista dei risultati prodotti dalla ricerca
 2. Companion riceve i risultati della ricerca

3 Analisi dei requisiti

UC04: Ricerca full-text

- **Attore principale:** Companion
- **Precondizioni:**
 - Il sistema di ricerca è attivo
- **Postcondizioni:**
 - Companion ha ricevuto i risultati della ricerca
- **Scenario principale:**
 1. Companion inserisce una query → **UC03.1**
 2. Il sistema di ricerca esegue la query valutando la pertinenza in base alla ricerca full-text
 3. Il sistema di ricerca può ottimizzare la ricerca basandosi sulla lingua
 4. Companion riceve i risultati → **UC03.2**
- **Trigger:** Companion vuole eseguire una ricerca full-text su una singola entità

UC05: Ricerca semantica

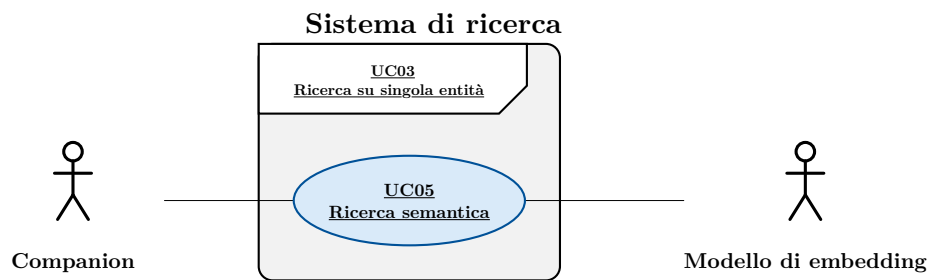


Figura 9: Diagramma del caso d'uso: Ricerca semantica

- **Attore principale:** Companion
- **Attore secondario:** Modello di embedding
- **Precondizioni:**
 - Il sistema di ricerca è attivo
- **Postcondizioni:**
 - Companion ha ricevuto i risultati della ricerca
- **Scenario principale:**
 1. Companion inserisce una query → **UC03.1**
 2. Il sistema di ricerca utilizza il modello di embedding per calcolare il vettore di embedding da usare durante la ricerca
 3. Il sistema di ricerca esegue la query valutando la pertinenza in base alla ricerca semantica
 4. Companion riceve i risultati → **UC03.2**
- **Trigger:** Companion vuole eseguire una ricerca semantica su una singola entità

3 Analisi dei requisiti

UC06: Ricerca ibrida

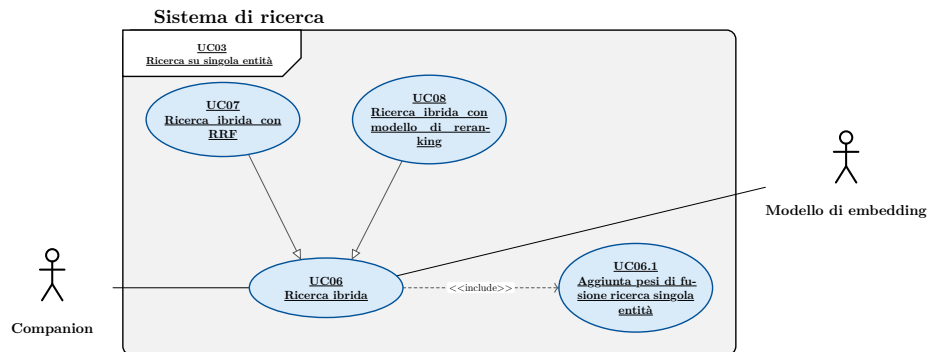


Figura 10: Diagramma del caso d'uso: Ricerca ibrida

- **Attore principale:** Companion
- **Attore secondario:** Modello di embedding
- **Precondizioni:**
 - Il sistema di ricerca è attivo
- **Postcondizioni:**
 - Companion ha ricevuto i risultati della ricerca
- **Scenario principale:**
 1. Companion inserisce una query → **UC03.1**
 2. Companion può inserire i pesi da utilizzare durante la fusione di ricerca full-text e semantica → **UC06.1**
 3. Il sistema di ricerca utilizza il modello di embedding per calcolare il vettore di embedding da usare durante la ricerca
 4. Il sistema di ricerca esegue la query valutando la pertinenza in base alla ricerca semantica
 5. Il sistema di ricerca esegue la query valutando la pertinenza in base alla ricerca full-text
 6. Il sistema di ricerca fonde i risultati
 7. Companion riceve i risultati → **UC03.2**
- **Inclusioni:**
 - **UC06.1**
- **Specializzazioni:**
 - **UC07**
 - **UC08**
- **Trigger:** Companion vuole eseguire una ricerca ibrida su una singola entità

UC06.1: Aggiunta pesi di fusione ricerca singola entità

- **Attore principale:** Companion
- **Precondizioni:**
 - Companion sta eseguendo una ricerca ibrida su singola entità
- **Postcondizioni:**
 - Companion ha inserito i pesi da usare durante la fusione dei risultati di ricerca
- **Scenario principale:**
 1. Companion inserisce il peso da applicare alla ricerca full-text
 2. Companion inserisce il peso da applicare alla ricerca semantica

UC07: Ricerca ibrida con RRF

- **Attore principale:** Companion
- **Precondizioni:**
 - Il sistema di ricerca è attivo
- **Postcondizioni:**
 - Companion ha ricevuto i risultati della ricerca
- **Scenario principale:**
 1. Companion inserisce una query → **UC03.1**
 2. Companion può inserire i pesi da utilizzare durante la fusione di ricerca full-text e semantica → **UC06.1**
 3. Il sistema di ricerca utilizza il modello di embedding per calcolare il vettore di embedding da usare durante la ricerca
 4. Il sistema di ricerca esegue la query valutando la pertinenza in base alla ricerca semantica
 5. Il sistema di ricerca esegue la query valutando la pertinenza in base alla ricerca full-text
 6. Il sistema di ricerca fonde i risultati tramite RRF
 7. Companion riceve i risultati → **UC03.2**
- **Trigger:** Companion vuole eseguire una ricerca ibrida su una singola entità

UC08: Ricerca ibrida con modello di reranking

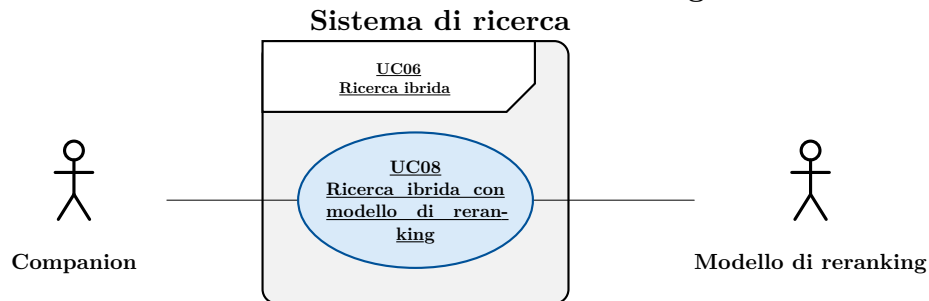


Figura 11: Diagramma del caso d'uso: Ricerca ibrida con modello di reranking

- **Attore principale:** Companion
- **Attore secondario:** Modello di reranking
- **Precondizioni:**
 - Il sistema di ricerca è attivo
- **Postcondizioni:**
 - Companion ha ricevuto i risultati della ricerca
- **Scenario principale:**
 1. Companion inserisce una query → **UC03.1**
 2. Companion può inserire i pesi da utilizzare durante la fusione di ricerca full-text e semantica → **UC06.1**
 3. Il sistema di ricerca utilizza il modello di embedding per calcolare il vettore di embedding da usare durante la ricerca
 4. Il sistema di ricerca esegue la query valutando la pertinenza in base alla ricerca semantica
 5. Il sistema di ricerca esegue la query valutando la pertinenza in base alla ricerca full-text
 6. Il sistema di ricerca fonde i risultati affidandosi a un modello di reranking esterno
 7. Companion riceve i risultati → **UC03.2**
- **Trigger:** Companion vuole eseguire una ricerca ibrida su una singola entità

3 Analisi dei requisiti

UC09: Inserimento query su singola entità non valida

- **Attore principale:** Companion
- **Precondizioni:**
 - Nel sistema di ricerca è in corso una ricerca su una singola entità
 - Companion ha inserito una query non valida
- **Postcondizioni:**
 - Companion riceve notifica esplicativa dell'errore
- **Scenario principale:**
 1. Il sistema di ricerca rileva la non validità della query
 2. Il sistema di ricerca interrompe la ricerca
 3. Companion riceve un messaggio d'errore esplicativo

UC10: Errore ricerca su singola entità

- **Attore principale:** Companion
- **Precondizioni:**
 - Nel sistema di ricerca è in corso una ricerca su una singola entità
 - Nel sistema di ricerca si è verificato un errore durante la ricerca
- **Postcondizioni:**
 - Companion riceve notifica esplicativa dell'errore
- **Scenario principale:**
 1. Il sistema di ricerca interrompe la ricerca
 2. Companion riceve un messaggio d'errore esplicativo

3 Analisi dei requisiti

UC11: Ricerca linked

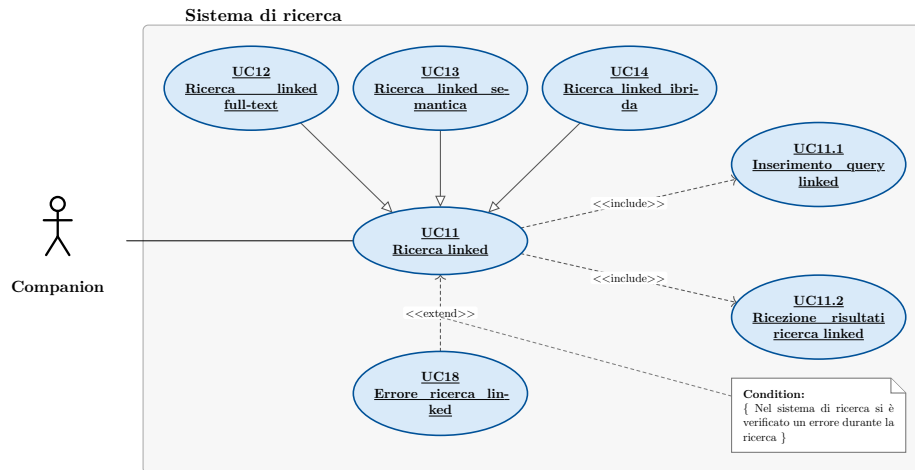


Figura 12: Diagramma del caso d'uso: Ricerca linked

- **Attore principale:** Companion
- **Precondizioni:**
 - Il sistema di ricerca è attivo
- **Postcondizioni:**
 - Companion ha ricevuto i risultati della ricerca
- **Scenario principale:**
 1. Companion inserisce una query → UC11.1
 2. Il sistema di ricerca valida la query
 3. Il sistema di ricerca esegue la query
 4. Companion riceve i risultati → UC11.2
- **Scenari alternativi:**
 - Errore durante la ricerca → UC18
- **Inclusioni:**
 - UC11.1
 - UC11.2
- **Estensioni:**
 - UC18
- **Specializzazioni:**
 - UC12
 - UC13
 - UC14
- **Trigger:** Companion vuole eseguire una ricerca su tutte le entità

3 Analisi dei requisiti

UC11.1: Inserimento query linked Sistema di ricerca

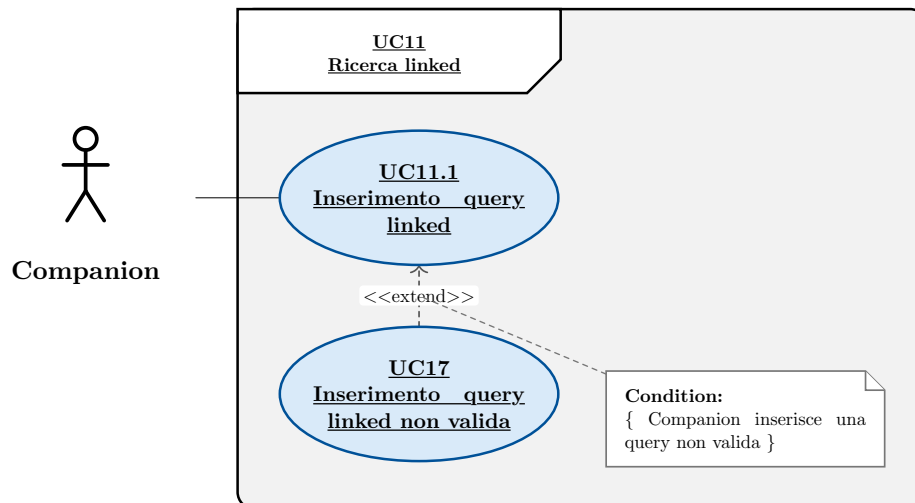


Figura 13: Diagramma del caso d'uso: Inserimento query linked

- **Attore principale:** Companion
- **Precondizioni:**
 - Nel sistema di ricerca è in corso una ricerca su tutte le entità
- **Postcondizioni:**
 - Il sistema di ricerca ha elaborato la query
- **Scenario principale:**
 1. Companion inserisce la query di ricerca
- **Scenari alternativi:**
 - Errore query non valida → UC17
- **Estensioni:**
 - UC17

UC11.2: Ricezione risultati ricerca linked

- **Attore principale:** Companion
- **Precondizioni:**
 - Il sistema di ricerca ha eseguito la query
- **Postcondizioni:**
 - Companion ha ricevuto i risultati della ricerca
- **Scenario principale:**
 1. Il sistema di ricerca ritorna la lista dei risultati prodotti dalla ricerca
 2. Companion riceve i risultati della ricerca

3 Analisi dei requisiti

UC12: Ricerca linked full-text

- **Attore principale:** Companion
- **Precondizioni:**
 - Il sistema di ricerca è attivo
- **Postcondizioni:**
 - Companion ha ricevuto i risultati della ricerca
- **Scenario principale:**
 1. Companion inserisce una query → UC11.1
 2. Il sistema di ricerca esegue la query valutando la pertinenza in base alla ricerca full-text
 3. Il sistema di ricerca può ottimizzare la ricerca basandosi sulla lingua
 4. Companion riceve i risultati → UC11.2
- **Trigger:** Companion vuole eseguire una ricerca full-text su tutte le entità

UC13: Ricerca linked semantica

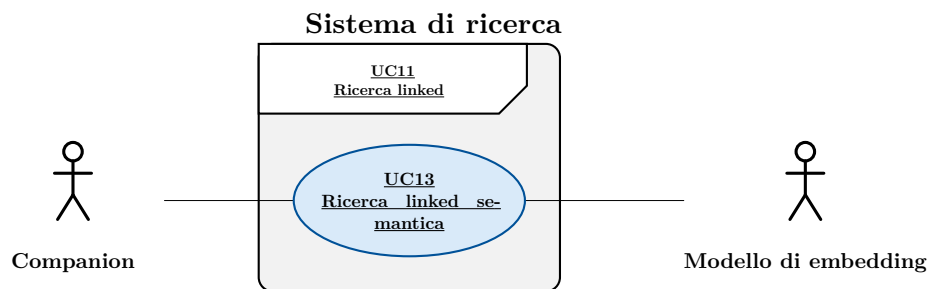


Figura 14: Diagramma del caso d'uso: Ricerca linked semantica

- **Attore principale:** Companion
- **Attore secondario:** Modello di embedding
- **Precondizioni:**
 - Il sistema di ricerca è attivo
- **Postcondizioni:**
 - Companion ha ricevuto i risultati della ricerca
- **Scenario principale:**
 1. Companion inserisce una query → UC11.1
 2. Il sistema di ricerca utilizza il modello di embedding per calcolare il vettore di embedding da usare durante la ricerca
 3. Il sistema di ricerca esegue la query valutando la pertinenza in base alla ricerca semantica
 4. Companion riceve i risultati → UC11.2
- **Trigger:** Companion vuole eseguire una ricerca semantica su tutte le entità

3 Analisi dei requisiti

UC14: Ricerca linked ibrida

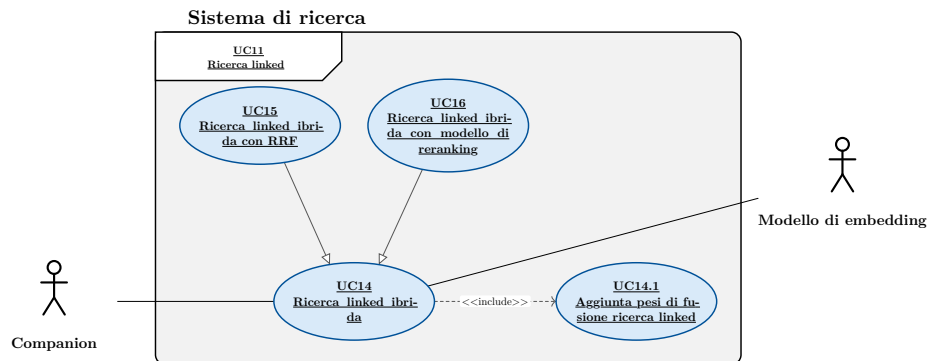


Figura 15: Diagramma del caso d'uso: Ricerca linked ibrida

- **Attore principale:** Companion
- **Attore secondario:** Modello di embedding
- **Precondizioni:**
 - Il sistema di ricerca è attivo
- **Postcondizioni:**
 - Companion ha ricevuto i risultati della ricerca
- **Scenario principale:**
 1. Companion inserisce una query → **UC11.1**
 2. Companion inserisce i pesi da usare durante la fusione dei risultati di ricerca semantica e full-text → **UC14.1**
 3. Il sistema di ricerca utilizza il modello di embedding per calcolare il vettore di embedding da usare durante la ricerca
 4. Il sistema di ricerca esegue la query valutando la pertinenza in base alla ricerca semantica
 5. Il sistema di ricerca esegue la query valutando la pertinenza in base alla ricerca full-text
 6. Il sistema di ricerca fonde i risultati
 7. Companion riceve i risultati → **UC11.2**
- **Inclusioni:**
 - **UC14.1**
- **Trigger:** Companion vuole eseguire una ricerca ibrida su tutte le entità

UC14.1: Aggiunta pesi di fusione ricerca linked

- **Attore principale:** Companion
- **Precondizioni:**
 - Companion sta eseguendo una ricerca ibrida in modalità linked
- **Postcondizioni:**
 - Companion ha inserito i pesi da usare durante la fusione dei risultati di ricerca
- **Scenario principale:**
 1. Companion inserisce il peso da applicare alla ricerca full-text
 2. Companion inserisce il peso da applicare alla ricerca semantica

UC15: Ricerca linked ibrida con RRF

- **Attore principale:** Companion
- **Precondizioni:**
 - Il sistema di ricerca è attivo
- **Postcondizioni:**
 - Companion ha ricevuto i risultati della ricerca
- **Scenario principale:**
 1. Companion inserisce una query → UC11.1
 2. Companion inserisce i pesi da usare durante la fusione dei risultati di ricerca semantica e full-text → UC14.1
 3. Il sistema di ricerca utilizza il modello di embedding per calcolare il vettore di embedding da usare durante la ricerca
 4. Il sistema di ricerca esegue la query valutando la pertinenza in base alla ricerca semantica
 5. Il sistema di ricerca esegue la query valutando la pertinenza in base alla ricerca full-text
 6. Il sistema di ricerca fonde i risultati tramite RRF
 7. Companion riceve i risultati → UC11.2
- **Trigger:** Companion vuole eseguire una ricerca ibrida su tutte le entità usando una combinazione dei risultati basata su RRF

UC16: Ricerca linked ibrida con modello di reranking

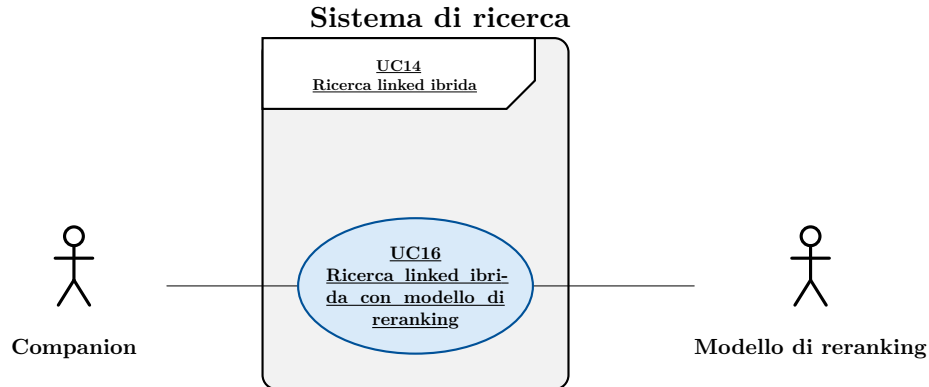


Figura 16: Diagramma del caso d'uso: Ricerca linked ibrida con modello di reranking

- **Attore principale:** Companion
- **Attore secondario:** Modello di reranking
- **Precondizioni:**
 - Il sistema di ricerca è attivo
- **Postcondizioni:**
 - Companion ha ricevuto i risultati della ricerca
- **Scenario principale:**
 1. Companion inserisce una query → **UC11.1**
 2. Companion inserisce i pesi da usare durante la fusione dei risultati di ricerca semantica e full-text → **UC14.1**
 3. Il sistema di ricerca utilizza il modello di embedding per calcolare il vettore di embedding da usare durante la ricerca
 4. Il sistema di ricerca esegue la query valutando la pertinenza in base alla ricerca semantica
 5. Il sistema di ricerca esegue la query valutando la pertinenza in base alla ricerca full-text
 6. Il sistema di ricerca fonde i risultati tramite un modello di reranking
 7. Companion riceve i risultati → **UC11.2**
- **Trigger:** Companion vuole eseguire una ricerca ibrida su tutte le entità usando una combinazione dei risultati basata su modello di reranking

3 Analisi dei requisiti

UC17: Inserimento query linked non valida

- **Attore principale:** Companion
- **Precondizioni:**
 - Nel sistema di ricerca è in corso una ricerca su tutte le entità
 - Companion ha inserito una query non valida
- **Postcondizioni:**
 - Companion riceve notifica esplicativa dell'errore
- **Scenario principale:**
 1. Il sistema di ricerca rileva la non validità della query
 2. Il sistema di ricerca interrompe la ricerca
 3. Companion riceve un messaggio d'errore esplicativo

UC18: Errore ricerca linked

- **Attore principale:** Companion
- **Precondizioni:**
 - Nel sistema di ricerca è in corso una ricerca su tutte le entità
 - Nel sistema di ricerca si è verificato un errore durante la ricerca
- **Postcondizioni:**
 - Companion riceve notifica esplicativa dell'errore
- **Scenario principale:**
 1. Il sistema di ricerca interrompe la ricerca
 2. Companion riceve un messaggio d'errore esplicativo

3 Analisi dei requisiti

UC19: Avvia test

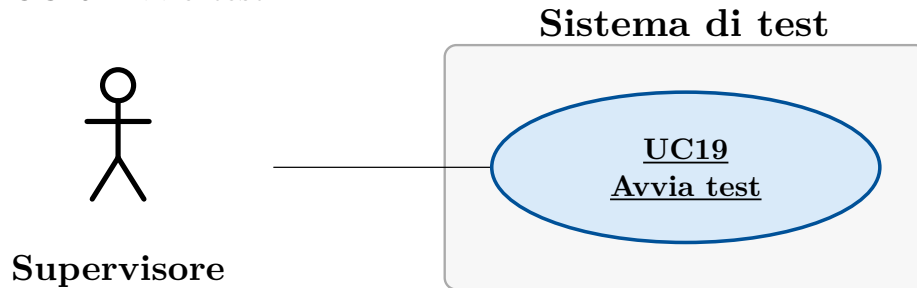


Figura 17: Diagramma del caso d'uso: Avvia test

- **Attore principale:** Supervisore
- **Precondizioni:**
 - Il sistema di test è attivo
 - Il sistema di ricerca è attivo
 - Nel sistema di test non ci sono altre run di test attive
- **Postcondizioni:**
 - Il sistema di test ha avviato l'esecuzione delle ricerche di test
- **Scenario principale:**
 1. Il supervisore seleziona l'avvio dei test
 2. Il sistema di test avvia i test
- **Trigger:** Il supervisore vuole avviare un test di performance

3 Analisi dei requisiti

UC20: Visualizza dashboard performance

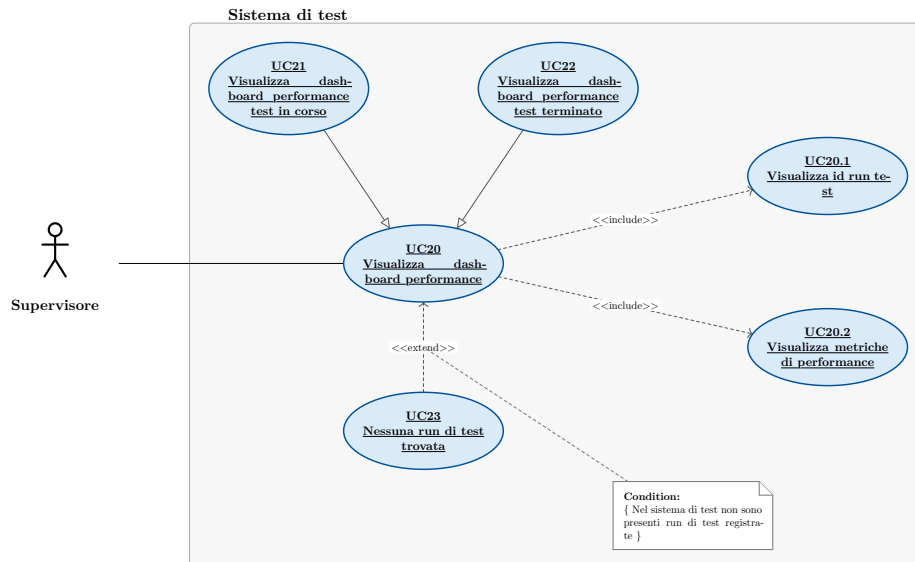


Figura 18: Diagramma del caso d'uso: Visualizza dashboard performance

- **Attore principale:** Supervisore
- **Precondizioni:**
 - Il sistema di test è attivo
- **Postcondizioni:**
 - Il supervisore ha visualizzato lo stato dell'ultima run di test avviata
- **Scenario principale:**
 1. Il supervisore visualizza l'id della run di test → UC20.1
 2. Il supervisore visualizza la lista delle metriche → UC20.2
- **Scenari alternativi:**
 - Nel sistema di test non vi sono run di test registrate → UC23
- **Inclusioni:**
 - UC20.1
 - UC20.2
- **Estensioni:**
 - UC23
- **Specializzazioni:**
 - UC21
 - UC22

3 Analisi dei requisiti

UC20.1: Visualizza id run test

- **Attore principale:** Supervisore
- **Precondizioni:**
 - Il supervisore sta visualizzando la dashboard delle metriche
 - Nel sistema di test sono presenti i dati relativi all'ultimo test svolto o in corso
- **Postcondizioni:**
 - Il supervisore ha visualizzato l'id relativo alla run di test a cui si riferiscono le performance
- **Scenario principale:**
 1. Il supervisore visualizza l'id relativo alla run di test a cui si riferiscono le performance

3 Analisi dei requisiti

UC20.2: Visualizza metriche di performance

Sistema di ricerca

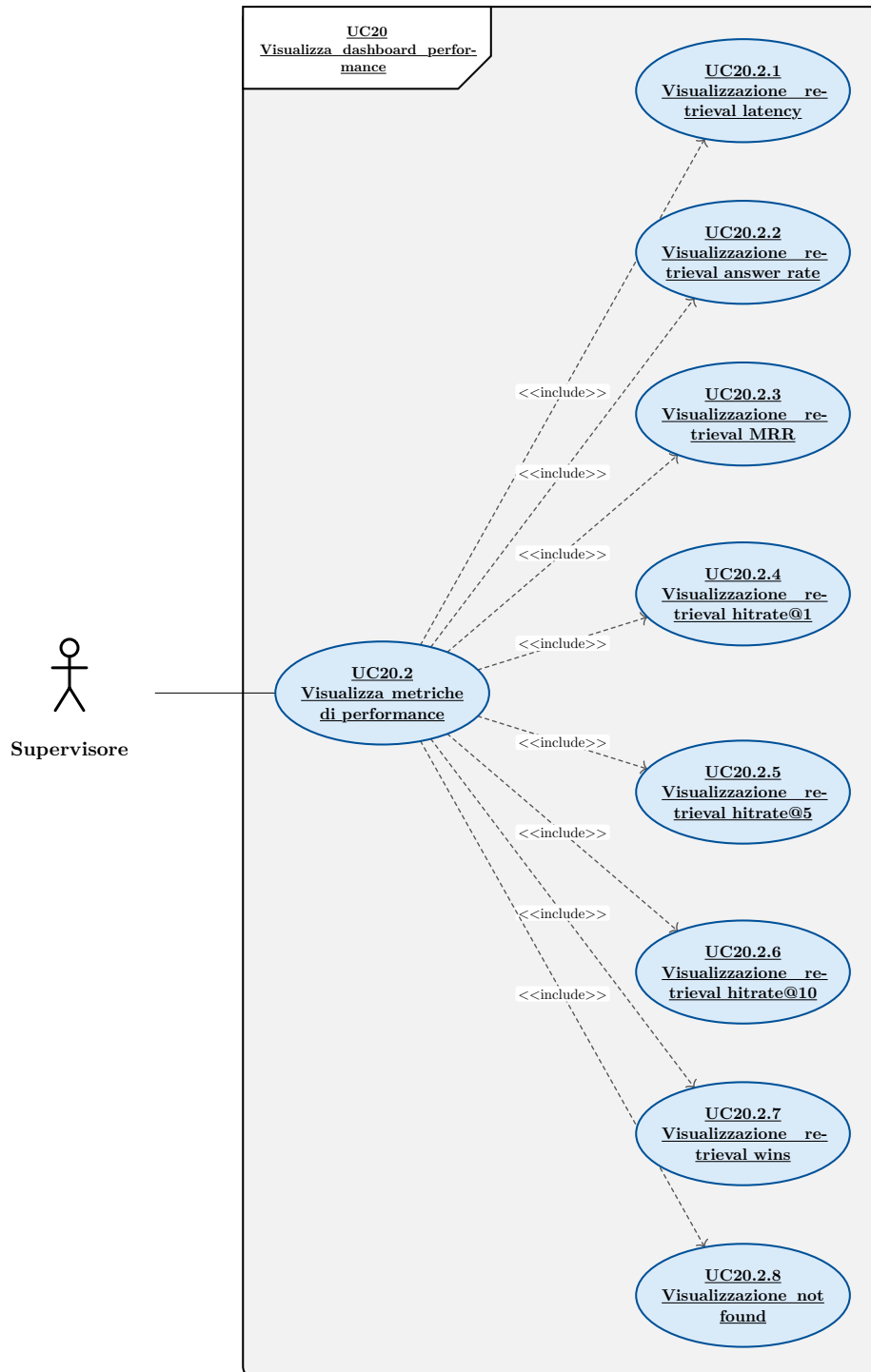


Figura 19: Diagramma del caso d'uso: Visualizza metriche di performance

3 Analisi dei requisiti

- **Attore principale:** Supervisore
- **Precondizioni:**
 - Il supervisore sta visualizzando la dashboard delle metriche
- **Postcondizioni:**
 - Il supervisore ha visualizzato la lista delle metriche
- **Scenario principale:**
 1. Il sistema di test calcola le metriche
 2. Il supervisore visualizza l'elenco delle metriche
 3. Il supervisore visualizza la *retrieval latency_G* → UC20.2.1
 4. Il supervisore visualizza la *retrieval answer rate_G* → UC20.2.2
 5. Il supervisore visualizza la retrieval *MRR_G* → UC20.2.3
 6. Il supervisore visualizza il retrieval *hitrate@1_G* → UC20.2.4
 7. Il supervisore visualizza il retrieval hitrate@5 → UC20.2.5
 8. Il supervisore visualizza il retrieval hitrate@10 → UC20.2.6
 9. Il supervisore visualizza il *retrieval wins_G* → UC20.2.7
 10. Il supervisore visualizza il *not found_G* → UC20.2.8
- **Trigger:** Il supervisore vuole visualizzare le metriche di performance del sistema di ricerca

3 Analisi dei requisiti

UC20.2.1: Visualizzazione retrieval latency

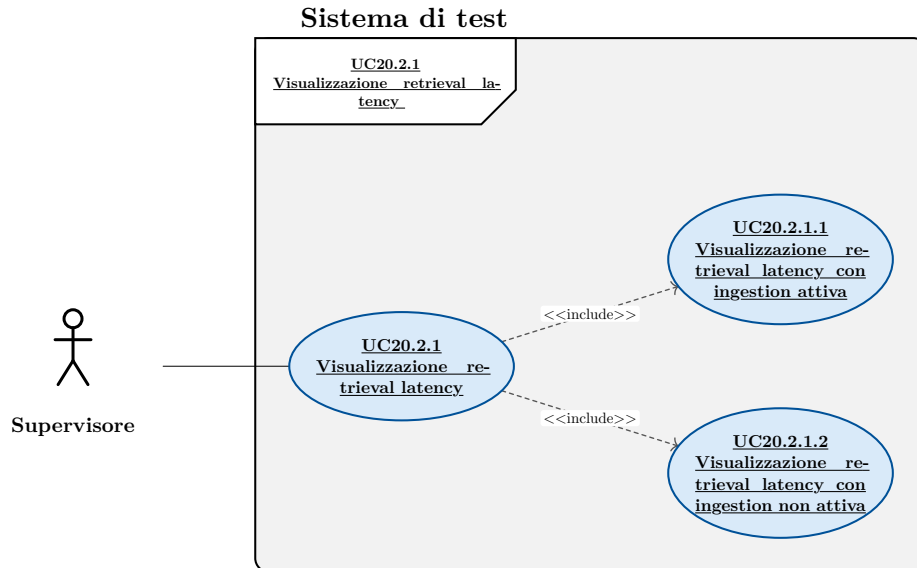


Figura 20: Diagramma del caso d'uso: Visualizzazione retrieval latency

- **Attore principale:** Supervisore
- **Precondizioni:**
 - Il supervisore sta visualizzando la lista delle metriche
 - Nel sistema di test sono presenti i dati relativi all'ultimo test svolto o in corso
- **Postcondizioni:**
 - Il supervisore ha visualizzato la retrieval latency media
- **Scenario principale:**
 1. Il supervisore visualizza la retrieval latency media calcolata durante la presenza di un processo di ingestion → **UC20.2.1.1**
 2. Il supervisore visualizza la retrieval latency calcolata media durante l'assenza di un processo di ingestion → **UC20.2.1.2**
- **Inclusioni:**
 - **UC20.2.1.1**
 - **UC20.2.1.2**

3 Analisi dei requisiti

UC20.2.1.1: Visualizzazione retrieval latency con ingestion attiva

- **Attore principale:** Supervisore
- **Precondizioni:**
 - Il supervisore sta visualizzando la lista delle metriche
 - Nel sistema di test sono presenti i dati relativi all'ultimo test svolto o in corso
- **Postcondizioni:**
 - Il supervisore ha visualizzato la retrieval latency media durante la presenza di un processo di ingestion
- **Scenario principale:**
 1. Il supervisore visualizza la retrieval latency media calcolata durante la presenza di un processo di ingestion

UC20.2.1.2: Visualizzazione retrieval latency con ingestion non attiva

- **Attore principale:** Supervisore
- **Precondizioni:**
 - Il supervisore sta visualizzando la lista delle metriche
 - Nel sistema di test sono presenti i dati relativi all'ultimo test svolto o in corso
- **Postcondizioni:**
 - Il supervisore ha visualizzato la retrieval latency media durante l'assenza di un processo di ingestion
- **Scenario principale:**
 1. Il supervisore visualizza la retrieval latency media calcolata durante l'assenza di un processo di ingestion

UC20.2.2: Visualizzazione retrieval answer rate

- **Attore principale:** Supervisore
- **Precondizioni:**
 - Il supervisore sta visualizzando la lista delle metriche
 - Nel sistema di test sono presenti i dati relativi all'ultimo test svolto o in corso
- **Postcondizioni:**
 - Il supervisore ha visualizzato il retrieval answer rate
- **Scenario principale:**
 1. Il supervisore visualizza il retrieval answer rate

UC20.2.3: Visualizzazione retrieval MRR

- **Attore principale:** Supervisore
- **Precondizioni:**
 - Il supervisore sta visualizzando la lista delle metriche
 - Nel sistema di test sono presenti i dati relativi all'ultimo test svolto o in corso
- **Postcondizioni:**
 - Il supervisore ha visualizzato il retrieval mean reciprocal rank
- **Scenario principale:**
 1. Il supervisore visualizza il retrieval mean reciprocal rank

UC20.2.4: Visualizzazione retrieval hitrate@1

- **Attore principale:** Supervisore
- **Precondizioni:**
 - Il supervisore sta visualizzando la lista delle metriche
 - Nel sistema di test sono presenti i dati relativi all'ultimo test svolto o in corso
- **Postcondizioni:**
 - Il supervisore ha visualizzato il retrieval hitrate@1
- **Scenario principale:**
 1. Il supervisore visualizza il retrieval hitrate@1

UC20.2.5: Visualizzazione retrieval hitrate@5

- **Attore principale:** Supervisore
- **Precondizioni:**
 - Il supervisore sta visualizzando la lista delle metriche
 - Nel sistema di test sono presenti i dati relativi all'ultimo test svolto o in corso
- **Postcondizioni:**
 - Il supervisore ha visualizzato il retrieval hitrate@5
- **Scenario principale:**
 1. Il supervisore visualizza il retrieval hitrate@5

UC20.2.6: Visualizzazione retrieval hitrate@10

- **Attore principale:** Supervisore
- **Precondizioni:**
 - Il supervisore sta visualizzando la lista delle metriche
 - Nel sistema di test sono presenti i dati relativi all'ultimo test svolto o in corso
- **Postcondizioni:**
 - Il supervisore ha visualizzato il retrieval hitrate@10
- **Scenario principale:**
 1. Il supervisore visualizza il retrieval hitrate@10

UC20.2.7: Visualizzazione retrieval wins

- **Attore principale:** Supervisore
- **Precondizioni:**
 - Il supervisore sta visualizzando la lista delle metriche
 - Nel sistema di test sono presenti i dati relativi all'ultimo test svolto o in corso
- **Postcondizioni:**
 - Il supervisore ha visualizzato il retrieval wins
- **Scenario principale:**
 1. Il supervisore visualizza il retrieval wins

UC20.2.8: Visualizzazione not found

- **Attore principale:** Supervisore
- **Precondizioni:**
 - Il supervisore sta visualizzando la lista delle metriche
 - Nel sistema di test sono presenti i dati relativi all'ultimo test svolto o in corso
- **Postcondizioni:**
 - Il supervisore ha visualizzato il retrieval not found
- **Scenario principale:**
 1. Il supervisore visualizza il retrieval not found

UC20.3: Visualizza metriche di consumo delle risorse

- **Attore principale:** Supervisore
- **Precondizioni:**
 - Il supervisore sta visualizzando la dashboard delle performance
- **Postcondizioni:**
 - Il supervisore ha visualizzato le metriche relative al consumo di risorse del database
- **Scenario principale:**
 1. Il supervisore visualizza le metriche relative al consumo di risorse del database

3 Analisi dei requisiti

UC21: Visualizza dashboard performance test in corso

- **Attore principale:** Supervisore
- **Precondizioni:**
 - Il sistema di test è attivo
 - Nel sistema di test è in corso una run di test
- **Postcondizioni:**
 - Il supervisore ha visualizzato lo stato in tempo reale dell'ultima run di test avviata
- **Scenario principale:**
 1. Il sistema di test mantiene aggiornata la dashboard
 2. Il supervisore visualizza l'id della run di test → UC20.1
 3. Il supervisore visualizza la lista delle metriche → UC20.2
- **Scenari alternativi:**
 - Nel sistema di test non vi sono run di test registrate → UC23

UC22: Visualizza dashboard performance test terminato

- **Attore principale:** Supervisore
- **Precondizioni:**
 - Il sistema di test è attivo
 - Nel sistema di test non sono in corso run di test
- **Postcondizioni:**
 - Il supervisore ha visualizzato lo stato dell'ultima run di test avviata
- **Scenario principale:**
 1. Il sistema di test calcola una singola volta i dati da mostrare
 2. Il supervisore visualizza l'id della run di test → UC20.1
 3. Il supervisore visualizza la lista delle metriche → UC20.2
- **Scenari alternativi:**
 - Nel sistema di test non vi sono run di test registrate → UC23

UC23: Nessuna run di test trovata

- **Attore principale:** Supervisore
- **Precondizioni:**
 - Il sistema di test è attivo
- **Postcondizioni:**
 - Il supervisore è stato informato dell'assenza di run di test
- **Scenario principale:**
 1. Il sistema di test non trova nessuna run di test da mostrare
 2. Il supervisore viene informato dell'assenza di run di test su cui calcolare le metriche

3.3 Tracciamento dei requisiti

Ad ogni requisito è associato un codice costruito in base alle sue caratteristiche:

$$\mathbf{R(F/Q/C)(M/D/O)}$$

F (*Functional*): definiscono una funzione di un sistema o dei suoi componenti;

Q (*Qualitative*): rappresentano come il sistema deve essere per soddisfare i requisiti dello stakeholder;

C (*Constraint*): rappresentano dei vincoli o dei limiti che il sistema deve rispettare;

M (*Mandatory*): irrinunciabili per qualcuno degli stakeholder;

D (*Desirable*): non strettamente necessari ma a valore aggiunto riconoscibile;

O (*Optional*): relativamente utili oppure contrattabili anche in fasi avanzate del progetto;

R (*Requirement*): requisito

In **Tabella 2**, **Tabella 3** e **Tabella 4** sono riassunti i requisiti e il loro tracciamento con gli use case delineati in fase di analisi.

Codice	Descrizione	Fonti
RFM01	Companion deve poter caricare documenti nel sistema di ricerca per renderli ricercabili.	<u>UC01</u>
RFM02	Companion deve poter esplicitamente avviare una sessione di ingestion dei documenti.	<u>UC01.1</u>
RFM03	Companion deve poter caricare liste di documenti, relativi a più tipi di entità, da rendere disponibili per la ricerca.	<u>UC01.2</u>
RFM04	Companion deve poter caricare una lista di documenti, relativi a un singolo tipo di entità, da rendere disponibili per la ricerca.	<u>UC01.2.1</u>

3 Analisi dei requisiti

Codice	Descrizione	Fonti
RFM05	Companion deve poter caricare un numero variabile di documenti in blocco.	<u>UC01.2.1.1</u>
RFM06	Companion deve poter caricare una lista di ticket da rendere disponibili per la ricerca.	<u>UC01.2.2</u>
RFM07	Companion deve poter caricare una lista di conversation item da rendere disponibili per la ricerca.	<u>UC01.2.3</u>
RFM08	Companion deve poter caricare una lista di attachment da rendere disponibili per la ricerca.	<u>UC01.2.4</u>
RFM09	Companion deve poter indicare esplicitamente la fine della sessione di ingestion dei documenti.	<u>UC01.3</u>
RFM10	Companion deve poter essere notificato di errori durante la terminazione della sessione di ingestion.	<u>UC01.3.1</u>
RFM11	Companion deve poter essere notificato del fallimento di uno o più record durante il processo di ingestion.	<u>UC02</u>
RFM12	Companion deve poter effettuare ricerche basate sulla similarità del testo su una singola entità.	<u>UC03</u>

3 Analisi dei requisiti

Codice	Descrizione	Fonti
RFM13	Companion deve poter inserire una query di ricerca valida. Una query valida è composta dalle seguenti informazioni obbligatorie: • nome dell'entità su cui eseguire la ricerca, • l'elenco dei campi da ritornare al termine della ricerca, • l'elenco dei campi su cui effettuare la ricerca per similarità, • il testo da usare per la ricerca, • il numero di risultati voluti. Una query valida è composta dalle seguenti informazioni opzionali: • il filtro, • l'elenco dei pesi da applicare ai campi usati nel calcolo della similarità, • l'informazione di lingua opzionale.	<u>UC03.1</u>
RFM14	Companion deve poter ricevere i risultati della ricerca. Il risultato della ricerca deve contenere le seguenti informazioni: • i valori dei campi richiesti tramite la query, • il nome del campo di testo su cui è stato trovato il match, • il testo su cui è stata rilevata la corrispondenza, • il numero del chunk (si vedano i requisiti di vincolo), • il punteggio di similarità.	<u>UC03.2</u>
RFM15	Companion deve poter effettuare ricerche full-text su una singola entità.	<u>UC04</u>
RFM16	Companion deve poter effettuare ricerche semantiche su una singola entità.	<u>UC05</u>

3 Analisi dei requisiti

Codice	Descrizione	Fonti
RFM17	Companion deve poter effettuare ricerche ibride, basate sia su ricerca full-text sia su ricerca semantica, su una singola entità.	<u>UC06</u>
RFM18	Companion deve poter inserire dei pesi da utilizzare durante la fusione dei risultati di ricerca da usare al posto dei pesi di default.	<u>UC06.1</u>
RFM19	Companion deve poter effettuare ricerche ibride, basate sia su ricerca full-text sia su ricerca semantica, su una singola entità. Per la combinazione dei risultati viene usato RRF.	<u>UC07</u>
RFM20	Companion deve poter ricevere una notifica di errore in caso di query non valida.	<u>UC09</u>
RFM21	Companion deve poter ricevere una notifica di errore in caso di errori durante la ricerca.	<u>UC10</u>
RFM22	Companion deve poter eseguire ricerche basate sulla similarità su tutte le entità e combinarne i risultati.	<u>UC11</u>

3 Analisi dei requisiti

Codice	Descrizione	Fonti
RFM23	Companion deve poter inserire una query di ricerca linked. La query deve contenere i seguenti dati: • elenco dei campi dati da ritornare indicati tramite nome dell'entità e nome del campo dati, • elenco dei campi dati su cui eseguire la ricerca semantica indicati tramite nome dell'entità e nome del campo dati, • il testo da utilizzare per la ricerca semantica, • il numero di risultati voluti. La query può contenere i seguenti dati: • il filtro da applicare durante la ricerca iniziale, precedente ai join, su ogni entità, • il filtro da applicare successivamente ai join, • i pesi da applicare ai singoli campi su cui calcolare la similarità, • l'informazione di lingua.	<u>UC11.1</u>
RFM24	Companion deve ricevere i risultati della ricerca linked. I risultati devono contenere le seguenti informazioni: • i valori dei campi richiesti tramite la query, • il nome dell'entità e il nome del campo di testo su cui è stato trovato il match, • il testo su cui è stata rilevata la corrispondenza, • il numero del chunk (si vedano i requisiti di vincolo), • il punteggio di similarità.	<u>UC11.2</u>

3 Analisi dei requisiti

Codice	Descrizione	Fonti
RFM25	Companion deve poter eseguire una ricerca full-text in modalità linked.	<u>UC12</u>
RFM26	Companion deve poter eseguire una ricerca semantica in modalità linked.	<u>UC13</u>
RFM27	Companion deve poter eseguire una ricerca ibrida in modalità linked.	<u>UC14</u>
RFM28	Companion deve poter inserire dei pesi per la fusione di ricerca linked semantica e linked full-text da usare al posto dei pesi di default.	<u>UC14.1</u>
RFM29	Companion deve poter effettuare una ricerca ibrida in modalità linked, combinando i risultati tramite RRF.	<u>UC15</u>
RFM30	Companion deve poter ricevere una notifica esplicativa in caso venga inserita una query di ricerca linked non valida.	<u>UC17</u>
RFM31	Companion deve poter ricevere un messaggio di errore esplicativo in caso di fallimenti nella ricerca linked.	<u>UC18</u>
RFM32	Il supervisore deve poter avviare i test di performance della ricerca.	<u>UC19</u>
RFM33	Il supervisore deve poter visualizzare la dashboard riepilogativa delle performance della ricerca.	<u>UC20</u>
RFM34	Il supervisore deve poter visualizzare a quale run di test appartengono le metriche presenti sulla dashboard.	<u>UC20.1</u>
RFM35	Il supervisore deve poter visualizzare le metriche di performance relative a qualità e prestazioni della ricerca.	<u>UC20.2</u>

3 Analisi dei requisiti

Codice	Descrizione	Fonti
RFM36	Il supervisore deve poter visualizzare il tempo di risposta medio a una query di ricerca.	<u>UC20.2.1</u>
RFM37	Il supervisore deve poter visualizzare il tempo di risposta medio a una query di ricerca quando è attivo il processo di ingestion dei dati.	<u>UC20.2.1.1</u>
RFM38	Il supervisore deve poter visualizzare il tempo di risposta medio a una query di ricerca quando non è attivo il processo di ingestion dei dati.	<u>UC20.2.1.2</u>
RFM39	Il supervisore deve poter visualizzare quanto spesso il risultato atteso è presente tra i risultati reali della ricerca, indipendentemente dalla posizione.	<u>UC20.2.2</u>
RFM40	Il supervisore deve poter visualizzare il mean reciprocal rank medio delle ricerche.	<u>UC20.2.3</u>
RFM41	Il supervisore deve poter visualizzare quante volte il risultato atteso è presente come primo risultato della ricerca.	<u>UC20.2.4</u>
RFM42	Il supervisore deve poter visualizzare quante volte il risultato atteso è presente tra i primi cinque risultati della ricerca.	<u>UC20.2.5</u>
RFM43	Il supervisore deve poter visualizzare quante volte il risultato atteso è presente tra i primi dieci risultati della ricerca.	<u>UC20.2.6</u>

3 Analisi dei requisiti

Codice	Descrizione	Fonti
RFM44	Il supervisore deve poter visualizzare quante volte la ricerca ha prodotto risultati, indipendentemente dalla loro correttezza.	<u>UC20.2.7</u>
RFM45	Il supervisore deve poter visualizzare quante volte la ricerca non ha prodotto risultati.	<u>UC20.2.8</u>
RFM46	Il supervisore deve poter visualizzare le performance della run di test attualmente in corso.	<u>UC21</u>
RFM47	Il supervisore deve poter visualizzare le performance dell'ultima run di test eseguita.	<u>UC22</u>
RFM48	Il supervisore deve poter ricevere una notifica esplicativa in caso di mancanza di run di test da poter visualizzare.	<u>UC23</u>
RFD01	Il supervisore deve poter visualizzare le metriche di consumo delle risorse relative al database.	<u>UC20.3</u>
RFO01	Companion deve poter effettuare ricerche ibride in modalità linked combinando i risultati tramite un modello di reranking.	<u>UC16</u>
RFO02	Companion deve poter effettuare ricerche ibride su singole entità combinando i risultati tramite un modello di reranking.	<u>UC08</u>

Tabella 2: Requisiti funzionali

3 Analisi dei requisiti

Codice	Descrizione	Fonti
RQM01	Vanno verificati i benchmark previsti sul seguente volume di dati di 10.000 ticket, 50.000 conversation item e 60.000 attachment.	Colloquio con il tutor
RQD01	Vanno verificati i benchmark previsti su un volume di dati di 100.000 ticket, 500.000 conversation item e 600.000 attachment.	Colloquio con il tutor
RQO01	Vanno verificati i benchmark previsti sul seguente volume di dati di 1.000.000 ticket, 5.000.000 conversation item e 6.000.000 attachment.	Colloquio con il tutor

Tabella 3: Requisiti di qualità

Codice	Descrizione	Fonti
RCM01	Il sistema di ricerca e il sistema di test devono essere realizzati con Python.	Colloquio con il tutor
RCM02	Il sistema di ricerca deve utilizzare FastAPI per la realizzazione delle API.	Colloquio con il tutor
RCM03	Il sistema di ricerca deve utilizzare un modello di embedding remoto di proprietà dell'azienda.	Colloquio con il tutor
RCM04	Il sistema di ricerca deve utilizzare un database relazionale Postgres. La ricerca semantica va realizzata sfruttando l'estensione pgvector.	Piano di lavoro
RCM05	Il sistema di test deve utilizzare Grafana per la costruzione e gestione della dashboard.	Colloquio con il tutor

3 Analisi dei requisiti

Codice	Descrizione	Fonti
RCM06	Il sistema di ricerca deve realizzare l'indicizzazione testuale e vettoriale per il modello dati del service desk HDA. Il modello è costituito dalle seguenti entità: • ticket, • conversation item, • attachment.	Piano di lavoro
RCM07	Il sistema di ricerca e il suo modello dati devono essere altamente configurabili. La richiesta di configurabilità è da intendersi nel seguente modo: • le entità che compongono il sistema di ricerca devono essere configurabili esternamente, • i campi da utilizzare nella ricerca per similarità devono essere configurabili esternamente, • i campi filtrabili devono essere configurabili esternamente, • i pesi da usare nella ricerca devono poter subire override nell'ambito di una singola ricerca • i pesi da usare per la fusione di ricerca semantica e full-text devono poter subire override nell'ambito di una singola ricerca	Colloquio con il tutor
RCM08	Il sistema di ricerca deve implementare i seguenti tipi di ricerche di similarità sulle singole entità: • semantica, • full-text, • ibrida.	Piano di lavoro

3 Analisi dei requisiti

Codice	Descrizione	Fonti
RCM09	Il sistema di ricerca deve rispettare il seguente comportamento per le ricerche full-text. Se la query di ricerca contiene l'informazione di lingua il sistema di ricerca limita il pool di candidati solo ai chunk della medesima lingua e utilizza solo la configurazione linguistica assegnata a tale lingua. Altrimenti tutti i chunk vengono valutati sia usando una configurazione di testo agnostica rispetto alla lingua sia usando la configurazione di testo per la lingua assegnata.	Colloquio con il tutor
RCM10	Il sistema di ricerca deve implementare per ogni tipo di ricerca di similarità su singola entità anche la rispettiva versione linked. Le sequenze di linking sono le seguenti: • conversation item → ticket, • attachment → ticket, • attachment → conversation item → ticket.	Colloquio con il tutor
RCM11	Il sistema di ricerca deve effettuare ogni tipo di ricerca tramite una singola query interpellando il database una sola volta.	Colloquio con il tutor
RCM12	Il sistema di ricerca deve implementare la funzionalità di ingestion dei dati.	Piano di lavoro
RCM13	Il sistema di test deve simulare dieci utenti che eseguono una query ogni dieci secondi.	Colloquio con il tutor

3 Analisi dei requisiti

Codice	Descrizione	Fonti
RCD01	Il sistema di ricerca deve supportare il deployment tramite Kubernetes.	Colloqui con il tutor
RCD02	Il sistema di ricerca deve supportare flussi di dati paralleli e non ordinati durante l'ingestion. Con non ordinati si intende che è possibile caricare prima gli attachment e poi i ticket.	Colloquio con il tutor
RCO01	Il sistema di ricerca deve supportare la ricerca ibrida con utilizzo di un modello di reranking per la fusione dei risultati.	Piano di lavoro

Tabella 4: Requisiti di vincolo

Di seguito, nella **Tabella 5** ho inserito il riepilogo dei requisiti, suddivisi per tipologia e necessità.

Tipo	Mandatory	Desirable	Optional	Somma
Functional	48	1	2	51
Qualitative	1	1	1	3
Constraint	13	2	1	16
Totale	62	4	4	70

Tabella 5: Riepilogo dei requisiti

Capitolo 4

Introduzione Teorica

In questo capitolo approfondisco le basi teoriche rilevanti per la realizzazione del progetto, le tecnologie rilevanti per il progetto e i relativi ruoli nella risoluzione dei problemi affrontati, quali strumenti sono stati adottati e altri strumenti adottati durante lo sviluppo.

4.1 Contesto e problema

Il progetto ha una finalità esplorativa: valuta quanti e quali workaround siano necessari affinché un sistema di ricerca basato su Postgres realizzi funzionalità di information retrieval pari all'attuale sistema di ricerca aziendale, basato su Elasticsearch. Non si tratta di un confronto assoluto tra le due tecnologie, ma di una valutazione mirata ai casi d'uso specifici di questo progetto. Elasticsearch nasce come motore di ricerca dedicato, mentre Postgres è un database relazionale a cui sono state aggiunte funzionalità di retrieval.

È prassi comune rivalutare periodicamente le tecnologie che compongono uno stack software, specialmente quando un'alternativa promette di semplificare l'architettura complessiva. I criteri tipici di una simile valutazione includono la complessità di utilizzo e manutenzione del sistema, le prestazioni e il rispetto di proprietà come l'atomicità e la consistenza delle transazioni. Il presente progetto si colloca in questo tipo di valutazione: verifica se Postgres, unificando ricerca full-text e semantica in un unico sistema relazionale, possa rappresentare un'alternativa valida a un'architettura che oggi si appoggia a un motore di ricerca dedicato ma disaccoppiato dal database relazionale primario, con i relativi costi di sincronizzazione tra i due sistemi.

L'adeguatezza di Postgres in questo contesto è valutata secondo i seguenti criteri:

- la complessità del database e delle funzioni di ricerca;
- i tempi di risposta;
- la qualità del retrieval, misurata tramite metriche di hitrate a diversi livelli di granularità. La definizione completa delle metriche è riportata nel caso d'uso **UC20.2 - Visualizza metriche di performance**.

4 Introduzione Teorica

La *ground truth_C* per i test con dati reali viene definita dall'attuale sistema di ricerca basato su Elasticsearch, in quanto rappresenta il comportamento di riferimento attualmente accettato e in uso in azienda. Per i test locali si adotta un approccio opposto: si genera la query partendo dal risultato e vengono applicate modifiche minime o nulle al testo per valutare correttamente i tempi di risposta e controllare che il sistema effettivamente funzioni.

È stato esplicitamente concordato con il tutor aziendale che le metriche relative al consumo di risorse non sono prioritarie per il tirocinio, in quanto una volta effettuato il deployment su server aziendale è già realizzato il tracciamento del consumo di risorse ed è quindi possibile estendere la dashboard Grafana per mostrare anche quello.

Per giustificare alcune scelte e capire meglio l'utilità di alcune funzionalità, in particolare della ricerca linked, è utile ricordare che il sistema di information retrieval si colloca dentro un sistema RAG, per ulteriori informazioni si veda il **Capitolo 2**.

Come già detto nel requisito **RCM06**, il modello dati di riferimento è quello del service desk HDA.

Tale modello dati è costituito dai seguenti elementi:

- ticket,
- conversation item,
- attachment.

E dalle seguenti relazioni uno a molti:

- ticket $\rightarrow$ conversation item,
- ticket $\rightarrow$ attachment,
- conversation item $\rightarrow$ attachment.

La ricerca per similarità viene eseguita in due modalità: su singola entità e linked.

La ricerca su singola entità esegue la ricerca solo su una delle entità del modello dati, mentre la ricerca linked cerca su tutte le entità del modello e ricostruisce per ogni entità cercata una visione globale del risultato tramite join, per poi unire i dati ottenuti in un unico risultato. Il funzionamento dettagliato è descritto nel **Capitolo 5**.

4.1.1 Caratteristiche di Elasticsearch

Elasticsearch è un motore di ricerca nato per l'indicizzazione e l'interrogazione di documenti e offre nativamente funzionalità avanzate di information retrieval. Tra i suoi punti di forza rilevanti per questo confronto vi sono l'ottimizzazione dell'ordinamento top-k, flessibilità nell'uso di *tokenizer_C* linguistici, la granularità dei filtri disponibili per la ricerca full-text e una maggiore manipolabilità dei dati indicizzati. In particolare, l'ordinamento top-k beneficia di algoritmi di pruning come *Block-Max WAND_C*[1], che sfruttano limiti superiori precalcolati

4 Introduzione Teorica

sul punteggio dei documenti per evitare il calcolo completo del ranking su candidati che non potrebbero comunque rientrare nei risultati finali, riducendo il costo computazionale della ricerca in presenza di termini ad alta frequenza. Elasticsearch offre inoltre *sharding*[2] nativo, ma tale funzionalità non è necessaria ai fini di questo progetto.

In quanto sistema orientato ai documenti, Elasticsearch non supporta i join tra entità distinte, salvo particolari configurazioni comunque incompatibili con il sistema aziendale: i join vengono realizzati lato backend. Questo è un trade-off intrinseco al suo modello dati, non un limite implementativo: è la stessa ragione per cui, all'opposto, un database relazionale come Postgres supporta facilmente i join per ricostruire una visione completa delle informazioni, ma non ha la stessa qualità di ricerca full-text. Nel contesto di questo progetto, tale caratteristica rende Elasticsearch meno adatto a gestire nativamente la ricerca linked, che richiede di correlare più entità del modello dati.

4.1.2 Caratteristiche di Postgres

Postgres è un database relazionale e supporta nativamente i join necessari alla ricerca linked. Attraverso *tsvector*, *tsquery* e pgvector, discusse nel dettaglio in [Sezione 4.5.1](#), è inoltre possibile realizzare ricerca full-text e semantica all'interno dello stesso sistema, riducendo la complessità architetturale rispetto all'uso di un motore di ricerca dedicato.

A differenza di Elasticsearch, le funzionalità di ricerca per similarità di Postgres non sono centrali, poiché il database è pensato principalmente per carichi di lavoro transazionali. Questo comporta che, per raggiungere un livello di funzionalità equivalente a quello offerto nativamente da Elasticsearch, siano necessari in alcuni casi workaround applicativi o strutturali, discussi in [Sezione 4.1.5](#).

4.1.3 Motivi del confronto

Le principali motivazioni alla base di questa valutazione sono le seguenti:

- pgvector ha introdotto di recente funzionalità di ricerca semantica avanzate in Postgres, rendendo oggi plausibile un suo utilizzo per l'information retrieval;
- l'utilizzo di un database relazionale anche per le funzionalità di ricerca permette di arricchire più facilmente database relazionali già esistenti, senza introdurre un sistema aggiuntivo dedicato.

4.1.4 Limiti del sistema attuale

I limiti descritti in questa sezione derivano dal modo in cui la ricerca è stata implementata nell'applicativo aziendale che utilizza Elasticsearch, non da limitazioni intrinseche del motore di ricerca.

Un limite riguarda l'impossibilità di eseguire la ricerca per similarità su sottoinsiemi di campi di testo divisi in chunk: ad esempio, non è possibile effettuare la ricerca semantica solo sul campo Problem o solo sul campo Solution di un ticket, poiché durante il calcolo degli embedding questi campi vengono concatenati. Tale funzionalità è invece disponibile per la ricerca full-text.

Per questo motivo il sistema di ricerca realizzato durante il tirocinio non rispecchierà il sistema attuale sotto questo aspetto, allineandosi invece con il comportamento desiderato dall'impresa.

4.1.5 Limiti di Postgres e pgvector

Le configurazioni testuali di Postgres sono piuttosto avanzate: supportano sinonimi, frasi sinonimo, *stemming* morfologico e un'ampia varietà di lingue[3].

Il limite principale non riguarda la completezza delle funzionalità linguistiche disponibili, quanto la flessibilità nel modificarle: definire o modificare una configurazione di ricerca testuale in Postgres richiede operazioni di data definition relativamente complesse, mentre in Elasticsearch un *analyzer*[4] può essere definito o modificato in modo molto più semplice, anche al momento della creazione dell'indice. Di conseguenza, la disponibilità di analyzer preconfigurati equivalenti a quelli offerti di default da Elasticsearch non è replicabile in Postgres se non tramite workaround.

Un'ulteriore limitazione riguarda il filtraggio: Postgres non offre nativamente la possibilità di filtrare i risultati in base al numero di parole della query che trovano corrispondenza in un documento, funzionalità invece disponibile in Elasticsearch tramite `minimum_should_match`[5]. Anche sul piano del ranking, la ricerca full-text nativa di Postgres non implementa l'algoritmo *BM25c*, a differenza di Elasticsearch.

Un limite distinto, ma collegato, riguarda l'ordinamento top-k: Postgres seleziona già in modo efficiente i primi k risultati tramite un top-k heap sort, evitando quindi un ordinamento completo dell'insieme dei candidati. Il costo computazionale non eliminato riguarda però la fase precedente: il punteggio di rilevanza viene comunque calcolato per intero su tutti i documenti che soddisfano il filtro di ricerca, prima che possano essere confrontati con l'heap dei migliori k risultati. Elasticsearch, tramite algoritmi come Block-Max WAND, evita questo costo. Esclude i documenti che non potrebbero comunque competere per le prime

posizioni usando un limite superiore precalcolato. Questa ottimizzazione non è replicabile in Postgres tramite configurazione o tuning: richiede che le strutture dati alla base dell'indice siano progettate per questo scopo, cosa che l'infrastruttura *GIN_G* non offre. La sua implementazione richiederebbe di fatto la sostituzione del motore di indicizzazione testuale con un motore di ricerca embedded (ad esempio Tantivy, tramite estensioni come ParadeDB), scelta esclusa a priori per le stesse ragioni di licenza discusse nella **Sezione 4.5**. Per questo motivo il limite viene documentato ma non affrontato tramite workaround e la sua rilevanza pratica per il progetto è discussa in **Sezione 4.5.1** in relazione ai volumi di dati coinvolti. I risultati concreti sono documentati nel **Capitolo 8**. Un limite più generale riguarda la ricerca ibrida: Elasticsearch espone nativamente un'unica interfaccia in grado di combinare ricerca full-text e semantica all'interno della stessa richiesta[6], applicando internamente algoritmi di fusione come RRF. Postgres non offre un operatore equivalente: la fusione dei risultati deve essere implementata esplicitamente, ad esempio tramite *common table expression (CTE)_G*[7], spostando sullo sviluppatore la responsabilità di una logica che in Elasticsearch è gestita dal motore stesso.

Anche la combinazione tra ricerca semantica e filtri relazionali presenta un limite condiviso da entrambe le tecnologie, seppur gestito con un diverso grado di automazione. Negli indici approssimati basati su grafo, applicare un filtro dopo la ricerca può restituire un numero di risultati inferiore a quello richiesto, anche quando esistono abbastanza documenti che soddisfano il filtro: questo vale sia per pgvector[8] sia per Elasticsearch[9]. Entrambi i sistemi offrono meccanismi di mitigazione a runtime: il pre-filtering nativo in Elasticsearch e le *iterative scan_G* in pgvector oltre a soluzioni strutturali come indici parziali o partizionamento. La differenza principale tra le due tecnologie risiede nel grado di automazione: in Elasticsearch il passaggio tra le diverse strategie di filtraggio è deciso automaticamente dal motore in base alla selettività del filtro, mentre in Postgres le iterative scan devono essere abilitate esplicitamente dallo sviluppatore e configurate.

4.2 Basi teoriche

L'information retrieval si occupa di individuare, all'interno di una collezione di dati, gli elementi più pertinenti rispetto a una richiesta espressa dall'utente. Nel contesto di questo progetto la richiesta è rappresentata da una query testuale, mentre la collezione può coincidere con i dati di una singola entità del modello oppure con l'insieme delle entità collegate secondo le regole di join configurate.

4 Introduzione Teorica

I risultati restituiti da un sistema di information retrieval non costituiscono un insieme non ordinato, ma una lista ordinata secondo un criterio di rilevanza decrescente, detta ranking. Questo concetto è alla base delle metriche di valutazione adottate nel progetto, discusse in **Sezione 4.1**. Per recuperare i dati da un sistema di information retrieval vengono usate delle funzioni di *similarity search*.

All'interno del progetto vengono usati i seguenti tipi di ricerca per similarità:

- **ricerca full-text**, usa un criterio di similarità basato sulla corrispondenza di lessemi e parole chiave;
- **ricerca semantica**, usa un criterio di similarità basato sulla distanza dei vettori di embedding corrispondenti alle frasi confrontate;
- **ricerca ibrida**, utilizza sia ricerca semantica sia ricerca full-text e ne combina i risultati con tecniche che ne gestiscono la diversa scala di punteggi.

La ricerca ibrida è la tipologia più rilevante ai fini del progetto. Le altre due assumono invece un ruolo secondario, principalmente di supporto al debugging: valutare le query di ricerca full-text e semantica in isolamento permette di individuare più facilmente la causa di un comportamento inaspettato nella ricerca ibrida, che altrimenti ne combinerebbe gli effetti rendendone più difficile l'analisi.

La ricerca ibrida combina i punti di forza della ricerca semantica e di quella full-text. La ricerca semantica non offre buone prestazioni nel keyword matching, punto di forza della ricerca full-text; quest'ultima, di contro, non è in grado di tracciare termini simili ma con forma testuale molto diversa, né di cogliere significati legati al contesto, aspetti in cui la ricerca semantica eccelle.

La combinazione unisce i risultati di entrambe le ricerche, assegnando un punteggio maggiore a quelli individuati da entrambe, senza scartare i risultati rilevanti prodotti da una sola delle due.

Questa fusione avviene principalmente tramite RRF (casi d'uso **UC07 - Ricerca ibrida con RRF** e **UC15 - Ricerca linked ibrida con RRF**) oppure tramite modelli di reranking (casi d'uso **UC08 - Ricerca ibrida con modello di reranking** e **UC16 - Ricerca linked ibrida con modello di reranking**).

4.3 Architettura del progetto

Ho scelto di separare il progetto in due sistemi distinti e indipendenti:

- **sistema di ricerca**,
- **sistema di test**.

Tale separazione garantisce lo sviluppo e la manutenzione indipendenti dei due sistemi.

4 Introduzione Teorica

4.3.1 Sistema di ricerca

Il sistema di ricerca è responsabile dell'implementazione del sistema di information retrieval: offre funzionalità di ingestion dei dati e di ricerca secondo le diverse modalità descritte nella **Sezione 4.1**.

Segue il pattern dell'architettura esagonale per ridurre il rischio che dei bias modellino il sistema in modo da dare un vantaggio ingiusto a Postgres, come indicato dal rischio **R10**.

È pensato per l'esecuzione su server.

4.3.2 Sistema di test

Il sistema di test ha il ruolo di eseguire i test di performance della ricerca e calcolare le relative metriche. Simula un numero configurabile di utenti paralleli che inviano richieste di ricerca al sistema di ricerca, confronta i risultati ottenuti con la ground truth attesa e registra query di test, ground truth e risultati su un database dedicato.

Segue anch'esso il pattern dell'architettura esagonale, per coerenza con il sistema di ricerca e per facilitarne la manutenzione.

È pensato per l'esecuzione locale, sulla macchina da cui viene avviato il test.

4.3.3 Dashboard Grafana

Nel rispetto del requisito **RCM05**, la dashboard per il controllo delle performance è gestita con Grafana.

Legge il database del sistema di test per calcolare le metriche, gestendo automaticamente il refresh e la selezione della dashboard relativa all'esecuzione di test più recente[10].

Grafana non fa parte dei sistemi applicativi sviluppati: vengono forniti solamente i file YAML e JSON necessari a costruirla.

4.3.4 Relazione tra i sistemi

I due sistemi non condividono né codice (eccetto un riuso di tipo copia e incolla, per convenienza) né risorse e sono sviluppati su repository separati.

Il sistema di test comunica con il sistema di ricerca simulando client esterni che inviano richieste di ricerca, mentre Grafana accede direttamente al database del sistema di test, bypassandolo, per leggerne le metriche.

4.4 Criteri di scelta delle tecnologie

I criteri di scelta delle tecnologie sono differenti per i due sistemi, per cui vengono approfonditi separatamente nella **Sezione 4.4.1** per il sistema principale e **Sezione 4.4.2** per il sistema di test.

4.4.1 Sistema di ricerca

La maggior parte delle tecnologie è fissata dai requisiti di vincolo (**Tabella 4**). Sono rimaste come scelte libere la libreria di language detection e la scelta del meccanismo di ricerca full-text.

Per la ricerca full-text, oltre alla soluzione nativa di Postgres basata su tsvector e tsquery, sono state valutate alcune estensioni che la implementano tramite l'algoritmo BM25. I criteri richiesti sono: compatibilità della licenza con l'uso in un prodotto commerciale e un livello di funzionalità sufficientemente avanzato rispetto alle esigenze del progetto.

Un'altra scelta libera riguarda la libreria utilizzata per il riconoscimento della lingua del testo. Il criterio decisivo è stato la compatibilità con la versione di Python utilizzata dal progetto (3.14).

Inoltre si è preferito non adottare librerie di query building, per mantenere il massimo controllo possibile sul codice SQL prodotto e non nascondere la complessità, coerentemente con il criterio già seguito per l'architettura del sistema (**Sezione 4.3.1**).

4.4.2 Sistema di test

Non sono stati posti vincoli specifici sulle tecnologie adottate: la scelta è stata guidata dalla loro capacità di soddisfare i requisiti, cercando al contempo di non introdurre tecnologie ulteriori rispetto a quelle già usate nel sistema di ricerca, per non aumentare la curva di apprendimento necessaria allo sviluppo.

4.5 Tecnologie

Nelle seguenti sezioni viene analizzato l'insieme di tecnologie adottate per la realizzazione del progetto.

Ogni tecnologia è contrassegnata anche dal relativo numero di versione, in quanto vi potrebbero essere stati aggiornamenti significativi che rendono obsolete considerazioni tecniche presenti in questo documento. Il progetto è composto da due sistemi distinti e indipendenti, perciò i relativi stack tecnologici sono analizzati separatamente nelle sezioni **sistema di ricerca** e **sistema di test**.

4 Introduzione Teorica

Fanno eccezione Python, Poetry e Postgres, in quanto comuni ad entrambi gli stack tecnologici, mentre Grafana non fa formalmente parte di nessuno dei due sistemi.

Python v3.14.X

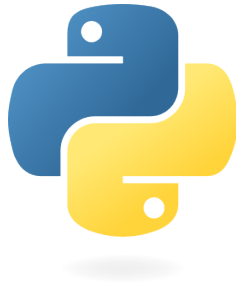


Figura 21: Logo Python

- **Descrizione:** Linguaggio di programmazione ad alto livello che supporta diversi paradigmi di programmazione.
- **Motivazione della scelta:** Richiesto dal requisito RCM01.

Poetry v2.0.0

- **Descrizione:** Strumento per la gestione delle dipendenze.
- **Motivazione della scelta:** Scelto perché è uno strumento che ho già usato in passato.

PostgreSQL v17.4.0

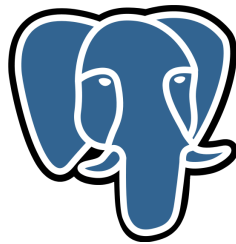


Figura 22: Logo Postgres

- **Descrizione:** Comunemente noto come **Postgres**, è un database open source che vanta una solida reputazione in termini di affidabilità, flessibilità e supporto degli standard tecnici aperti.
- **Motivazione della scelta:** Richiesto dal requisito RCM04.

4 Introduzione Teorica

Grafana v13.2.0



Figura 23: Logo Grafana

- **Descrizione:** Piattaforma software open source per la visualizzazione, l'analisi e il monitoraggio dei dati.
- **Motivazione della scelta:** Richiesto dal requisito **RCM05**.

4.5.1 Sistema di ricerca

Le tecnologie adottate all'interno del sistema di ricerca sono divise come segue.

4.5.1.1 Backend

FastAPI v0.115.0



Figura 24: Logo FastAPI

- **Descrizione:** Framework web veloce e moderno per la costruzione di API con Python.
- **Motivazione della scelta:** Richiesto dal requisito **RCM02**. Supporto nativo per l'asincronia.

Asyncpg v0.31.0

- **Descrizione:** Libreria Python dedicata alla comunicazione con database Postgres in un contesto Python asincrono.
- **Motivazione della scelta:** Scelta per via della sua efficienza e per il supporto all'asincronia.
- **Alternative valutate:**
 - **SQLAlchemy v2.0.0:** è un ORM, quindi è stato scartato in accordo con i criteri di scelta delle tecnologie stabiliti in **Sezione 4.4.1**.

pgvector-python v0.5.0

- **Descrizione:** Libreria dedicata al supporto di pgvector in Python.
- **Motivazione della scelta:** Necessaria per permettere ad **Asyncpg** di utilizzare funzioni e tipi di pgvector.

Py3langid v0.3.0

- **Descrizione:** Libreria dedicata alla language detection.
- **Motivazione della scelta:** Scelta per la compatibilità con **Python 3.14**.
- **Alternative valutate:**
 - **fasttext-langdetect v1.1.1:** minore manutenzione rispetto alla libreria scelta, incompatibile con Python 3.14.

4.5.1.2 Database

PostgreSQL Full-Text Search v17.4.0

- **Descrizione:** Meccanismo di ricerca full-text nativo di Postgres, basato sui tipi di dato tsvector e tsquery e sulla funzione di ranking `ts_rank`[11].
- **Motivazione della scelta:** Nessun problema di licenza legato all'uso in un prodotto commerciale e livello di funzionalità sufficientemente avanzato rispetto alle esigenze del progetto. Vi sono comunque presenti lacune di funzionalità che hanno richiesto dei workaround analizzati nel dettaglio nella **Sezione 6.4.4**.

Il limite tecnologico principale è dato dalla non implementazione della funzione di ranking BM25 e di tutte le ottimizzazioni che un motore di ricerca testuale offre. Il testo viene valutato solo sulla base del testo della tsquery e del testo del tsvector, senza utilizzo di un corpus di documenti per ripesare il punteggio del testo.

La differenza alla base del calcolo dello score può intaccare la qualità del ranking, ma garantisce che i punteggi calcolati su entità diverse siano direttamente comparabili.

Limite accettato dato che lo scopo del progetto è valutare la qualità della ricerca, con focus principale su pgvector.

Viene accettato anche il limite sulla non ottimizzazione WAND della ricerca in quanto per i limiti imposti nella scelta della tecnologia non è possibile valutare alternative; inoltre non va in conflitto con la finalità esplorativa: è utile valutare le prestazioni di una funzione nativa invece di aggiungere prematuramente un'ulteriore dipendenza.

- **Alternative valutate:**
 - **ParadeDB** v0.25.0: licenza incompatibile con i vincoli aziendali sull'uso commerciale.
 - **pg_textsearch (TimescaleDB)** v1.3.1: il rapporto tra vantaggi, limitazioni[12] e incognite da valutare non è stato ritenuto sufficientemente alto da giustificarne l'adozione. Le più importanti sono la mancanza di phrase query e l'incognita di comparabilità dei punteggi di diverse partizioni di una tabella: il partizionamento era già previsto per conformarsi alle raccomandazioni di pgvector. La stessa incognita va applicata alla confrontabilità dei punteggi di due entità diverse durante la ricerca linked.

pgvector v0.8.4

- **Descrizione:** Estensione Postgres che implementa funzionalità di ricerca semantica basate sui vettori.
- **Motivazione della scelta:** Richiesto dal requisito **RCM04**.

4.5.1.3 Deployment

Il sistema di ricerca è progettato per essere containerizzato tramite Docker, requisito necessario per il successivo deployment su Kubernetes nell'infrastruttura aziendale. Il deployment effettivo su Kubernetes, così come le valutazioni condotte sui dati e sui database aziendali reali, non rientrano nel lavoro svolto durante il tirocinio: sono demandati al team aziendale competente, sia per ragioni organizzative sia per assenza delle credenziali necessarie ad accedere all'infrastruttura aziendale interna.

Ai fini dello sviluppo e del test in locale è stato realizzato il Dockerfile per il sistema, insieme a una configurazione Docker Compose completa che include anche i servizi di supporto necessari (ad esempio Postgres). Docker Compose orchestra localmente gli stessi container Docker utilizzati poi in ambiente Kubernetes, garantendo coerenza di comportamento tra ambiente di sviluppo e ambiente di produzione.

4.5.1.4 Strumenti di supporto

Altri strumenti di supporto degni di nota sono **uvicorn**, **pydantic-settings**, **sqlparse**.

- Uvicorn è un server ASGI, scelta strettamente legata all'uso di FastAPI.
- Pydantic-settings viene usato per avere una gestione più pulita delle variabili d'ambiente, non richiede l'aggiunta di ulteriori dipendenze in quanto FastAPI utilizza già Pydantic.
- Sqlparse è una libreria di parsing e formattazione SQL, usata per migliorare la leggibilità delle query ricostruite durante il debug e il logging.

4.5.2 Sistema di test

Le tecnologie adottate all'interno del sistema di test sono divise come segue.

4.5.2.1 Backend

Locust v2.32.0

- **Descrizione:** Strumento per il testing di funzionalità che coinvolgono chiamate con protocollo http o altri protocolli.
- **Motivazione della scelta:** Scelto per la sua semplicità, adeguatezza al caso d'uso di simulazione di vari utenti paralleli e per la sua estensibilità.

4 Introduzione Teorica

Httpx v0.28.0

- **Descrizione:** Strumento per la comunicazione http.
- **Motivazione della scelta:** Scelto perché il più utilizzato, si integra bene con Locust.

Psycopg v3.0.0

- **Descrizione:** Driver per la comunicazione con un database Postgres.
- **Motivazione della scelta:** Scelto per la solidità e la compatibilità con Locust.
- **Alternative valutate:**
 - **Asynccpg** v0.31.0: scartato per la bassa compatibilità con Locust che lavora meglio su un contesto Python sincrono.

4.5.2.2 Database

Vengono usati due storage:

- un file JSONL per la persistenza delle query da eseguire e la ground truth, scelta dovuta alla semplicità di condivisione e alla mancanza di requisiti di prestazioni di scrittura e di lettura non sequenziale;
- un database Postgres, usato per loggare i risultati dei test, scelto per la facilità di integrazione con Grafana e per la necessità di scritture continue e ricalcolo continuo delle metriche, realizzato con una vista.

Per maggiori dettagli si vedano le informazioni di **Postgres**.

4.5.2.3 Deployment

Anche il sistema di test è containerizzato tramite Docker ed è orchestrato, insieme a database delle metriche e Grafana, tramite un'unica configurazione Docker Compose.

La containerizzazione di Locust non risponde a un'esigenza tecnica stretta: durante lo sviluppo è stato utilizzato esclusivamente in modalità headless e potrebbe quindi essere eseguito direttamente sulla macchina host. È stata comunque scelta per uniformità con il resto dello stack e per poter avviare l'intero ambiente di test con un unico comando, senza dover gestire manualmente le dipendenze.

Vi è inoltre il vantaggio di un'evoluzione più semplice qualora in futuro si volesse utilizzare l'interfaccia UI offerta da Locust.

4.5.2.4 Strumenti di supporto

Viene usato pydantic-settings per semplificare la gestione delle variabili d'ambiente.

Capitolo 5

Principi e caratteristiche del sistema

In questo capitolo descrivo i principi architetturali e le caratteristiche progettuali che guidano lo sviluppo del sistema di ricerca e del sistema di test a supporto della sua validazione.

5.1 Studio iniziale

Coerentemente con quanto stabilito nella **Sezione 2.3**, le prime due settimane sono state dedicate allo studio e all'analisi del contesto, con l'obiettivo di acquisire una comprensione solida dell'information retrieval, delle potenzialità di Postgres e delle capacità di Elasticsearch.

Nell'ambito di questo studio sono state valutate anche estensioni più evolute rispetto agli strumenti nativi di Postgres, come ParadeDB e pg_textsearch; entrambe sono state escluse dall'implementazione finale per i motivi discussi nell'**analisi della full-text search nativa**, ma il loro studio ha comunque contribuito alla comprensione delle capacità disponibili nell'ecosistema.

Per acquisire una comprensione adeguata delle capacità di Elasticsearch è stato condotto uno studio della documentazione ufficiale[13], che ha evidenziato, come atteso, una maggiore maturità e potenza rispetto a Postgres sia nell'ambito della ricerca full-text sia in quello della ricerca semantica.

Sul fronte della ricerca semantica non sono state individuate differenze sostanziali tra le due tecnologie in termini di funzionalità di base; Elasticsearch offre tuttavia una maggiore granularità nel controllo della precisione dei dati (con supporto a diversi livelli di precisione numerica, come float e double) e implementa un meccanismo di pre-filtering automatico quando una clausola di filtro risulta sufficientemente selettiva.

Sul fronte della ricerca full-text, lo studio ha invece evidenziato alcuni limiti significativi di Postgres rispetto a Elasticsearch:

5 Principi e caratteristiche del sistema

- le pipeline di elaborazione del testo sono utilizzabili solo se predefinite come configurazioni di testo, senza possibilità di composizione dinamica;
- manca il supporto a text analyzer multipli;
- i filtri disponibili sono limitati (corrispondenza a frase, a tutte le parole o ad almeno una parola), mentre Elasticsearch consente di richiedere la corrispondenza di un numero specifico di parole, anche determinato dinamicamente in base alla lunghezza della query;
- la funzione di scoring nativa risulta meno evoluta;
- nel calcolo dei punteggi e nel loro ordinamento non vengono applicate le stesse ottimizzazioni di Elasticsearch.

Parallelamente allo studio teorico, sono stati realizzati progressivamente alcuni proof of concept, con l'obiettivo di acquisire familiarità pratica con le tecnologie valutate.

Non tutte le capacità di Postgres analizzate in questa fase sono state successivamente impiegate nel sistema, in parte per la mancanza di un'integrazione organica con i casi d'uso del progetto; le scelte effettive sono motivate nelle sezioni seguenti.

5.2 Principi del sistema di ricerca

In questa sezione vengono trattati principi e caratteristiche relativi al sistema di ricerca.

5.2.1 Definizione modello dati

Il modello dati del sistema di ricerca deve rappresentare le entità del dominio del service desk HDA, articolate in ticket, conversation item e attachment come richiesto da **RCM06**, collegate tra loro secondo una struttura relazionale gerarchica.

Aggiungendo le caratteristiche richieste dal requisito **RCM07**, il modello dati così definito costituisce la fonte di verità del sistema e comprende:

- l'elenco delle entità;
- per ciascuna entità, la definizione dei campi (nome, tipo e obbligatorietà), i campi identificativi (con supporto anche a chiavi composte), i ruoli dei campi e i relativi pesi di default;
- l'elenco dei vincoli tra le entità. Questi non sono stati limitati fin da subito al solo vincolo relazionale, ma definiti in forma generica, di cui il vincolo relazionale rappresenta una specializzazione: si tratta di un punto di estensione esplicito, pensato per accomodare eventuali tipologie di vincolo non relazionale che potrebbero emergere in futuro;
- l'elenco delle regole di linking, trattate nel dettaglio nella **Sezione 5.2.3.4** dedicata alla ricerca linked.

5 Principi e caratteristiche del sistema

I dati effettivamente utilizzati in produzione dall'azienda non coincidono necessariamente con il modello analizzato in questo documento, poiché variano in base alle esigenze specifiche di ciascun cliente. Questa variabilità è una delle motivazioni alla base del requisito di configurabilità del sistema.

I ruoli supportati per i campi sono due:

- `searchable`, indica i campi su cui è possibile eseguire una ricerca di similarità;
- `filterable`, indica i campi utilizzabili come filtro.

A partire da questi due ruoli sono state definite alcune regole di progetto. Una parte è imposta direttamente dai requisiti raccolti, un'altra è frutto di scelte autonome, resa necessaria dalla natura esplorativa del progetto: Per decisione di progetto, non esiste un ruolo dedicato ai campi restituibili da una ricerca: si assume che siano restituibili tutti i campi privi di ruolo o con ruolo `filterable`. I campi `searchable` non sono restituibili per intero, per evitare di riportare porzioni di testo estese e poco significative; ogni ricerca restituisce comunque, a prescindere, il chunk di testo che ha determinato il match, il nome del campo di provenienza e il numero del chunk. Il recupero di porzioni di testo più estese è demandato a un caso d'uso dedicato, trattabile in implementazioni future, che a partire dalle informazioni identificative del match ricostruisce l'intorno testuale del chunk corrispondente; oppure lo step di espansione può essere incluso nella pipeline di recupero del testo. Tuttavia tale funzionalità non è ritenuta rilevante per un progetto esplorativo: andrebbe ad aggiungere uno step di join che richiede solo una query che filtra per chiave primaria, quindi non è necessario dimostrarne l'efficienza.

Un campo deve avere ruolo `searchable` se e solo se rappresenta testo suddiviso in chunk. La necessità che un campo `searchable` sia sempre diviso in chunk è un vincolo imposto dalla natura del problema; il vincolo complementare, ovvero che ogni testo diviso in chunk debba necessariamente avere ruolo `searchable`, è invece una decisione di progetto.

Come richiesto dal requisito **RCM07**, la fonte di verità deve essere definita esternamente, in modo da poter modificare il modello dati senza intervenire sul codice applicativo quando il sistema deve essere adattato all'evoluzione del contesto. Lo stesso requisito impone inoltre che siano configurabili esternamente i pesi di default della ricerca ibrida e i pesi di default utilizzati nelle singole ricerche di similarità; ogni richiesta di ricerca deve poter effettuare l'override di questi pesi, semplicemente fornendo i valori da utilizzare per quella specifica ricerca.

Per gestire l'arrivo di grandi volumi di dati non ordinati durante la fase di ingestion, come descritto dal requisito **RCM12**, preservando al contempo la consistenza del sistema, il modello dati adotta un principio di staging: i dati in ingresso transitano attraverso un'area intermedia prima

5 Principi e caratteristiche del sistema

di essere resi disponibili alla ricerca, disaccoppiando così il processo di scrittura da quello di lettura.

5.2.2 Caratteristiche del database

Il modello dati si traduce sul database secondo le seguenti regole.

Per ogni entità sono previste due tabelle:

- una tabella principale, contenente tutti i campi non searchable definiti dal modello dati (privi di ruolo o con ruolo filterable);
- una tabella dei chunk, dedicata alla gestione dei molteplici campi chunkabili. Ha come chiave primaria l'insieme composto dalla chiave esterna verso la tabella principale, dal campo di provenienza del testo e dal numero del chunk. Contiene inoltre tutti i campi necessari alla ricerca: il vettore di embedding, la lingua, il testo in chiaro e la rappresentazione utilizzata per la ricerca full-text (approfondita nella **Sezione 5.2.3.2.2**).

La scelta di normalizzazione deriva dal fatto che pgvector non supporta i multi-vector embedding in modo nativo, anche se in caso servano un numero fisso di vettori è possibile definire più colonne di embedding.

Per motivi di ottimizzazione, i campi con ruolo filterable vengono inoltre replicati sulla tabella dei chunk, in aggiunta alla loro presenza nella tabella principale. Questa denormalizzazione mira a riprodurre il meccanismo di pre-filtering di Elasticsearch: avendo i campi filterable già presenti e indicizzati direttamente sulla tabella dei chunk, il query planner di Postgres può scegliere di applicarlo autonomamente quando una clausola di filtro risulta sufficientemente selettiva.

Questa necessità nasce da una differenza di principio tra le due tecnologie: Elasticsearch ottimizza per default tutto ciò che non viene esplicitamente escluso dall'indicizzazione, mentre Postgres adotta l'approccio opposto, ottimizzando solo ciò che viene esplicitamente predisposto a tale scopo. La denormalizzazione dei campi filterable sulla tabella dei chunk è quindi il meccanismo con cui il sistema compensa questa differenza, rendendo esplicitamente disponibile al planner ciò che in Elasticsearch sarebbe già ottimizzato di default.

La stessa struttura facilita inoltre l'introduzione futura di ottimizzazioni mirate a sottoinsiemi di dati, ad esempio con gli indici parziali.

5.2.2.1 Gestione della staging area

Nella staging area vengono replicate solo le tabelle delle entità e dei chunk.

Poiché le sessioni di ingestion vengono avviate e concluse esplicitamente, il database deve tenerne traccia dello stato. A tale scopo sono previste due tabelle:

5 Principi e caratteristiche del sistema

- una tabella per registrare lo stato delle sessioni di ingestion;
- una tabella per tenere traccia delle richieste di ingestion, necessaria per verificare se vi siano ancora scritture in corso.

Una sessione può assumere i valori aperta, chiusa e finalizzata: aperta indica che l'ingestion è attiva e può accettare nuovi dati; chiusa indica che l'ingestion è ancora attiva ma non accetta più dati, poiché è in corso la promozione dei dati dalla staging area verso le tabelle reali; finalizzata indica una sessione conclusa, unica condizione a partire dalla quale può essere aperta una nuova sessione. Deve inoltre essere garantito che possa esistere al più una sessione di ingestion attiva (aperta o chiusa) alla volta.

La promozione è il processo che trasforma e trasferisce i dati dalla staging area verso le tabelle reali, eliminando contestualmente i dati trascritti con successo.

La staging area si differenzia dalle tabelle originali nei seguenti punti:

- la chiave primaria è uno **staging_id** incrementale, il che permette di gestire eventuali duplicati in fase di staging e garantisce una maggiore velocità di scrittura;
- non viene utilizzato il tipo **vector**: la staging area non necessita di essere interrogabile per similarità, pertanto gli embedding sono memorizzati come vettori di double;
- gli indici vengono creati esclusivamente sulle chiavi esterne, al fine di favorire i join necessari in fase di verifica per la promozione dei dati verso le tabelle definitive;
- le tabelle di staging dei chunk non sono denormalizzate: la denormalizzazione viene gestita dalla logica di promozione, in modo da mantenere l'ingestion disaccoppiata da questa responsabilità.

5.2.3 Tipologie di ricerca e requisiti implementativi

Data la natura esplorativa del progetto, per tutte le entità del modello dati vengono adottate le medesime impostazioni: i vettori di embedding sono calcolati con lo stesso modello, con le medesime ottimizzazioni e la medesima funzione di distanza; analogamente, per la ricerca full-text viene utilizzata la stessa configurazione testuale per tutte le entità e la stessa funzione di ranking.

Questa scelta semplifica la progettazione iniziale del sistema, rimandando l'eventuale differenziazione delle impostazioni per singola entità a sviluppi futuri, qualora emergessero esigenze specifiche non coperte da una configurazione uniforme.

L'adozione di impostazioni uniformi evita inoltre problemi in fase di combinazione dei risultati tra entità diverse, poiché garantisce che i punteggi prodotti dalle ricerche su entità distinte siano direttamente paragonabili. Anche qualora questa comparabilità diretta venisse meno,

5 Principi e caratteristiche del sistema

ad esempio in seguito a una futura differenziazione delle configurazioni per singola entità, il sistema resterebbe comunque estendibile adottando la stessa logica di fusione RRF già utilizzata per la ricerca ibrida. Questo introdurrebbe però un doppio livello di fusione: una prima volta all'interno della ricerca ibrida sulla singola entità, una seconda volta nella combinazione tra entità con il rischio di appiattire sfumature o differenze significative nella fase intermedia.

Il presente lavoro non affronta la questione della paragonabilità diretta tra i punteggi prodotti da due o più fusioni RRF: stabilire se un semplice *rescoring* sia sufficiente, oppure se sia necessario un ulteriore livello di fusione, resta un aspetto da approfondire in lavori futuri.

5.2.3.1 Ricerca semantica

La configurazione condivisa per la ricerca semantica, richiamata in apertura di sezione, recepisce le seguenti ottimizzazioni raccomandate da pgvector:

- **Partizionamento**[8]: le tabelle dei chunk vengono partizionate sul nome del campo dati di origine. pgvector raccomanda il partizionamento quando si effettuano filtri su un insieme ristretto di valori; tale filtro è necessario per supportare le ricerche ristrette a un sottoinsieme di campi.
- **Tipo di indice**[14]: viene adottato un indice *HNSW_G*. Si tratta di una scelta relativamente arbitraria tra le opzioni disponibili, motivata dal fatto che si adatta meglio a scenari con inserimenti incrementali, coerenti con il principio di staging adottato per l'ingestion.
- **Indice su espressione**[15]: l'indice HNSW viene costruito tramite un indice su espressione che esegue la riduzione di precisione dei vettori, al fine di contenere il consumo di memoria.
- **Oversampling**[15]: pgvector raccomanda l'utilizzo di *oversampling_G* con rescoring in combinazione con l'uso di indici, in particolare quando questi sono costruiti su vettori a precisione ridotta, come nel caso descritto sopra.
- **Convenzione sui punteggi**[16]: per motivi di ottimizzazione interna e di coerenza tra le metriche disponibili, pgvector restituisce sia la *cosine similarity_G* sia l'*inner product_G* in una forma per cui il risultato più pertinente corrisponde al valore più basso (calcolando l'inner product con segno negativo e la cosine similarity in modo analogo). Questa convenzione viene tradotta internamente, in modo che l'utente del sistema non debba conoscere né ragionare in base al funzionamento interno di pgvector.
- **Scelta della distanza**[17]: per i vettori già normalizzati viene utilizzato l'inner product al posto della cosine similarity, poiché sui vettori

normalizzati i due valori sono equivalenti a meno di una costante, ma il calcolo dell'inner product risulta meno oneroso.

5.2.3.2 Ricerca full-text

Le funzionalità e le caratteristiche della ricerca full-text derivano principalmente dal requisito **RCM09** e dalla necessità di implementare un insieme minimo di funzionalità che avvicinino la ricerca full-text a quella offerta da Elasticsearch, limitatamente ai casi d'uso di questo progetto. Le caratteristiche minime da replicare e i relativi workaround elaborati sono i seguenti:

- **Text analyzer multipli:**

per ogni chunk di testo sono necessari due text analyzer distinti, uno agnostico rispetto alla lingua e uno specifico per la lingua. Per sopprimere alla mancanza di un supporto nativo a configurazioni di testo multiple, si sfrutta il fatto che il numero di configurazioni necessarie per campo è limitato (una language-agnostic e una language-specific), utilizzando due tsvector distinti per la ricerca non specifica e per quella specifica per lingua. La gestione dei tsvector è approfondita nella **Sezione 5.2.3.2.2**.

- **Funzione di ranking granulare:**

Elasticsearch e altri motori di ricerca full-text avanzati permettono di definire query con meccanismi di boosting dei risultati basati sull'accuratezza della corrispondenza, ad esempio assegnando un punteggio più alto a un testo che soddisfa tutte le clausole di una query composta (clausole in OR, *phrase query_C*, *all-words query_C*, ecc.).

La ricerca full-text di Postgres non implementa nativamente questo comportamento: le sue funzioni di ranking si limitano a verificare il rispetto della query, calcolando un punteggio solo in caso positivo e restituendo zero altrimenti. Per simulare il comportamento desiderato è quindi necessario calcolare e sommare separatamente il punteggio delle singole sotto-query.

- **Query di filtro:**

nativamente Postgres offre tre tipologie di query per ranking e filtraggio: phrase query, all-words query e *any-word query_C*; a ciascuna di queste è possibile aggiungere, per singola parola, un carattere jolly per la ricerca per prefisso.

Non è previsto alcun supporto nativo per una corrispondenza di almeno X parole. La sua replicazione tramite composizione di tsquery è stata esclusa per l'eccessiva complessità computazionale. È stato quindi adottato un workaround che tratta i tsvector come array di testo ordinati lessicograficamente, ordinamento gestito nativamente da Postgres in fase di creazione del tsvector: questa preconditione di ordinamento consente di ridurre significativamente la complessità

5 Principi e caratteristiche del sistema

della ricerca, richiedendo tuttavia, ai fini dell'ottimizzazione, un tipo di indice diverso da quello utilizzato per le altre query full-text. Da questa scelta deriva che tale meccanismo non può essere utilizzato per il ranking, poiché non produce una tsquery; questo non costituisce una limitazione, poiché anche in Elasticsearch un vincolo di questo tipo viene utilizzato solo ai fini del filtraggio, mentre l'ultimo livello di ranking è affidato a una any-word query.

La ricerca avviene sempre per partizione, anche quando ciò non è strettamente necessario: questa scelta semplifica l'applicazione dei pesi e consente di riutilizzare la struttura già adottata per la ricerca semantica. Per quanto riguarda il comportamento richiesto dal requisito **RCM09**, la ricerca ottimizzata su lingua si limita a filtrare sul campo lingua del chunk, operazione ottimizzata tramite un indice parziale dedicato per ciascuna lingua supportata. La ricerca multilingua, invece, esegue una prima ricerca utilizzando la configurazione di testo non language-specific, seguita da una ricerca language-specific per ciascuna delle lingue previste a catalogo. Questo approccio è sostenibile proprio perché il numero di lingue supportate è ridotto, a fronte di un volume di dati considerevole per ciascuna di esse. I risultati delle diverse ricerche vengono infine combinati tramite rescoring.

5.2.3.2.1 Ricerca con soglia di corrispondenza

Elasticsearch permette di definire una soglia di corrispondenza minima tramite il parametro `minimum_should_match`, che consente di richiedere che solo una percentuale o un numero minimo di termini della query sia presente nel documento, con soglie che possono variare in funzione della lunghezza della query stessa (query già elaborata dalla configurazione testuale, con le stopwords rimosse). Postgres non offre alcun operatore nativo equivalente.

Prima di arrivare alla soluzione adottata, sono stati considerati e scartati due approcci alternativi:

- un prefiltro costruito come OR di tutti i termini della query, seguito dal conteggio esatto dei match sui soli candidati. Sebbene l'OR sia indicizzabile tramite GIN, per query lunghe o composte da termini poco selettivi il filtro produce un insieme di candidati che copre una porzione consistente della tabella, vanificando il beneficio dell'indice;
- la generazione combinatoria di tutte le clausole AND di k termini su n, unite da OR. Il numero di combinazioni cresce secondo $C(n,k)$, risultando rapidamente insostenibile all'aumentare della lunghezza della query.

La soluzione adottata sfrutta il fatto che Postgres ordina nativamente i lessemi in fase di creazione del tsvector (approfondito in **Sezione 5.2.3.2.2**): il tsvector viene trattato come un array ordinato

5 Principi e caratteristiche del sistema

lessicograficamente e il problema della corrispondenza minima viene ricondotto a un controllo di overlap tra array, tramite l'indice dedicato già introdotto nella sezione precedente.

L'ordinamento lessicografico è ciò che rende possibile ricondurre il problema a un semplice controllo su un prefisso dell'array, anziché a un'esplosione combinatoria di sottoinsiemi da verificare: garantendo un ordine deterministico e condiviso tra la query e ogni documento candidato, permette di individuare tramite un singolo slice, calcolato una sola volta sull'array della query, quali posizioni è sufficiente controllare.

Questo meccanismo costituisce un prefiltro, secondo lo stesso principio già adottato dalla ricerca full-text nativa di Postgres, dove il filtraggio tramite indice GIN precede il calcolo del ranking. Il prefiltro rappresenta una condizione necessaria, ma non sufficiente, rispetto alla soglia richiesta: per costruzione, un documento che soddisfa realmente la soglia richiesta contiene necessariamente almeno un match tra i lessemi verificati dal filtro, per un argomento riconducibile al *principio dei cassetti*. Se un documento matcha almeno k lessemi su n totali, non è possibile che tutti i match cadano al di fuori del prefisso controllato dal filtro, poiché al di fuori di esso restano solo $k-1$ posizioni, insufficienti a raggiungere la soglia. Il filtro può tuttavia produrre falsi positivi, poiché verifica solo la presenza di almeno un match nel prefisso, senza garantire che il numero totale di match nel documento raggiunga effettivamente la soglia richiesta. Per questo motivo il prefiltro deve sempre essere seguito da un conteggio esatto dei match sui soli candidati sopravvissuti, così da scartare gli eventuali falsi positivi e verificare la soglia effettiva.

Rispetto all'approccio precedentemente adottato, basato su una any-word query utilizzata anche ai fini del filtraggio, questa soluzione riduce sensibilmente il numero di candidati da valutare singolarmente, poiché il filtro per overlap è più selettivo pur restando indicizzabile.

5.2.3.2.2 Gestione dei tsvector

La documentazione ufficiale di Postgres[11] non esprime una preferenza netta tra due strategie di ottimizzazione delle ricerche full-text: l'uso di indici su espressione oppure la materializzazione dei tsvector in colonne dedicate. I primi sono più leggeri in termini di spazio occupato, ma più difficili da gestire rispetto ai vettori materializzati.

Per questo progetto si è scelto di materializzare i tsvector. Alla tabella dei chunk sono state aggiunte due colonne: una per il tsvector language-agnostic e una per quello language-specific. È stata prevista una sola colonna per la versione language-specific, non una per lingua, poiché ogni chunk ha una singola lingua assegnata e non richiede supporto multilingua a livello di singolo chunk.

5 Principi e caratteristiche del sistema

L'alternativa sarebbe stata mantenere due serie di indici su espressione distinti, uno per il ranking full-text nativo e uno per il filtro overlap. A livello di spazio occupato dagli indici stessi, le due strategie sono equivalenti: la materializzazione non comporta alcun risparmio in tal senso, anzi introduce un costo aggiuntivo, poiché le colonne materializzate occupano spazio extra su disco rispetto al calcolo del tsvector a runtime tramite indici su espressione. Il motivo principale della scelta è quindi di comodità implementativa: avere le colonne materializzate consente di costruire su di esse entrambi gli insiemi di indici senza dover ripetere la stessa espressione in più punti dello schema. Inoltre questa scelta si è rivelata utile per la realizzazione di un workaround meglio trattato nella **Sezione 6.4.4**.

Il trade-off tra il costo di ricalcolare il tsvector a ogni interrogazione (nel caso di indici su espressione) e lo spazio extra occupato dalla loro materializzazione resta aperto e non è stato oggetto di analisi approfondita in questo lavoro: una valutazione più rigorosa richiederebbe ulteriori considerazioni e i risultati di sperimentazioni reali.

I due insiemi di indici, quello per la ricerca full-text nativa e quello per il filtro overlap, coesistono sulle stesse colonne materializzate. Questa scelta è coerente con la natura esplorativa del progetto: mantenerli distinti rende le due strategie intercambiabili, semplificando la sperimentazione. Qualora si decidesse in futuro di abbandonare il filtro overlap o l'uso delle tsquery per il filtraggio, l'indice corrispondente può essere eliminato senza conseguenze, poiché le funzioni di ranking native di Postgres non richiedono la presenza di un indice per funzionare.

In entrambi i casi è necessario un indice generico per la ricerca language-agnostic e una serie di indici parziali, uno per lingua, per la ricerca language-specific.

5.2.3.3 Ricerca ibrida

La ricerca ibrida combina i risultati della ricerca semantica e della ricerca full-text sulla medesima entità, fondendoli tramite l'algoritmo di RRF. Si tratta di una tecnica standard per la fusione di risultati provenienti da fonti con punteggi non direttamente paragonabili tra loro, quali quelli prodotti dalla ricerca semantica e dalla ricerca full-text.

La fusione viene eseguita interamente lato database, in un'unica query, coerentemente con il requisito **RCM11**. Questa scelta non è motivata da una maggiore velocità di calcolo di Postgres rispetto al backend applicativo, ma dal fatto che eseguire la fusione lato applicativo richiederebbe un round-trip di rete aggiuntivo tra database e backend per un'operazione che può essere svolta interamente all'interno del database stesso, senza necessità di scambiare dati con l'esterno.

5 Principi e caratteristiche del sistema

Una pratica consigliata per la ricerca ibrida è l'applicazione di un oversampling sia sui risultati della ricerca semantica sia su quelli della ricerca full-text, prima della fusione. Questo permette di non perdere risultati che una delle due tecniche posiziona appena al di fuori del numero di risultati richiesto, mentre l'altra li colloca in una posizione sufficientemente alta: includerli nella fusione permette a tali risultati di ricevere, correttamente, un punteggio più alto di quanto riceverebbero se venissero esclusi a monte. Tale oversampling non è in alcun modo connesso all'oversampling della ricerca semantica. Nell'ambito di una ricerca ibrida i due oversampling vengono comunque sommati anche se in modo indiretto.

5.2.3.4 Ricerca linked

La ricerca linked permette di recuperare, a partire dalle singole entità, un quadro informativo più ampio ricostruito risalendo le relazioni verso le entità collegate, anziché restituire un singolo frammento isolato. Ad esempio, a partire da un attachment è possibile risalire le relazioni passando per il conversation item associato fino ad arrivare al ticket, restituendo le informazioni di tutte le entità coinvolte in un'unica chiamata. Questa funzionalità è particolarmente rilevante nel contesto dell'information retrieval applicato a basi di conoscenza strutturate e correlate tra loro, dove il contesto rilevante per un'informazione raramente è contenuto interamente in una singola entità isolata.

Nello specifico, la ricerca linked esegue prima una ricerca indipendente su ciascuna entità coinvolta, per poi ricostruire, seguendo le regole di linking definite nel modello dati, i collegamenti tra i risultati tramite join. Per risalita si intende la navigazione delle relazioni dall'entità figlia verso l'entità genitore (ad esempio da attachment verso conversation item e da conversation item verso ticket). Il progetto supporta esclusivamente questa direzione di navigazione: la discesa, oltre a non essere banale da implementare, non rientra tra gli interessi dell'azienda per questo tirocinio.

Alcune entità, come attachment, dispongono di più regole di linking possibili verso entità diverse (ad esempio verso conversation item oppure direttamente verso ticket). Per questi casi si è scelto di adottare la regola del primo cammino valido: viene applicata la prima regola di linking per cui è presente un riferimento effettivo e le successive vengono considerate solo in sua assenza; ad esempio: se un attachment non presenta un riferimento diretto a un ticket, viene utilizzato il riferimento al conversation item, qualora presente.

Questa regola nasce da una scelta di disaccoppiamento generale. Nei dati reali, un attachment non può avere contemporaneamente un riferimento sia a un conversation item sia a un ticket: si tratta di un vincolo di

integrità proprio del modello dati, che tuttavia non è stato implementato come vincolo a livello di singola entità (né tramite controllo a database né nella logica applicativa), poiché ritenuto fuori dal perimetro di questo progetto e di scarso beneficio pratico rispetto alla complessità che avrebbe introdotto. Anche qualora fosse stato implementato, associarlo direttamente alle regole di linking non sarebbe stata una soluzione opportuna, per lo stesso principio di separazione già adottato tra la configurazione della ricerca linked e la definizione dei vincoli relazionali tramite chiavi esterne. La regola del primo cammino valido permette quindi alla ricerca linked di funzionare correttamente a prescindere dall'esistenza o meno di un simile vincolo, mantenendo il meccanismo disaccoppiato e più facilmente estendibile in futuro.

Per la ricerca linked ibrida, la fusione RRF tra i risultati di ricerca semantica e full-text viene eseguita a livello di singola entità, prima dell'esecuzione del linking. Questa scelta rispecchia il comportamento di Elasticsearch nello stesso scenario.

5.2.3.5 Pipeline di esecuzione delle ricerche

In questa sezione viene illustrato l'ordine delle operazioni eseguite per ogni tipo di ricerca.

- La **ricerca semantica** si articola come segue:
 1. ricerca sulla singola partizione della tabella dei chunk, con applicazione dell'oversampling necessario, come descritto in precedenza, quando si opera su indici a precisione ridotta e applicazione del filtro definito dall'utente;
 2. applicazione della soglia di threshold sul raw score;
 3. applicazione del peso configurato per campo, successivamente al threshold e precedentemente alla combinazione dei risultati;
 4. combinazione dei risultati delle diverse partizioni tramite rescoring semplice, sufficiente perché i punteggi sono direttamente comparabili tra loro;
 5. join sulla tabella principale, per il recupero dei dati richiesti;
 6. restituzione dei dati richiesti dalla query.
- La **ricerca full-text** si articola in due varianti: ottimizzata sulla lingua e multilingua.

La ricerca ottimizzata sulla lingua si articola come segue:

1. ricerca sulla singola partizione, con selezione per lingua, esecuzione dell'overlap query o di un'altra query full-text configurata e applicazione del filtro definito dall'utente;
2. applicazione della soglia di threshold sul raw score;
3. applicazione del peso configurato per campo, successivamente al threshold e precedentemente alla combinazione dei risultati;

5 Principi e caratteristiche del sistema

4. combinazione dei risultati tramite rescoring semplice, per lo stesso motivo di comparabilità già descritto per la ricerca semantica;
5. join sulla tabella principale, per il recupero dei dati richiesti;
6. restituzione dei dati richiesti dalla query.

La ricerca multilingua riutilizza le ricerche specifiche per lingua appena descritte:

1. esecuzione della ricerca language-agnostic, articolata come le ricerche language-specific;
 2. esecuzione delle ricerche language-specific, una per ciascuna lingua supportata a catalogo, ciascuna strutturata secondo i passi della ricerca ottimizzata su lingua descritta sopra;
 3. combinazione dei risultati tramite rescoring semplice;
 4. restituzione dei dati richiesti dalla query.
- La **ricerca ibrida** si articola come segue:
 1. esecuzione della ricerca full-text, recuperando solo le chiavi primarie e i valori restituiti sempre (nome del campo, testo matchato, numero del chunk); la fusione RRF opera infatti sull'ordinamento dei risultati e non richiede il raw score;
 2. esecuzione della ricerca semantica, con lo stesso criterio di recupero minimale;
 3. combinazione dei risultati tramite RRF, applicando i pesi di bilanciamento tra ricerca semantica e full-text: a differenza del rescoring semplice, qui è necessaria la fusione RRF poiché i punteggi delle due tecniche non sono direttamente comparabili tra loro;
 4. join sull'entità principale, eseguito una sola volta sui risultati già fusi, evitando di recuperare i dati completi due volte per la stessa entità;
 5. restituzione dei dati richiesti dalla query.
 - La **ricerca linked**, indipendentemente dal tipo di ricerca sottostante, esegue le seguenti operazioni:
 1. esecuzione della ricerca dello stesso tipo specificato per ciascuna entità coinvolta, garantendo la restituzione delle chiavi primarie;
 2. join per ricostruire l'informazione completa, seguendo le regole di linking definite nel modello dati;
 3. applicazione di un filtro successivo al join;
 4. combinazione dei risultati delle ricerche sulle diverse entità tramite rescoring semplice: i punteggi prodotti dalle ricerche sulle singole entità sono in questo caso direttamente comparabili e non richiedono quindi una fusione RRF;
 5. restituzione dei dati richiesti dalla query.

Nel caso specifico della ricerca linked ibrida, i pesi per campo vengono applicati a livello di singola entità prima della fusione RRF che le combina, anziché successivamente. Si tratta di una deviazione rispetto

5 Principi e caratteristiche del sistema

al principio generale di applicare i pesi dopo la fase di filtraggio e prima della combinazione, ma è una scelta ritenuta accettabile poiché lo stesso comportamento è adottato da Elasticsearch in scenari analoghi.

Per garantire un round-trip unico, come richiesto dal requisito **RCM11**, le diverse sequenze di query vengono combinate tramite clausole **WITH**, costruendo la query complessiva in modo incrementale: ogni livello successivo può fare riferimento alle proprie sotto-query come se fossero dati già disponibili. È importante che le singole porzioni definite tramite **WITH** non materializzino porzioni di tabella di dimensione non limitata, ma vengano sempre delimitate a un numero finito e contenuto di record tramite clausole **LIMIT**.

5.2.4 Caratteristiche del backend

Il backend organizza le impostazioni generali di ricerca tramite oggetti di configurazione strutturati, costruiti a monte e iniettati nei livelli che compongono la ricerca. Questa struttura, coerentemente con il requisito **RCM07**, permette di effettuare l'override dei pesi per una singola richiesta di ricerca; in assenza di override, vengono applicati i pesi di default definiti nel modello dati.

Le configurazioni contenenti i pesi sono oggetti di dominio, agnostici rispetto alla tecnologia sottostante e associano a ciascuna entità i seguenti valori:

- la lista dei pesi da applicare ai vari campi dati durante il rescoring;
- i pesi da applicare nella fusione tra ricerca semantica e full-text.

Le configurazioni che gestiscono aspetti più tecnici, non separabili dalla tecnologia utilizzata, sono invece oggetti distinti e specifici per la ricerca full-text e per la ricerca semantica.

La configurazione per la ricerca semantica specifica le seguenti informazioni:

- il tipo di vettore a cui convertire il vettore di embedding durante la fase di oversampling;
- la distanza da utilizzare in fase di oversampling;
- il numero di record da sommare al numero di risultati richiesti, in fase di oversampling;
- il threshold, espresso come semplice valore in virgola mobile;
- la dimensione dei vettori di embedding;
- il tipo reale del vettore così come memorizzato nel database;
- il tipo di distanza da applicare durante il rescoring.

L'oggetto che gestisce concretamente il threshold in funzione del tipo di distanza utilizzato viene assemblato solo quando necessario, poiché deriva dalla combinazione del threshold, espresso come semplice valore, con il tipo di distanza applicato in quel contesto.

5 Principi e caratteristiche del sistema

La configurazione per la ricerca full-text contiene invece le seguenti informazioni:

- la funzione di ranking da utilizzare, `ts_rank` oppure `ts_rank_cd`;
- l'elenco delle tsquery da utilizzare nelle funzioni di ranking, necessario per gestire la composizione di una funzione di ranking più avanzata;
- un fattore opzionale per la normalizzazione dei punteggi che, come indicato dalla stessa documentazione di Postgres, ha un'utilità relativamente limitata;
- la clausola da utilizzare per il filtro full-text, che può essere una tsquery oppure una overlap query.

Per la ricerca ibrida, un ulteriore oggetto di configurazione definisce:

- l'oversampling applicato ai risultati precedentemente alla fusione, descritto nella sezione dedicata alla ricerca ibrida;
- la costante k che regola il comportamento della fusione RRF, il cui valore standard è 60[18], pur lasciando la possibilità di modificarlo.

Le diverse sotto-ricerche che compongono una ricerca, descritte nella pipeline di esecuzione, vengono costruite come un unico blocco di query, eseguito una sola volta, coerentemente con il requisito **RCM11**.

Il backend deve inoltre essere asincrono, per supportare l'accesso concorrente di più utenti senza che l'elaborazione di una richiesta blocchi le altre.

Anche la fase di ingestion adotta un principio di riduzione del numero di chiamate, applicato in modo trasversale ai diversi servizi coinvolti: i testi vengono organizzati e inviati a blocchi verso il modello di embedding remoto, così da pagare un costo di attesa complessivo inferiore rispetto all'esecuzione di numerose chiamate singole. Lo stesso principio si applica alle eventuali funzionalità di elaborazione a batch offerte dagli strumenti di language detection e alle scritture verso il database, effettuate a blocchi anziché tramite una serie di chiamate singole.

5.3 Principi del sistema di test

In questa sezione vengono descritti i principi guida e le caratteristiche del sistema di test.

5.3.1 Definizione modello dati

Il modello dati del sistema di test è composto da due parti: una riadattata dal modello dati del sistema di ricerca e una dedicata alla gestione della ground truth e del logging, trattata in **Sezione 5.3.4**.

Per quanto riguarda la parte riadattata, il sistema di test adotta una versione semplificata del modello dati del sistema di ricerca: la definizione delle entità è la medesima, ma vengono meno i dettagli legati alla

5 Principi e caratteristiche del sistema

configurabilità, quali le configurazioni di ricerca e i vincoli relazionali. Anche la struttura delle ricerche è la stessa di quella descritta per il sistema di ricerca; rimangono solo le parti ritenute utili per la finalità di allineamento all'ambiente di test.

Questa scelta è il risultato di un riutilizzo di comodità del codice esistente: i due progetti restano comunque indipendenti a livello di codice; l'unico punto di contatto tra i due sistemi è l'interfaccia API del sistema di ricerca, utilizzata dal sistema di test come da qualunque altro utente.

5.3.2 Metriche e il loro significato

Le metriche di valutazione trattate in questa sezione non sono state definite autonomamente, ma fornite durante un colloquio con il tutor aziendale, secondo la seguente definizione e significato.

Le metriche valutate sono le seguenti:

- **retrieval latency**: indica il tempo trascorso tra l'invio di una richiesta di ricerca e la ricezione della relativa risposta.
- **retrieval answer rate**: indica la frequenza con cui la ground truth attesa per una query è stata restituita tra i risultati, indipendentemente dalla posizione occupata.
- **retrieval mean reciprocal rank**: indica la media dei reciproci della posizione in cui la ground truth compare tra i risultati restituiti.
- **retrieval hitrate@1**: indica la frequenza con cui la ground truth viene restituita come primo risultato.
- **retrieval hitrate@5**: indica la frequenza con cui la ground truth viene restituita tra i primi cinque risultati.
- **retrieval hitrate@10**: indica la frequenza con cui la ground truth viene restituita tra i primi dieci risultati.
- **retrieval wins**: indica il numero di volte in cui una ricerca ha prodotto almeno un risultato.
- **retrieval not found**: indica il numero di volte in cui una ricerca non ha prodotto alcun risultato.

La differenza tra retrieval answer rate e retrieval wins, per quanto sottile, è particolarmente significativa: un answer rate basso a fronte di un numero elevato di wins indica che il sistema restituisce con frequenza risultati che, pur presenti, non sono rilevanti rispetto alla query.

5.3.3 Caratteristiche del backend

Il backend simula un insieme di utenti paralleli tramite Locust, secondo quanto richiesto dal requisito **RCM13**. Le richieste non vengono eseguite direttamente dagli utenti simulati, ma tramite una singola porta inbound (**run_one**), che preleva una query da una coda, la esegue e ne registra il risultato.

5 Principi e caratteristiche del sistema

Il backend non calcola direttamente le metriche di valutazione, ma si limita a registrare i risultati grezzi delle ricerche eseguite.

5.3.4 Caratteristiche del database

Il sistema di test adotta due meccanismi di persistenza distinti, ciascuno scelto in base alla natura dell'accesso richiesto.

Il registro delle query da eseguire, comprensivo della relativa ground truth, è mantenuto in un file JSONL: una soluzione ritenuta sufficiente data la natura strettamente sequenziale della sua lettura.

I risultati delle ricerche eseguite vengono invece registrati in un database Postgres, necessario per gestire in modo affidabile le scritture concorrenti provenienti dai diversi utenti simulati. Lo stesso database viene inoltre utilizzato per automatizzare il calcolo delle metriche, esposte sotto forma di viste, e può essere monitorato tramite Grafana.

5 Principi e caratteristiche del sistema

Capitolo 6

Implementazione del sistema di ricerca

In questo capitolo approfondisco le fasi di sviluppo del progetto legate al sistema di ricerca, descrivendo le scelte implementative concrete e le problematiche affrontate nella sua realizzazione.

6.1 Architettura del codice

La progettazione e la codifica seguono i principi dell'architettura esagonale. Il codice è quindi organizzato nelle seguenti categorie:

- inbound adapter,
- outbound adapter,
- inbound port,
- outbound port,
- service,
- classi di dominio.

La *composition root* è gestita tramite FastAPI.

Ciascuna categoria è ulteriormente suddivisa per funzionalità: ingestion e le diverse tipologie di ricerca dispongono ciascuna dei propri adapter, service e port dedicati. Fanno eccezione le classi di dominio condivise, trattate nella **Sezione 6.1.1** a loro dedicata.

6.1.1 Classi di dominio condivise

Le classi di dominio condivise rappresentano il modello dati descritto nella **Sezione 5.2.1** e costituiscono la fonte di verità del sistema. Vengono costruite e validate una sola volta nella composition root, da lì iniettate nelle componenti del sistema che necessitano di conoscere il modello dati.

EntityName e **FieldName** sono due semplici wrapper attorno a una stringa, adottati per rendere il codice più leggibile e per impedire, a livello di firma, di confondere un identificativo di entità con uno di campo o con una stringa qualunque.

6 Implementazione del sistema di ricerca

FieldReference rappresenta un riferimento a un campo specifico di un'entità specifica, tramite la coppia nome dell'entità e nome del campo.

FieldDefinition descrive un campo dati di un'entità: nome, tipo e obbligatorietà, coerentemente con quanto già descritto nel modello dati.

FieldRole e **FieldType** sono due enumerazioni che rappresentano rispettivamente i due ruoli supportati (searchable, filterable) e i tipi di dato supportati per un campo.

Entity rappresenta una singola entità dello schema, aggregando l'insieme dei suoi campi, i campi identificativi e la configurazione di ricerca (ruoli e pesi per campo). In fase di costruzione, Entity verifica la biimplicazione tra ruolo searchable e tipo di campo suddiviso in chunk, già descritta come principio di progetto: un campo è marcato searchable se e solo se il suo tipo è testo suddiviso in chunk.

6 Implementazione del sistema di ricerca

```
1 @dataclass(frozen=True, slots=True)
2 class Entity:
3     entity_name: EntityName
4     identifier: frozenset[FieldName]
5     field_definitions: Mapping[FieldName, FieldDefinition]
6     entity_search_configuration: EntitySearchConfiguration
7
8     def __post_init__(self) -> None:
9         :
10         for field_name, role in
11             self.entity_search_configuration.roles.items():
12             if role == FieldRole.SEARCHABLE and
13                 definition.field_type != FieldType.CHUNKED_TEXT:
14                 raise
15                 UnsupportedFieldTypeForSearchableRoleError(
16                     f"Il campo {field_name!r} di tipo
17                     {definition.field_type!r} è marcato "
18                     f"SEARCHABLE, ma solo
19                     {FieldType.CHUNKED_TEXT!r} è supportato
20                     per questo ruolo."
21                 )
22             if definition.field_type ==
23                 FieldType.CHUNKED_TEXT and role !=
24                 FieldRole.SEARCHABLE :
25                 raise NonSearchableChunkTextError(
26                     f"Il campo {field_name!r} di tipo
27                     {definition.field_type!r} non è marcato "
28                     f"SEARCHABLE, ma {FieldType.CHUNKED_TEXT!r}
29                     deve essere searchable."
30                 )
31         :
```

Codice 2: Entity - dataclass e vincolo searchable/chunked_text

MergeWeights rappresenta i pesi utilizzati per la fusione tra ricerca semantica e full-text, con l'unico vincolo che non possano essere negativi.

6.1.1.1 Vincoli sullo schema

Il punto di estensione esplicito descritto nel modello dati, pensato per accomodare vincoli non necessariamente relazionali, è realizzato tramite il pattern *Visitor*. **SchemaConstraint** è l'interfaccia astratta comune a ogni tipo di vincolo; **RelationalSchemaConstraint** è l'unica imple-

6 Implementazione del sistema di ricerca

mentazione concreta attualmente presente e rappresenta l'equivalente concettuale di una chiave esterna tra due entità.

```
1 class SchemaConstraint(ABC):
2     @abstractmethod
3     def accept(self, visitor: "SchemaConstraintVisitor") ->
      None:
4         ...
5 class SchemaConstraintVisitor(ABC):
6     @abstractmethod
7     def visit_relational_constraint(self, constraint:
      "RelationalSchemaConstraint") -> None: ...
```

Codice 3: SchemaConstraint e RelationalSchemaConstraint

Ogni tipo di vincolo espone un metodo `accept`, che delega a un'implementazione di **SchemaConstraintVisitor** l'operazione specifica per quel tipo. L'introduzione di un nuovo tipo di vincolo richiede quindi un nuovo metodo `visit` dedicato sull'interfaccia del visitor: da quel momento, qualunque visitor che non lo implementi non può più essere istanziato e l'errore emerge già in fase di costruzione, non alla prima volta in cui il visitor incontra quel tipo di vincolo a runtime.

6.1.1.2 Ricerca linked e struttura a grafo

GraphEdge rappresenta un arco del grafo di navigazione tra entità utilizzato dalla ricerca linked, collegando due `FieldReference` in entità diverse; un arco non può collegare un'entità a se stessa, per evitare cicli non gestiti nell'attraversamento.

LinkedSearchConfiguration rappresenta l'intero grafo di navigazione, come mappa di adiacenza tra entità e archi uscenti, insieme ai pesi utilizzati nella fusione dei risultati tra entità diverse. In fase di costruzione viene verificata l'aciclicità del grafo; una volta garantita, per ciascuna entità vengono precalcolati tutti i cammini possibili verso la propria radice, rappresentati come sequenze ordinate di `GraphEdge` (**TraversalPath**).

6 Implementazione del sistema di ricerca

```
1 @dataclass(frozen=True, slots=True) Python
2 class LinkedSearchConfiguration:
3     adjacency: Mapping[EntityName, Sequence[GraphEdge]]
4     field_weights: LinkedFieldWeights
5     _paths_to_root_cache: Mapping[EntityName,
6     tuple[TraversalPath, ...]] = MappingProxyType({})
7
8 class TraversalPath:
9     hops: tuple[GraphEdge, ...]
10    @property
11    def origin_entity(self) -> EntityName:
12        :
13    @property
14    def root_entity(self) -> EntityName:
15        :
16    @property
17    def intermediate_entities(self) ->
18    tuple[EntityName, ...]:
19        :
20    @property
21    def entities_in_order(self) -> tuple[EntityName, ...]:
22        :
```

Codice 4: LinkedSearchConfiguration e TraversalPath

È importante notare che LinkedSearchConfiguration si limita a esporre, per ciascuna entità, l'insieme di tutti i cammini possibili verso la radice: non seleziona autonomamente un unico cammino. La regola del primo cammino valido, descritta nella [Sezione 5.2.3.4](#), viene applicata a valle, dal query builder che genera la query SQL della ricerca linked, sulla base dei cammini restituiti da questa classe. Questa separazione è coerente con il principio di disaccoppiamento già discusso a proposito dei vincoli di integrità: il dominio si limita a descrivere le possibilità strutturali, mentre la logica di scelta concreta, che dipende dai dati effettivamente presenti, è responsabilità di un livello successivo.

6.1.1.3 Radice dell'aggregato

SchemaConfiguration è la *radice dell'aggregato*: raccoglie l'insieme delle entità, dei vincoli di schema e la configurazione di ricerca linked in un'unica struttura immutabile. Le entità sono rappresentate come mappa da EntityName a Entity, anziché come semplice sequenza, per

6 Implementazione del sistema di ricerca

garantire per costruzione l'assenza di duplicati e un accesso diretto in fase di risoluzione dei riferimenti.

```
1 @dataclass(frozen=True, slots=True)
2 class SchemaConfiguration:
3     entities: Mapping[EntityName, Entity]
4     schema_constraints: Sequence[SchemaConstraint]
5     linked_search_configuration: LinkedSearchConfiguration
```

Codice 5: SchemaConfiguration - dataclass

SchemaConfiguration valida, in fase di costruzione, esclusivamente la coerenza strutturale locale (presenza di almeno un'entità, coerenza tra chiave e valore nella mappa delle entità). La validazione dell'integrità referenziale più profonda, l'esistenza e la compatibilità di tipo dei campi citati nei vincoli, l'esistenza delle entità citate nella configurazione di ricerca linked richiede invece una visione d'insieme dell'intero schema, non disponibile al singolo oggetto preso in isolamento, ed è per questo demandata a **SchemaConfigurationBuilder**.

Il builder accumula incrementalmente entità e vincoli, per poi eseguire, al momento della costruzione finale, l'intera catena di validazioni referenziali: la verifica dei vincoli di schema tramite il visitor descritto in precedenza, la coerenza della configurazione di ricerca linked rispetto alle entità effettivamente presenti e la corrispondenza reciproca tra campi searchable e pesi configurati per la ricerca linked. Solo un'istanza di SchemaConfiguration prodotta da questo builder può quindi considerarsi garantita come valida nella sua interezza.

```
1 class SchemaConfigurationBuilder:
2
3     def build(self) -> SchemaConfiguration:
4         :
```

Codice 6: SchemaConfigurationBuilder - firma di build()

6.2 Sistema di persistenza

Il diagramma in **Figura 25** rappresenta lo schema entità-relazione concreto adottato in questo progetto, coerente con il modello dati del service desk HDA descritto nel capitolo precedente. Trattandosi di un sistema configurabile, entità e campi possono essere personalizzati a seconda del contesto applicativo; lo schema qui presentato è quindi una delle possibili istanze concrete del modello dati, non un vincolo strutturale del sistema.

Inoltre si precisa che l’inizializzazione del database non avviene tramite codice SQL scritto a mano, ma tramite script che leggono la schema configuration e altre configurazioni, come quella di pgvector. Questo è per comodità nella modifica dei parametri per ulteriori test futuri. Non è stata dedicata particolare cura a questo script, in quanto reputato esterno allo scopo del tirocinio ma una semplice comodità per il testing: vi è ampio margine di miglioramento.

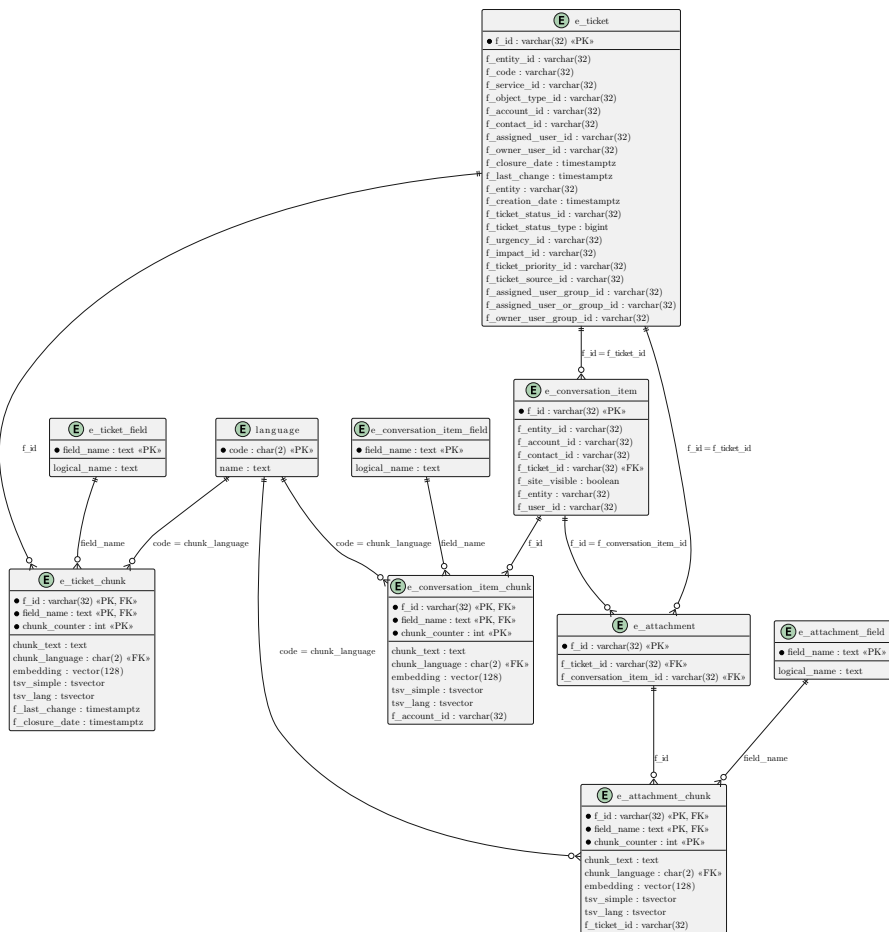


Figura 25: Diagramma ER del core del database

6 Implementazione del sistema di ricerca

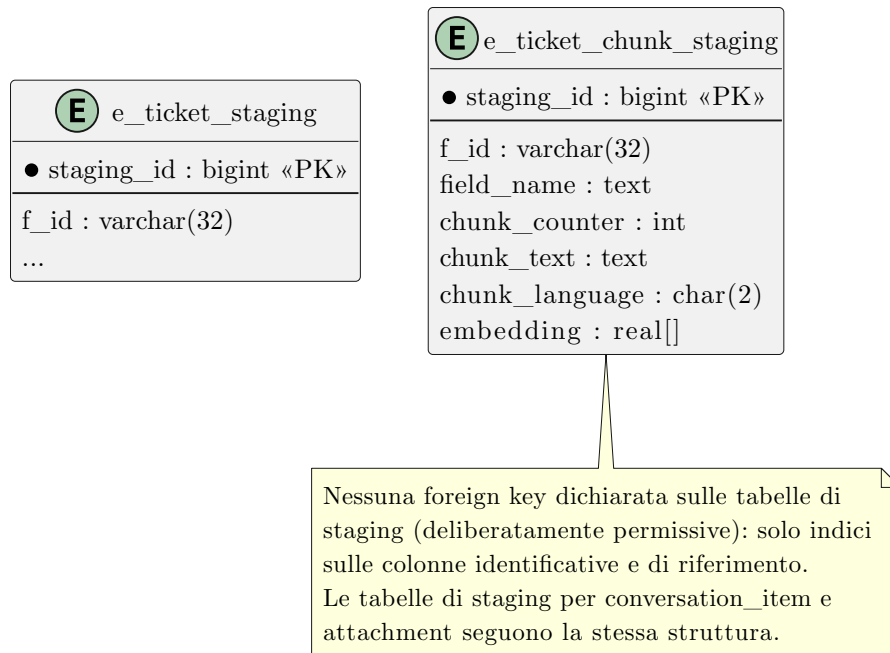


Figura 26: Diagramma ER delle tabelle di supporto allo staging

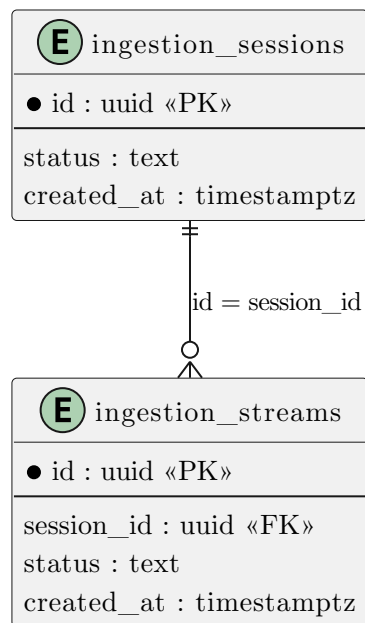


Figura 27: Diagramma ER delle tabelle di supporto al tracking

6 Implementazione del sistema di ricerca

Alcuni aspetti rilevanti dello schema non sono rappresentabili graficamente in un diagramma entità-relazione; vengono quindi descritti di seguito: il partizionamento delle tabelle e gli indici definiti su di esse.

Coerentemente con quanto descritto nella **Sezione 5.2.3.1** in cui viene analizzata la ricerca semantica, la tabella dei chunk di ciascuna entità è partizionata per lista sul campo di provenienza del testo (**field_name**): ogni campo searchable dell'entità corrisponde a una partizione distinta. Su ciascuna tabella dei chunk sono definiti i seguenti indici:

1. Gli indici GIN «classici» sulle colonne **tsv_simple** e **tsv_lang**, che servono a velocizzare il filtering delle query full-text implementate da Postgres. Per **tsv_lang** l'indice è ulteriormente suddiviso in un indice parziale per lingua, filtrato sul valore di **chunk_language**;
2. Gli indici GIN con opclass **array_ops** sull'espressione **tsvector_to_array(...)** delle colonne **tsv_simple** e **tsv_lang**, utilizzati dal filtro di corrispondenza minima descritto in **Sezione 5.2.3.2.1**, che si appoggia all'operatore di overlap tra array anziché agli operatori di match della ricerca full-text. Anche questi indici sono suddivisi per lingua sulla colonna **tsv_lang**, visto che il lookup è sempre filtrato per lingua;
3. un indice HNSW sulla colonna **embedding**, utilizzato dalla ricerca semantica. L'indice non è necessariamente costruito sui valori a piena precisione della colonna: la configurazione applicativa può specificare un tipo di vettore e una distanza «candidati», eventualmente più leggeri (ad esempio una *quantizzazione binaria* con *distanza di Hamming*), utilizzati per generare rapidamente l'insieme di candidati su cui viene poi eseguito l'oversampling e il rescoring finale sui valori reali. Nella configurazione concreta di questo progetto, l'indice è costruito su una quantizzazione binaria dell'embedding, con distanza di Hamming.

Tutti gli indici elencati sono creati sulla tabella partizionata: Postgres li propaga automaticamente a ciascuna partizione.

Un ulteriore indice, non legato alle tabelle dei chunk, è definito come univoco su un'espressione costante, sulla tabella delle sessioni di ingestion, filtrato sulle sole righe con stato aperto o chiuso: questo garantisce, a livello di database e non solo applicativo, che possa esistere al più una sessione attiva alla volta, coerentemente con il principio già descritto nella **Sezione 5.2.2.1**.

6.3 Ingestion

La funzionalità di ingestion espone quattro porte inbound, ciascuna dedicata a una funzione specifica:

1. avvio della sessione di ingestion,

6 Implementazione del sistema di ricerca

2. chiusura della sessione di ingestion,
3. lettura dello stato della sessione di ingestion (in corso o terminata),
4. elaborazione di uno stream di batch di dati grezzi.

L'avvio e la chiusura scrivono sulla tabella delle sessioni di ingestion, che funge da fonte di verità esterna al processo: qualsiasi worker FastAPI, nel tentativo di aprire o chiudere una sessione, osserva lo stesso database e le stesse informazioni, indipendentemente da quale istanza dell'applicazione lo serva. La chiusura, in particolare, avvia anche il processo di promozione da staging a tabelle reali, descritto più avanti in questa sezione.

6.3.1 Elaborazione dei batch e uso dello stream

Non vi è contraddizione tra l'uso del batching interno e il fatto che la porta di elaborazione accetti uno stream: il sistema ottimizza operazioni come il calcolo degli embedding, il rilevamento della lingua e la scrittura sul database per i singoli batch di record, ma la porta stessa espone un iteratore asincrono di batch in ingresso, così da poter gestire un flusso di dati potenzialmente continuo.

6.3.2 Validazione, arricchimento e scrittura

Per rappresentare i valori dei campi si è scelto di adottare un'astrazione dedicata, **FieldValue**, invece di un insieme eterogeneo di tipi primitivi: l'obiettivo principale è evitare la proliferazione di tipi primitivi diversi all'interno del dominio. A partire da questa astrazione sono definite union type dedicate, rispettivamente per i dati in ingresso e per i dati destinati alla scrittura sul database.

Per ciascun batch dello stream in ingresso, il service esegue in sequenza:

1. **validazione**, verifica la coerenza dei dati ricevuti con la schema configuration e scarta i record non conformi. È in questa fase che avviene anche il casting dei tipi non deducibili nell'adapter inbound: il tipo di un dato viene normalmente dedotto dal suo formato, ma per i valori data/ora la conversione da stringa a datetime può essere ambigua (un campo di tipo testo il cui valore ha una forma di data, se convertito, genererebbe errori più avanti). Il service applica quindi questa trasformazione solo dove necessario, guidato dalla schema configuration;
2. **arricchimento**, un processor riceve alla costruzione le porte out-bound per la language detection e per il calcolo degli embedding; aggrega le liste di testi da passare a ciascuna porta e costruisce i **refined_entity_batch**. Se la lingua è già nota a priori viene usata quella, altrimenti si ricorre a language detection; i record per cui l'arricchimento fallisce vengono scartati;

6 Implementazione del sistema di ricerca

3. **scrittura**, i record rimanenti vengono scritti sulle tabelle di staging tramite comando **COPY**, attraverso una porta outbound write repository dedicata.

Per notificare gli scarti a ciascuno di questi passaggi viene usata una classe **RejectedRecord**, che contiene l'identificativo del record e il motivo del fallimento. Il metodo che orchestra l'intera pipeline accetta in input un **AsyncIterator** di batch grezzi e restituisce in output un **AsyncIterator** di **RejectedRecord**, che l'adapter **FastAPI** inoltra al chiamante come streaming response.

```
1 class IngestBatchStreamUseCase(ABC):  
2     @abstractmethod  
3     async def ingest_batch_stream(  
4         self, command: IngestBatchStreamCommand  
5     ) -> AsyncIterator[RejectedRecord]:...
```

Python

Codice 7: Firma porta di ingestion dei dati

Lo stato di scrittura di ciascuno stream viene inoltre tracciato tramite una porta outbound dedicata, in modo da garantire che, al momento della chiusura della sessione, tutti gli stream in corso abbiano effettivamente terminato la scrittura prima di avviare la promozione.

Il caso d'uso di lettura dello stato di attività della sessione di ingestion serve esclusivamente al sistema di test, per determinare se durante l'esecuzione di una query sia in corso un processo di ingestion (si veda il [Capitolo 7](#)).

6.3.3 Promozione da staging a tabelle reali

Lo scheduling della promozione è calcolato in base ai vincoli relazionali definiti nella schema configuration, dando priorità prima alle tabelle dei metadati e poi a quelle dei chunk, in modo da rispettare le dipendenze referenziali. La promozione avviene interamente lato database, tramite una query che seleziona i dati validi e li trasferisce sulla tabella reale senza farli transitare da Python.

```
1 WITH staging_window AS (  
2     SELECT staging_id FROM {source_table}  
3     WHERE {where_clause}  
4     ORDER BY staging_id  
5     LIMIT {self._batch_size}  
6 ),
```

SQL

Codice 8: Staging promotion - selezione degli elementi candidati

6 Implementazione del sistema di ricerca

Questo primo CTE seleziona fino a `self._batch_size` elementi candidati alla promozione, cioè rimovibili dalla tabella di staging e inseribili nella tabella reale. Il sistema si occupa solo di costruire un'opportuna `where_clause`, che verifica che, dopo la promozione, restino rispettati i vincoli di integrità referenziale: ad esempio, per l'entità radice (i ticket) non è necessaria alcuna clausola aggiuntiva, mentre per un'entità figlia (i conversation item) viene richiesto che il ticket a cui fanno riferimento esista già nella tabella reale.

```
1 batch AS (SQL
2     SELECT DISTINCT ON ({conflict_columns}) s.staging_id
3     FROM {source_table} s
4     JOIN staging_window w ON s.staging_id = w.staging_id
5     ORDER BY {conflict_columns}, s.staging_id DESC
6 ),
```

Codice 9: Staging promotion - gestione dei duplicati

La staging window viene qui espansa a tutti i possibili duplicati degli elementi selezionati e ordinata in ordine decrescente di `staging_id`. Questo, combinato con l'autoincremento dello `staging_id` e con `DISTINCT ON`, garantisce che venga selezionata solo la versione più recente di ciascun dato, anche in presenza di più caricamenti duplicati dello stesso record.

Questa soluzione non è riconosciuta come ottimale, ma è funzionalmente corretta e, per un progetto di natura esplorativa, sufficiente: lo staging, per quanto utile alla robustezza del caricamento, non è centrale al problema di ricerca affrontato dal tirocinio. Viene qui esplicitamente riconosciuto come debito tecnico, da sanare in eventuali evoluzioni successive del sistema.

```
1 promoted AS (SQL
2     DELETE FROM {source_table}
3     WHERE staging_id IN (SELECT staging_id FROM
4                           staging_window)
5     RETURNING staging_id, {column_list}
6 ),
```

Codice 10: Staging promotion - estrazione dei valori

Questo CTE rimuove dalla tabella di staging gli elementi candidati e ne salva temporaneamente il valore, tramite la clausola `RETURNING`, per il passo successivo.

6 Implementazione del sistema di ricerca

```
1 inserted AS (SQL
2     INSERT INTO {target_table} ({column_list})
3     SELECT {column_list} FROM promoted
4     WHERE staging_id IN (SELECT staging_id FROM batch)
5     ON CONFLICT ({conflict_columns}) {conflict_action}
6     RETURNING 1
7 )
```

Codice 11: Staging promotion - inserimento nella tabella reale

Questo passo carica sulla tabella reale solo gli elementi già filtrati dal CTE batch (cioè una versione per ciascuna chiave), specificando come gestire eventuali conflitti con righe già presenti nella tabella reale. Da non confondere con la rimozione dei duplicati interni allo staging, gestita al passo precedente. L'azione concreta in caso di conflitto (tipicamente un *upsert*) è definita dal backend, non hard-coded nella query.

```
1 SELECT count(*) FROM staging_windowSQL
```

Codice 12: Staging promotion - conteggio degli elementi processati

La query restituisce infine il numero di elementi processati nella finestra, non il numero di record effettivamente inseriti: la promozione viene rieseguita finché la query non restituisce 0, cioè finché non rimangono più elementi promuovibili nella tabella di staging.

6.4 Ricerca

Ogni tipologia di ricerca è esposta tramite un endpoint dedicato, realizzato con un adapter FastAPI specifico.

Le classi di dominio impiegate differiscono tra ricerca su singola entità e ricerca linked: quest'ultima richiede un filtro con struttura annidata più complessa, che abbina un filtro a ciascuna entità coinvolta nell'attraversamento. Per evitare una duplicazione eccessiva di codice tra le due varianti, senza però introdurre relazioni di subtyping scorrette, si è adottato l'uso di **Generic** e **type alias**: i Generic permettono il riuso della logica di dominio comune, mentre i type alias vengono usati da tutte le classi esterne alla catena di generici per facilitare eventuali modifiche future. Questa scelta si è rivelata utile concretamente durante la realizzazione della ricerca linked, che ha richiesto di distinguere due serie di filtri distinte a partire dalla stessa gerarchia generica.

Per questo motivo, nel seguito vengono descritte solo le parti generiche condivise e un esempio di come vengono specializzate tramite type alias.

6 Implementazione del sistema di ricerca

Le parti che si sono discostate da questo pattern sono state trattate esplicitamente.

6.4.1 Filtering

Il filtro adotta una struttura ad albero: viene definita un'interfaccia comune, **FilterCondition**, con quattro implementazioni concrete: **AtomicExpression**, **AndCondition**, **OrCondition**, **NotCondition**.

```
1  class FilterCondition(ABC, Generic[R]):
2      @abstractmethod
3      def accept(self, visitor: FilterVisitor[R, T]) -> T: ...
4
5  @dataclass(frozen=True, slots=True)
6  class AtomicExpression(FilterCondition[R], Generic[R]):
7      field: R
8      operator: Operator
9      value: FieldValue
10     def accept(self, visitor: FilterVisitor[R, T]) -> T:
11         return visitor.visit_atomic_expression(self)
12
13 @dataclass(frozen=True, slots=True)
14 class AndCondition(FilterCondition[R], Generic[R]):
15     terms: tuple[FilterCondition[R], ...]
16     def accept(self, visitor: FilterVisitor[R, T]) -> T:
17         return visitor.visit_and(self)
18
19 @dataclass(frozen=True, slots=True)
20 class OrCondition(FilterCondition[R], Generic[R]):
21     terms: tuple[FilterCondition[R], ...]
22     def accept(self, visitor: FilterVisitor[R, T]) -> T:
23         return visitor.visit_or(self)
24
25 @dataclass(frozen=True, slots=True)
26 class NotCondition(FilterCondition[R], Generic[R]):
27     term: FilterCondition[R]
28     def accept(self, visitor: FilterVisitor[R, T]) -> T:
29         return visitor.visit_not(self)
```

Codice 13: Ricerca - interfaccia FilterCondition e implementazioni concrete

6 Implementazione del sistema di ricerca

Questa gerarchia ha il solo scopo di rappresentare e comporre il filtro, non di applicarlo direttamente a un valore. Per applicarlo si usa quindi, anche qui, il pattern Visitor: concretamente, un'implementazione di **FilterVisitor** viene usata per validare il filtro all'interno dei service di dominio e un'altra per costruire le clausole **WHERE** all'interno degli adapter SQL, sia per il filtro su singola entità, sia per il post-join filter della ricerca linked.

```
1 SingleEntityFilter = FilterCondition[FieldName]
2
3 LinkedPostJoinFilter = FilterCondition[FieldReference]
```

Codice 14: Ricerca - type alias per filtro su singola entità e post-join

L'uso di Generic e type alias, anziché di un'interfaccia comune implementata separatamente da `FilterCondition[FieldName]` e `FilterCondition[FieldReference]`, permette un discreto riuso di codice senza introdurre vincoli di subtyping che sarebbero stati concettualmente scorretti: le due specializzazioni non sono l'una sottotipo dell'altra, condividono solo la struttura.

6.4.2 Query e specializzazioni

```
1 @dataclass(frozen=True, slots=True)
2 class Query(Generic[R]):
3     return_fields: frozenset[R]
4     filter: FilterCondition[R] | None = None
5
6
7 SingleEntityQuery = Query[FieldName]
8
9
10 class LinkedQuery:
11     return_fields: frozenset[FieldReference]
12     filter: LinkedFilter = LinkedFilter({})
```

Codice 15: Ricerca - Query e specializzazioni

In `LinkedQuery` si osserva il primo punto di disallineamento tra la gerarchia generica e la sua specializzazione linked: è dovuto alla necessità, propria della sola ricerca linked, di abbinare un filtro distinto a ciascuna entità coinvolta, tramite `LinkedFilter`, anziché un unico filtro sull'intera query.

6 Implementazione del sistema di ricerca

```
1 @dataclass(frozen=True, slots=True)
2     class QueryResult(Generic[R]):
3         results: tuple[Mapping[R, FieldValue], ...]
4
5     @dataclass(frozen=True, slots=True)
6     class SimilaritySearchResult(Generic[R]):
7         query_result: QueryResult[R]
8         matched_field: R
9         chunk_counter: int
10        matched_text: str
11        score: float
12
13    LinkedQueryResult = QueryResult[FieldReference]
14    LinkedSimilaritySearchResult =
        SimilaritySearchResult[FieldReference]
```

Codice 16: Ricerca - Query result e specializzazioni

Invece nei risultati della ricerca il riuso del codice viene applicato direttamente in modo pulito.

La query, così come ricevuta dall'adapter inbound, può contenere campi con valore nullo, ad esempio i pesi di fusione o il numero di risultati desiderati (**top_k**). È compito del service, in fase di validazione, completare questi campi quando assenti, attingendo ai valori di default configurati; il service non esegue invece language detection, che rimane responsabilità del solo processor di ingestion. Ogni tipologia di ricerca dispone infine di un proprio command dedicato, usato per comunicare con la rispettiva porta outbound. Il risultato contiene invece, oltre ai normali risultati della query, per tutte le corrispondenze trovate le informazioni richieste dalla query, con l'aggiunta del testo che ha dato origine al match e le informazioni relative al campo dati di origine e al numero di chunk; il punteggio è più una comodità ai fini di debug.

Ogni tipo di ricerca ha un suo use case dedicato; lo stesso vale per i service e per gli adapter inbound. Sono contati sei tipi di ricerca perché semantica, full-text e ibrida vanno contati separatamente sia per la ricerca su singola entità che per la ricerca linked. Il calcolo degli embedding per la ricerca semantica e ibrida avviene sfruttando la stessa porta outbound utilizzata in fase di ingestion dei dati. L'interazione con il database avviene sempre tramite porte in accordo con i principi dell'architettura esagonale.

6 Implementazione del sistema di ricerca

6.4.3 Ricerca semantica

La ricerca semantica opera in due fasi, coerentemente con quanto descritto nella **Sezione 5.2.3.1**: viene eseguita una query per ciascuna partizione della tabella dei chunk coinvolta (una per ciascun campo searchable interessato dalla ricerca), i cui risultati vengono poi combinati.

```
1  WITH candidates_{field_name} AS (
2      SELECT
3          id, field_name, chunk_counter, chunk_text,
4          -- il -1 è necessario perché pgvector calcola l'inner
           product negato
5          -1 * (embedding <#> {embedding}) *
           {weight_field_name} AS refined_score
6      FROM {chunk_ticket_table}
7      WHERE field_name = {field_name}
8          AND binary_quantize(embedding) <~>
           binary_quantize({embedding}) <= {threshold}
9          -- AND {eventuali clausole aggiuntive specificate dal
           filtro}
10     ORDER BY binary_quantize(embedding) <~>
           binary_quantize({embedding}) ASC
11     LIMIT {k + oversampling}
12 ),
```

Codice 17: Ricerca semantica - selezione dei candidati da una singola partizione

Questo primo CTE viene generato una volta per ciascun campo searchable coinvolto nella ricerca (`candidates_{field_name_1}`, `candidates_{field_name_2}`, ...); l'ORDER BY/LIMIT opera sulla distanza approssimata quantizzata, in modo da sfruttare l'indice HNSW descritto nella **Sezione 6.2**, mentre `refined_score` viene già calcolato sul valore esatto per il passo di combinazione successivo.

6 Implementazione del sistema di ricerca

```
1  -- segue dai CTE candidates_{field_name_1..n} definiti sopra
2  candidates AS (
3      SELECT * FROM candidates_{field_name_1}
4      UNION ALL
5      SELECT * FROM candidates_{field_name_2}
6      UNION ALL
7      SELECT * FROM candidates_{field_name_3}
8      ORDER BY refined_score {order_clause}
9      LIMIT {k}
10 ),
```

Codice 18: Ricerca semantica - combinazione dei candidati tra partizioni

La combinazione si limita a un'unione tramite **UNION ALL** dei frammenti già ordinati e pesati, seguita dal taglio ai primi k risultati complessivi: il taglio avviene prima del join con la tabella principale, così da eseguirlo solo sulle righe già selezionate come rilevanti, non sull'intero insieme dei candidati.

```
1  -- segue dal CTE candidates definito sopra
2  SELECT {return_fields}
3  FROM {ticket_table}
4  JOIN candidates ON {ticket_table}.id = candidates.id
5  ORDER BY candidates.refined_score {order_clause}
```

Codice 19: Ricerca semantica - join con la tabella principale

Infine si esegue un join con la tabella principale per recuperare i campi richiesti dalla query. L'**ORDER BY** finale va ripetuto esplicitamente anche dopo il join: Postgres non garantisce che l'ordine dei risultati del CTE `candidates` sopravviva al join successivo, quindi il ranking calcolato nel passo precedente va riaffermato per essere preservato nel risultato finale. Per rendere il sistema facilmente riconfigurabile, la descrizione della configurazione di `pgvector` (tipo di distanza, tipo di vettore, soglia utente, fattore di oversampling, ...) è iniettata tramite *dependency injection* di un oggetto dedicato, `PgVectorEngineConfig`, costruito da una `factory` che gestisce automaticamente le variabili d'ambiente.

A supporto di questa configurazione, alcune classi ausiliarie coniugano il comportamento nativo di `pgvector` con le metriche di similarità attese dal dominio, eseguendo le trasformazioni inverse rispetto alle ottimizzazioni applicate in fase di ricerca (ad esempio la conversione tra spazio di distanza grezza, usato internamente da `pgvector`, e spazio di similarità normalizzato, esposto verso l'esterno).

6.4.4 Ricerca full-text

La ricerca full-text adotta un approccio strutturalmente simile alla semantica: partition query per campo searchable, poi combinazione. Non è un vincolo tecnico, ma una scelta di riuso del codice della ricerca semantica: in questo caso la suddivisione per partizione non è strettamente necessaria, ma nemmeno errata. Semplifica la ricerca su sottoinsiemi di campi e l'applicazione dei pesi.

La ranking function nativa di Postgres per la ricerca full-text non offre, a differenza di Elasticsearch, un meccanismo di boosting progressivo: non è possibile, con un'unica chiamata, dare un punteggio alto a un match di frase esatta, uno intermedio a un match su tutte le parole ma non in ordine, uno basso a un match parziale. Per simulare questo comportamento si sommano più ranking function calcolate sulla stessa query: una phrase query e una all-words query, che restituiscono lo stesso punteggio in caso di match di frase, mentre la phrase query restituisce 0 se l'ordine delle parole non è rispettato.

Una seconda limitazione nativa è l'impossibilità di filtrare per un criterio di corrispondenza minima (match di almeno una certa percentuale di parole): anche questo viene reimplementato tramite operazioni sugli array. I lessemi della query vengono estratti direttamente all'interno di Postgres, anziché con uno strumento esterno, per evitare un rischio di stemming incoerente tra la fase di estrazione e quella di confronto.

6 Implementazione del sistema di ricerca

```
1  WITH query_lexemes AS (
2      SELECT arr, cardinality(arr) AS n,
3          LEAST(
4              CASE
5                  WHEN cardinality(arr) > 9 THEN
6                      GREATEST(CEIL(cardinality(arr) * 0.5)::int, 1)
7                  WHEN cardinality(arr) > 4 THEN
8                      GREATEST(CEIL(cardinality(arr) * 0.6)::int, 1)
9                  WHEN cardinality(arr) > 2 THEN
10                     GREATEST(CEIL(cardinality(arr) * 0.9)::int, 1)
11                  ELSE cardinality(arr)
12              END,
13          15) AS k
14  FROM (
15      SELECT array_agg(lex ORDER BY lex) AS arr
16      FROM unnest(tsvector_to_array(to_tsvector('italian',
17          'problema di accesso al portale utente')) AS lex
18  ) AS lexemes
19 ),
```

Codice 20: Ricerca full-text - estrazione dei lessemi e calcolo della soglia di corrispondenza

Questo CTE estrae l'insieme ordinato dei lessemi della query (**arr**), la sua cardinalità (**n**) e il numero minimo di lessemi in comune richiesto per considerare un chunk rilevante (**k**), calcolato con una soglia percentuale decrescente al crescere del numero di parole nella query, comunque limitato a un massimo di 15.

6 Implementazione del sistema di ricerca

```
1  candidates AS MATERIALIZED (
2      SELECT
3          "ticket_id",
4          "chunk_text" AS matched_text,
5          "chunk_counter",
6          (
7              ts_rank_cd("tsv_lang", to_tsquery('italian',
8              (plainto_tsquery('italian', 'problema di accesso al
9              portale utente'))::text || ':*'))
10             + ts_rank_cd("tsv_lang", to_tsquery('italian',
11             (phraseto_tsquery('italian', 'problema di accesso al
12             portale utente'))::text || ':*'))
13             + ts_rank_cd("tsv_lang", to_tsquery('italian',
14             replace(plainto_tsquery('italian', 'problema di accesso
15             al portale utente'))::text, ' & ', ' | ')))
16         ) AS raw_score,
17         cardinality(ARRAY(
18             SELECT unnest(tsvector_to_array("tsv_lang"))
19             INTERSECT
20             SELECT unnest(arr) FROM query_lexemes
21         )) AS matched_count,
22         (SELECT k FROM query_lexemes) AS required_k
23     FROM public.ticket_chunks
24     WHERE "field_name" = 'description'
25           AND tsvector_to_array("tsv_lang") && (SELECT arr[1 :
26           GREATEST(n - k + 1, 1)] FROM query_lexemes)
27           AND "chunk_language" = 'it'
28 )
```

Codice 21: Ricerca full-text - selezione dei candidati per corrispondenza e ranking combinato

Il punteggio (**raw_score**) è la somma delle tre ranking function citate sopra (any-word, phrase, all-words). Il conteggio dei lessemi in comune (**matched_count**), confrontato con la soglia **required_k**, realizza il filtro di corrispondenza minima descritto in [Sezione 5.2.3.2.1](#); la condizione nella clausola WHERE sull'operatore di overlap (&&) applicato a una porzione dell'array dei lessemi della query è un filtro di pre-selezione più permissivo, pensato per sfruttare l'indice GIN con **array_ops** descritto nella sezione sul sistema di persistenza, prima del calcolo esatto di **matched_count**.

6 Implementazione del sistema di ricerca

```
1  SELECT
2    "ticket_id",
3    'description' AS "field_name",
4    matched_text,
5    "chunk_counter",
6    raw_score,
7    1.0::float8 AS field_weight
8  FROM candidates
9  WHERE raw_score >= 0.0
10     AND matched_count >= required_k;
```

Codice 22: Ricerca full-text - filtro finale sulla soglia di corrispondenza

Poiché il conteggio di corrispondenza minima non è una funzionalità nativa della full-text search di Postgres, non è utilizzabile direttamente nella ranking function, ma solo come filtro applicato a valle, con soglie e percentuali configurabili. Rimane comunque possibile, in configurazione, usare una semplice all-words query come filtro; tuttavia, su query lunghe e basi di dati ampie, una condizione di questo tipo da sola non riduce in modo significativo l'insieme di righe da valutare, da qui la necessità del meccanismo sopra descritto.

Il frammento di query appena descritto viene generato per ogni configurazione testuale rilevante ed eseguito ripetutamente: per la ricerca su lingua non nota, la ricerca viene eseguita tre volte: una sul vettore `tsv_simple` (agnostico rispetto alla lingua) e una per ciascuna lingua supportata sul vettore `tsv_lang` (ad esempio una volta con `chunk_language = 'it'` e una con `chunk_language = 'en'`) scorrendo quindi la base di dati più volte; i risultati delle diverse esecuzioni vengono infine ricombinati con `UNION ALL`, con la stessa logica di combinazione descritta per la ricerca semantica.

Sono stati esplorati tre ordinamenti per i lessemi delle frasi:

- ordine lessicografico,
- lunghezza dei lessemi,
- frequenza dei lessemi.

Per applicare il principio di cassette è sufficiente un qualsiasi tipo di ordinamento. Tuttavia le configurazioni testuali semplici non eliminano le stopwords: ciò porta a un elevato numero di match su cui calcolare il punteggio causando un overhead molto alto per l'ordine lessicografico.

Il codice relativo all'ordinamento lessicografico è omissso perché già trattato in **Codice 20**; per gli altri ordinamenti viene mostrata solo la parte differente.

Tramite `EXPLAIN ANALYZE` è stato analizzato anche l'ordinamento per lunghezza: ha prodotto risultati migliori, ma con una consistenza altale-

6 Implementazione del sistema di ricerca

nante. Si è vista una discreta riduzione del candidate pool per ticket e conversation item, ma pressoché nulla sugli attachment.

```
1 ...
2 SELECT array_agg(lex ORDER BY length(lex) DESC) AS arr
```

Codice 23: Ricerca full-text - ordinamento dei lessemi per lunghezza

Invece l'ordinamento per frequenza ha dato risultati significativamente più bassi, portando all'incirca ad un dimezzamento costante sulle ricerche.

```
1 ...
2 SELECT array_agg(lex ORDER BY COALESCE(freq.ndoc, 0) ASC)
   AS arr
3 FROM unnest(tsvector_to_array(to_tsvector(
4   {language.regconfig_literal}, $1"
5   ))) AS lex
6 LEFT JOIN lexeme_frequency freq ON freq.word = lex
```

Codice 24: Ricerca full-text - ordinamento dei lessemi per frequenza

La frequenza dei lessemi però richiede un ulteriore overhead in memoria. Richiede una vista materializzata che va creata esplicitamente, tuttavia ha portato una consistente riduzione del pool di candidati che ha comportato una discreta riduzione dei tempi.

```
1 CREATE MATERIALIZED VIEW lexeme_frequency AS
2   SELECT word, ndoc
3   FROM ts_stat('SELECT tsv_simple FROM e_ticket_chunk
4                   UNION ALL SELECT tsv_simple FROM
5                   e_conversation_item_chunk
6                   UNION ALL SELECT tsv_simple FROM
7                   e_attachment_chunk');
7 CREATE UNIQUE INDEX ON lexeme_frequency (word);
```

Codice 25: Ricerca full-text - Materialized view per le frequenze

6.4.5 Ricerca ibrida

Le query costruite per la ricerca semantica e per la ricerca full-text non vengono eseguite immediatamente al momento della loro costruzione: vengono prima create come frammenti tramite classi helper dedicate

6 Implementazione del sistema di ricerca

e solo in un secondo momento eseguite. Questo disaccoppiamento tra costruzione ed esecuzione è ciò che rende possibile realizzare la ricerca ibrida come reciprocal rank fusion (RRF): i due frammenti vengono avvolti con **RANK() OVER**, per ottenere la posizione in classifica di ciascun motore a partire dal punteggio pesato già calcolato internamente da ciascuna pipeline e poi fusi tramite un **FULL OUTER JOIN** sui campi identificativi dell'entità, anziché con una **UNION ALL**: questo permette a un risultato trovato da un solo motore di comparire comunque nell'output finale, con il contributo dell'altro motore posto a zero tramite **COALESCE**. Il contributo di ciascun motore alla fusione viene pesato secondo MergeWeights: il peso di ciascuna sorgente compare come numeratore nella rispettiva formula RRF, permettendo di dare più importanza alla ricerca semantica o a quella full-text a seconda della configurazione.

```
1  -- segue dai CTE semantic_ranked e full_text_ranked
2  SELECT
3      COALESCE(sem."{return_field}", fts."{return_field}") AS
4          "{return_field}",
5      -- un COALESCE analogo per ciascun campo richiesto in
6      return_fields
7      COALESCE(sem."field_name", fts."field_name") AS
8          "field_name",
9      COALESCE(sem."chunk_counter", fts."chunk_counter") AS
10         "chunk_counter",
11      COALESCE(sem."matched_text", fts."matched_text") AS
12         "matched_text",
13      COALESCE({semantic_weight}::float8 / ({rrf_k}::float8 +
14          sem."rank_position"), 0.0)
15      + COALESCE({text_weight}::float8 / ({rrf_k}::float8 +
16          fts."rank_position"), 0.0)
17      AS weighted_score
18  FROM semantic_ranked sem
19  FULL OUTER JOIN full_text_ranked fts
20      ON sem."{identifier_field}" = fts."{identifier_field}"
21      -- una condizione AND per ciascun campo identificativo
22      dell'entità
23  ORDER BY weighted_score DESC
24  LIMIT {real_limit}
```

Codice 26: Ricerca ibrida - fusione RRF pesata tramite FULL OUTER JOIN

6.4.6 Ricerca linked

La ricerca linked riusa i frammenti di query già descritti per la ricerca su singola entità, generandone uno per ciascuna entità effettivamente cercata: si considerano cioè le sole entità presenti tra i target di ricerca della query, non tutte le entità coinvolte nell'attraversamento. A partire dalla `LinkedSearchConfiguration` e dai `TraversalPath` precalcolati (**Sezione 6.1.1**), per ciascuna entità cercata viene generato un ramo di query per ciascun cammino possibile verso la radice, che risale l'attraversamento tramite una catena di `LEFT JOIN` sulle tabelle reali delle entità intermedie.

```

1  -- CTE entity_search_{entity}, alias "s", prodotta a
   monte da un builder single-entity
2  SELECT
3      '{entity_name}' AS matched_entity,
4      s."field_name", s."matched_text", s."chunk_counter",
5      s."weighted_score",
6      (
7          jsonb_build_object('{entity_name}',
8                              jsonb_build_object({own_field_pairs}))
9          || jsonb_build_object('{hop_1_entity}',
10                                jsonb_build_object({hop_1_field_pairs}))
11          -- un jsonb_build_object analogo per ciascun hop
12          -- successivo del path
13      ) AS linked_data
14 FROM entity_search_{entity} s
15 LEFT JOIN {hop_1_real_table} AS hop1 ON
16 hop1."{hop_1_target_column}" = s."{hop_1_origin_field}"
17 -- un LEFT JOIN analogo per ciascun hop successivo del path
18 WHERE s."{hop_1_origin_field}" IS NOT NULL
19 -- AND s."{sibling_origin_field}" IS NULL per ogni arco che
20 -- precede
21 -- questo hop nell'ordine di priorità dichiarato in
22 -- adjacency
23 -- AND {eventuali termini generati dal post-join filter}

```

Codice 27: Ricerca linked - un ramo di risalita lungo un `TraversalPath`

Ogni ramo produce un'unica colonna JSONB, ottenuta fondendo un `jsonb_build_object` per ciascuna entità effettivamente raggiunta in quel ramo. L'uso di JSONB qui non è una scelta di modellazione del dominio, ma una semplificazione della sola fase di serializzazione: entra in gioco esclusivamente nell'ultimo passo, quando i risultati vengono trasferiti dal database al backend, che li traduce poi nelle proprie classi

6 Implementazione del sistema di ricerca

di dominio. Rappresentare ogni entità come chiave di un oggetto JSON, anziché come gruppo di colonne dedicate, evita di dover dichiarare, per ogni ramo della combinazione finale, colonne per ogni entità possibile del traversal, con alte probabilità di essere nulle: un'entità non raggiunta in un dato ramo semplicemente non compare come chiave.

La clausola **WHERE** del ramo realizza la regola del primo cammino valido già introdotta a proposito di **LinkedSearchConfiguration**: il campo di origine del primo hop dev'essere valorizzato e tutti gli archi che lo precedono nell'ordine dichiarato in **adjacency** devono invece essere nulli, così da garantire che, tra più cammini possibili per una stessa riga, venga sempre seguito quello di priorità più alta. Le righe per cui nessun cammino risulta valido, cioè per cui nessuna FK di primo hop è valorizzata, non vengono scartate: un ramo aggiuntivo, analogo a quello mostrato ma privo di join, le include comunque nel risultato finale, con i soli dati dell'entità cercata.

Una volta effettuati i join, viene applicato il post-join filter descritto nella sezione sul filtering (**LinkedPostJoinFilter**), che opera sui risultati già combinati tra le diverse entità raggiunte in quel ramo, non più sui singoli frammenti di ricerca.

```
1  -- segue dalle CTE entity_search_{entity} e dai rami
   definiti sopra,
2  -- uno per ciascun TraversalPath di ciascuna entità cercata,
3  -- più un ramo senza risalita per le righe prive di un
   cammino valido
4  WITH all_branches AS (
5      ( /* ramo per il primo TraversalPath della prima entità
        cercata */ )
6      UNION ALL
7      ( /* ramo senza risalita della prima entità cercata */ )
8      UNION ALL
9      ( /* rami analoghi per le altre entità cercate */ )
10 )
11 SELECT * FROM all_branches
12 ORDER BY weighted_score DESC
13 LIMIT {top_k}
```

Codice 28: Ricerca linked - combinazione di tutti i rami e ordinamento finale

I rami di tutte le entità cercate vengono infine combinati con una singola **UNION ALL**, ordinati e limitati sul punteggio pesato, stessa logica di fusione già vista per la ricerca semantica e full-text, applicata qui a livello di traversal invece che di singolo campo. L'intera catena di

6 Implementazione del sistema di ricerca

frammenti per singola entità, risalita, filtro post-join e combinazione viene eseguita in un'unica chiamata SQL: la ricerca linked, per quanto concettualmente più complessa, non richiede round-trip aggiuntivi verso il database rispetto alle altre tipologie di ricerca.

6 Implementazione del sistema di ricerca

Capitolo 7

Implementazione del sistema di test

In questo capitolo approfondisco le fasi di sviluppo del progetto legate al sistema di test, descrivendo le scelte implementative concrete e le problematiche affrontate nella sua realizzazione.

Il sistema di test, indicato anche come retriever-trial, ha una struttura più semplice rispetto al sistema di ricerca e ne è cliente: espone due soli casi d'uso, l'avvio di una sessione di test, **start_test**, e l'esecuzione di una singola query di test, **run_one**.

Locust simula un numero configurabile di client paralleli che eseguono ricerche contro il sistema di ricerca. Lo **start_test** viene invocato una sola volta, all'avvio della sessione di Locust; ciascun client simulato si limita poi a invocare ripetutamente **run_one**.

Il sistema di test riusa la schema configuration del sistema di ricerca, ma in una forma semplificata contenente le sole entità necessarie ai fini del test e riutilizza le classi di query già definite nel sistema di ricerca, estese con una classe dedicata per rappresentare la ground truth di ciascuna query.

Ogni esecuzione di **run_one**, tramite una porta dedicata, recupera una query di test e la relativa ground truth; tramite una seconda porta esegue la query contro il sistema di ricerca, ottenendo sia il risultato reale sia lo stato corrente della sessione di ingestion (per poter distinguere, in fase di analisi, i risultati raccolti durante un'ingestion in corso da quelli raccolti a dati stabili); infine, tramite una terza porta, registra l'esito su un database dedicato. Da questo database, tramite una vista, vengono calcolate le metriche di interesse, che Grafana si limita a leggere e visualizzare.

7.1 Perimetro di test

L'implementazione e la realizzazione dei test sono state ritenute molto dispendiose in termini di tempo, sia a livello di codice che di preparazione

7 Implementazione del sistema di test

dei dati di test; ciò ha portato alla scelta di ridurre l'esecuzione automatica alla ricerca linked ibrida non ottimizzata per lingua: essendo la più complessa tra tutte, costituisce un limite superiore ai tempi di esecuzione delle altre. I problemi di accuratezza delle singole ricerche sono intrinseci al tipo di ricerca (**Sezione 4.2**) e vengono mitigati proprio dall'uso della ricerca ibrida.

Le altre tipologie di ricerca vengono comunque analizzate tramite test manuali e script al fine di poter comunque esprimere un giudizio su di esse. L'estensione del sistema di test al fine di gestire tutte le tipologie di ricerca è lasciata a evoluzioni successive del sistema di test.

7.2 Architettura del codice

Anche il sistema di test segue il principio dell'architettura esagonale, con la stessa suddivisione in adapter, port, service e classi di dominio già vista per il sistema di ricerca.

Le classi relative al data modelling riutilizzate dal sistema di ricerca sono le relative alle query e ai risultati della ricerca. Sono state anche riutilizzate le classi entity seppur in modo ridotto; viene usata anche una classe schema configuration ma ridimensionata a collezione di entità.

Sono state aggiunte le seguenti classi di dominio:

- TestQuery, un semplice wrapper che contiene un id e una SearchRequest;
- SearchRequest, aggrega una SimilarityQuery o una HybridSearchRequest alla relativa GroundTruth;
- GroundTruth, rappresenta il risultato atteso dalla ricerca;
- RealResult, rappresenta il risultato reale di una ricerca;
- LogItem, aggrega una query di test con il relativo risultato e altre informazioni come l'id della sessione o lo stato di attività dell'ingestion.

La logica applicativa vera è realizzata da un singolo service descritto in **Codice 29**. Il service utilizza delle porte outbound per comunicare con l'esterno:

- TestQueryRepositoryPort, rappresenta il sistema di permanenza per le query da eseguire;
- LinkedSearchExecutorPort, rappresenta il sistema di ricerca da sottoporre a valutazione;
- IngestionStatusPort, recupera lo stato di attività della sessione di ingestion;
- LinkedEvaluationLogPort, rappresenta il sistema di permanenza su cui vengono salvati le TestQuery e il relativo RealResult.

7 Implementazione del sistema di test

```
1  class EvaluationService(StartTestUseCase,
2      RunOneUseCase):
3      def __init__(
4          self,
5          repository: TestQueryRepositoryPort,
6          linked_executor: LinkedSearchExecutorPort,
7          ingestion_status: IngestionStatusPort,
8          linked_logger: LinkedEvaluationLogPort,
9      ) -> None:
10         :
11     def start_test(self) -> TestSessionId:
12         :
13     def run_one(self) -> bool:
14         :
```

Codice 29: Sistema di test - EvaluationService

7.3 Sistema di persistenza

Il sistema di test ha due sistemi di permanenza con funzionalità ed esigenze distinte; uno si occupa di memorizzare le query di ricerca da eseguire, **TestQueryRepositoryPort**, l'altro registra i risultati delle ricerche, **LinkedEvaluationLogPort**.

TestQueryRepositoryPort viene solo letto in modo sequenziale, perciò si è scelto di utilizzare un semplice file JSONL. Questo rende facile la deserializzazione, riutilizzando lo stesso codice usato dal sistema di ricerca per la deserializzazione del payload JSON delle richieste HTTP. La sua modifica si traduce in una modifica ad un file. Ha anche una maggiore facilità di condivisione e tracciamento.

LinkedEvaluationLogPort viene usato dal backend Python per scritture continue al fine di registrare i risultati delle ricerche, serve inoltre un ricalcolo continuo al fine di calcolare le metriche. Postgres risponde a queste esigenze, le scritture sono veloci e il ricalcolo continuo è eseguito tramite una vista. Inoltre Grafana e Postgres sono direttamente compatibili, quindi il backend non deve occuparsi né di calcolare le metriche né di comunicarle alla dashboard.

7.4 Start test

La sessione di test viene avviata da Locust tramite un apposito adapter inbound, viene sempre rispettata la struttura esagonale.

Un'esecuzione tipica segue il seguente flusso:

1. avvio, il service si occupa di generare e memorizzare l'id della sessione di test e di recuperare la lista delle TestQuery da eseguire tramite la porta TestQueryRepositoryPort e le organizza in una coda;
2. esecuzione periodica delle ricerche, sono avviate da Locust sempre attraverso un adapter,
 1. la TestQuery viene recuperata dalla coda,
 2. tramite la porta IngestionStatusPort il service recupera l'informazione relativa allo stato di attività dell'ingestion,
 3. l'adapter RemoteIngestionStatusAdapter rinterroga il database Postgres per recuperare l'informazione relativa allo stato di attività dell'ingestion,
 4. la ricerca viene eseguita tramite la LinkedSearchExecutorPort per recuperare il RealResult,
 5. l'adapter RemoteLinkedSearchExecutorAdapter invia la query di ricerca all'endpoint appropriato,
 6. il LogItem e l'id della sessione di test corrente vengono persistiti tramite la porta LinkedEvaluationLogPort,
 7. l'adapter PostgresLinkedEvaluationLogAdapter si occupa di salvare sul database il LogItem, eventualmente rendendo esplicite informazioni come la posizione della GroundTruth.

7.5 Visualizzazione dei risultati

Grafana si interfaccia direttamente con il database Postgres utilizzando una sintassi SQL-like. Per mantenere le query semplici ho scelto di utilizzare la seguente vista per semplificare l'accesso.

7 Implementazione del sistema di test

```
1 CREATE VIEW evaluation_metrics AS WITH session_starts
  AS (
2     SELECT test_session_id, min(executed_at) AS
      session_started_at
3     FROM evaluation_log GROUP BY test_session_id)
  SELECT e.test_session_id, s.session_started_at,
4  e.ingestion_active, count(*) AS total_queries,
  avg(e.elapsed_time) AS avg_elapsed_time,
5  avg(CASE
6      WHEN e.rank_of_expected IS NOT NULL THEN 1.0
7      ELSE 0.0
8  END) AS retrieval_answer_rate,
9  avg(CASE
10     WHEN e.rank_of_expected IS NOT NULL THEN 1.0 /
      e.rank_of_expected
11     ELSE 0.0
12  END) AS mean_reciprocal_rank,
13  avg(CASE
14     WHEN e.rank_of_expected IS NOT NULL
15     AND e.rank_of_expected <= 1 THEN 1.0
16     ELSE 0.0
17  END) AS hit_rate_1,
18  avg(CASE
19     WHEN e.rank_of_expected IS NOT NULL
20     AND e.rank_of_expected <= 5 THEN 1.0
21     ELSE 0.0
22  END) AS hit_rate_5,
23  avg(CASE
24     WHEN e.rank_of_expected IS NOT NULL
25     AND e.rank_of_expected <= 10 THEN 1.0
26     ELSE 0.0
27  END) AS hit_rate_10,
28  avg(CASE
29     WHEN NOT e.has_results THEN 1.0
30     ELSE 0.0
31  END) AS not_found_rate
32 FROM evaluation_log e JOIN session_starts s USING
  (test_session_id)
33 GROUP BY
  e.test_session_id,s.session_started_at,e.ingestion_active;
```

Codice 30: Sistema di test - Vista metriche

7 Implementazione del sistema di test

Grafana utilizza una variabile che recupera in automatico la sessione più recente e parametrizza su di essa la dashboard.

```
1 SELECT
2     test_session_id AS __value,
3     to_char(session_started_at, 'YYYY-MM-DD HH24:MI:SS') || '
  - ' || test_session_id AS __text
4 FROM (
5     SELECT DISTINCT test_session_id, session_started_at
6     FROM evaluation_metrics
7 ) sessions
8 ORDER BY session_started_at DESC
```

Codice 31: Sistema di test - Variabile Grafana

Per la visualizzazione dei dati sono adottati due schemi, un indicatore di tipo time series per vedere i tempi di ingestion e un indicatore di tipo gauge per i valori unitari, indipendentemente dalla scala dei dati (percentuale o scalare).

```
1 SELECT
2     executed_at AS time,
3     elapsed_time
4 FROM evaluation_log
5 WHERE test_session_id = '$session' AND NOT ingestion_active
6 ORDER BY executed_at;
```

Codice 32: Sistema di test - Time series

```
1 SELECT
2     not_found_rate
3 FROM evaluation_metrics
4 WHERE test_session_id = '$session'
5     AND ingestion_active = false
```

Codice 33: Sistema di test - Gauge

7.6 Limitazioni imposte da elementi esterni

Nell'adapter dedicato al calcolo degli embedding tramite modello remoto è stato necessario introdurre un rallentamento artificiale delle prestazioni, a causa di blocchi temporanei imposti dal servizio remoto: un numero

7 Implementazione del sistema di test

eccessivo di chiamate in un breve intervallo causa un periodo di blocco durante il quale il servizio restituisce sistematicamente un errore 503.

Questo adapter è condiviso da più componenti del sistema di ricerca. La fase di arricchimento dell'ingestion e i service di ricerca semantica e ibrida, motivo per cui la limitazione descritta in questa sezione, per quanto discussa qui in un unico punto, si ripercuote su tutte queste componenti.

Da un punto di vista architetturale, questo intervento non introduce un nuovo collo di bottiglia nel sistema, ma sposta parzialmente, dall'esterno verso l'interno del sistema, un collo di bottiglia già esistente e non altrimenti evitabile. Per gestirlo sono stati introdotti meccanismi di retry con attesa esponenziale e numero massimo di tentativi, oltre a un limite al numero di richieste concorrenti verso il servizio remoto, realizzato tramite semafori e contatori.

7 Implementazione del sistema di test

Capitolo 8

Conclusioni

In questo capitolo traggio le conclusioni sul progetto.

8.1 Consuntivo finale

Una volta terminato il progetto ho redatto il consuntivo orario finale nella **Tabella 6** che suddivide in maniera approssimata le ore dedicate alle varie fasi.

Fase	Ore
Onboarding del progetto e studio delle tecnologie	48
Progettazione della struttura del db e prototipazioni	64
Progettazione e codifica della schema configuration	20
Progettazione e codifica della ricerca semantica	24
Progettazione e codifica della ricerca full-text	40
Progettazione e codifica della ricerca ibrida	8
Progettazione e codifica della ricerca linked	12
Progettazione e codifica del sistema di ingestion	32
Progettazione e codifica del sistema di test e observability	68
Totale	316

Tabella 6: Consuntivo orario finale

8.2 Requisiti soddisfatti

Tutti i requisiti descritti nella **Sezione 3.3** sono stati implementati come descritto nel riepilogo della **Tabella 7**.

Tipo	Obbligatoria	Desiderabili	Opzionali	Somma
Funzionali	48/48	0/1	0/2	48/51
Qualità	1/1	0/1	0/1	1/3
Vincolo	13/13	2/2	0/1	15/16
Totale	62/62	2/4	0/4	64/70

Tabella 7: Riepilogo dei requisiti soddisfatti

8.3 Rischi occorsi e mitigati

I rischi documentati nella **Sezione 2.4** emersi durante lo stage sono riportati in **Tabella 8**.

Descrizione	Mitigazione
R01: Incompletezza o ambiguità dei requisiti	Il rischio è stato incontrato durante lo studio iniziale. La mitigazione è stata il confronto con il tutor per colmare le lacune e definire chiaramente i requisiti.

Descrizione	Mitigazione
R02: Sovradimensionamento del progetto rispetto alle tempistiche	Il rischio è stato incontrato tra la fase intermedia e finale del progetto: implementare un sistema di test che permettesse di esaminare tutti i tipi di ricerca richiedeva diverse considerazioni non banali in fase di progettazione e una discreta quantità di tempo. Si è perciò limitato lo scope del test alla ricerca più importante.
R07: Rappresentatività dei dati di test e accesso limitato alla produzione	Il rischio è stato incontrato durante la preparazione dei dati di test. Non avendo accesso ai dati veri di produzione si è optato per usare dati fittizi, generati a partire da dataset in lingua italiana e inglese.
R08: Difficoltà nella valutazione oggettiva dei risultati di retrieval	Il rischio è stato incontrato durante la progettazione del sistema di test, mitigato tramite confronto con il tutor in cui sono state fissate le metriche di interesse.
R10: Bias tecnologico e deriva dei requisiti	Il rischio è stato incontrato nelle fasi iniziali dell'analisi delle funzionalità da realizzare, mitigato dando priorità alle caratteristiche del problema.

Tabella 8: Rischi occorsi con la loro mitigazione

8.4 Valutazione complessiva sulle tecnologie

Dei vari punti che questo progetto si prefiggeva di chiarire segue un resoconto ed un'analisi.

8 Conclusioni

- Adeguatezza di pgvector alla ricerca semantica: **Confermato**, le query non sono particolarmente complesse, i tempi di risposta sono sufficientemente bassi, in media tra i 200-300 millisecondi;
- Fusione di full-text e semantica lato db: **Confermata**, trattando le query SQL di semantica e full-text come subquery è facile implementare l'RRF;
- Linking lato db: **Confermato**, è possibile eseguire senza problemi i join tra le varie entità, essendo join su chiave primaria il lookup avviene tramite indice;
- Ricerca full-text: **Non confermata**, Elasticsearch senza troppe sorprese è estremamente più veloce di Postgres per la ricerca full-text, per quanto l'accuratezza tramite i workaround sia diventata sufficiente. Il tempo medio di una ricerca full-text non ottimizzata per lingua va dai 5 ai 10 secondi, contro circa 3 secondi per le ricerche ottimizzate per lingua. Tale risultato è coerente con quanto visto nella **Sezione 4.1.1**.

L'accuratezza generale viene considerata buona per tutti i vari tipi di ricerca: il retrieval answer rate è stato sempre del 100%, ma questo è dovuto al dataset di test, essendo generato automaticamente. In uno scenario realistico è probabile che sarà più bassa, circa 80%.

All'accuratezza contribuiscono inoltre diversi fattori esterni al sistema o elementi da gestire tramite configurazione:

- modello di embedding utilizzato,
- configurazioni di lingua,
- tipologia di vettori e distanza,
- qualità della query, testi poco precisi implicano risposte poco valide (questo aspetto è in genere gestito dalla parte generativa della RAG, query rewriting),
- composizione del corpus documentale.

8.5 Valutazione personale

Dal punto di vista personale ho trovato questo progetto molto stimolante. L'argomento mi è risultato molto interessante, seppur alcune parti come la costruzione automatica delle query siano state particolarmente ostiche.

Per quanto il non essere riuscito a raggiungere un risultato soddisfacente con la ricerca full-text sia stato una delusione, non è stata una sorpresa in quanto era un limite tecnologico noto fin dalle prime fasi di studio.

Per il resto mi ritengo soddisfatto.

Glossario

ACID – Atomicità, Coerenza, Isolamento e Durabilità: Rappresentano le quattro proprietà fondamentali di un DBMS che garantiscono affidabilità e correttezza. 2

Adapter: Denota un design pattern utilizzato in informatica nella programmazione orientata agli oggetti. 9

Radice dell'aggregato – Aggregate root: Nel Domain-Driven Design, oggetto che funge da punto di accesso unico a un insieme di entità e valori correlati che devono essere trattati come un'unica unità consistente, garantendo che le invarianti dell'intero aggregato siano rispettate a ogni modifica. 97

All-words query: Modalità di ricerca full-text che richiede la corrispondenza un intero insieme di parole senza un ordine specifico. 81

Analyzer: Componente che definisce come il testo viene elaborato prima di essere indicizzato o interrogato, tipicamente composto da una catena di filtri di normalizzazione e da un tokenizer, responsabile della segmentazione del testo in singoli token. 64

Any-word query: Modalità di ricerca full-text che richiede la corrispondenza di una singola parola. 81

API – Application Programming Interface: È un'interfaccia utile a permettere o facilitare la comunicazione tra diversi software o parti di un singolo software. 15

Architettura esagonale: Paradigma di progettazione del software ideato da Alistair Cockburn il cui obiettivo principale è isolare la logica di business centrale dai sistemi esterni, come database, interfacce utente o API di terze parti. 12

Block-Max WAND: Algoritmo di pruning per l'ordinamento top-k che sfrutta limiti superiori precalcolati sul punteggio dei documenti per escludere candidati che non potrebbero rientrare nei risultati finali, riducendo il costo computazionale della ricerca. 62

BM25: Indica una funzione di scoring avanzata usata nella ricerca full-text. Questa funzione è in grado di gestire lo scoring in base alla composizione della base documentale, attribuisce punteggi più bassi a parole molto frequenti e punteggi più alti a quelle più rare. 64

Chunk – Chunk di testo: Indica un segmento di testo frutto di divisioni di testi più lunghi. Necessario a trattare in modo più semplice

testi lunghi, in particolare il significato di un testo eccessivamente lungo non può essere trasposto in un vettore di embedding. **16**

Composition root: Punto unico del programma in cui vengono create e collegate tra loro le istanze concrete delle dipendenze dell'applicazione, separando la costruzione degli oggetti dalla logica applicativa che li utilizza. **93**

Configurazione testuale: Insieme di regole che definisce come Postgres elabora il testo per l'indicizzazione e la ricerca full-text: comprende la scelta del parser che suddivide il testo in token e dei dizionari che, per ciascun tipo di token, applicano trasformazioni come normalizzazione, rimozione delle stop word e stemming [3].

Cosine similarity: Metrica di similarità tra due vettori basata sul coseno dell'angolo che li separa, indipendente dalla loro magnitudine e quindi adatta a confrontare vettori di embedding normalizzati o di lunghezza variabile. **80**

CTE – Common Table Expression: Indica un set di risultati temporaneo e con un nome proprio, definito all'interno dell'ambito di esecuzione di una singola istruzione SQL (come **SELECT**, **INSERT**, **UPDATE** o **DELETE**). Funziona come una tabella virtuale o una sotto-query con un nome, creata al volo durante l'esecuzione della query principale ed eliminata subito dopo.

65

Dependency injection: Tecnica secondo cui le dipendenze di un componente non vengono create direttamente al suo interno, ma fornite dall'esterno al momento della sua costruzione, disaccoppiando il componente dall'implementazione concreta delle proprie dipendenze e facilitandone la sostituzione e il test. **110**

Distanza di Hamming: Metrica di distanza tra due sequenze di bit di uguale lunghezza, definita come il numero di posizioni in cui i bit corrispondenti differiscono; è la metrica naturalmente associata al confronto tra vettori quantizzati in forma binaria. **101**

Elasticsearch: Motore di analisi e ricerca distribuita. Funge da piattaforma di retrieval e salva dati strutturati, non strutturati e dati vettoriali. **2**

Embedding – Vettore di embedding: È una sequenza di numeri reali che trasforma un oggetto complesso (una parola, una frase, un'immagine o un file audio) in una rappresentazione matematica comprensibile per un computer. **2**

Framework: Indica un insieme strutturato di componenti software, librerie e regole che forniscono una base riutilizzabile per sviluppare applicazioni. Un framework offre funzionalità già pronte e un'architettura predefinita, guidando lo sviluppatore nelle scelte progettuali e riducendo il lavoro necessario per creare nuovi programmi.

GIN – Generalized Inverted Index: Indica una struttura dati che mappa i contenuti (come parole o numeri) direttamente alle loro posizioni o ai documenti in cui compaiono, consentendo ricerche full-text rapide. Nel contesto di Postgres indica un tipo di indice progettato per gestire valori composti (o elementi multipli all'interno di una singola riga) ed effettuare ricerche efficienti su di essi.

65

Ground truth: Indica un singolo risultato o un insieme di risultati considerati corretti e usati come riferimento per valutare l'accuratezza e le prestazioni di un sistema. **62**

Hitrate@k: È un indice statistico usato per valutare i sistemi di ricerca e recupero informazioni in base a quanto in alto posizionano la prima risposta corretta. Misura la percentuale di tentativi o interazioni che hanno prodotto un risultato corretto con rank minore o uguale a k rispetto al totale dei tentativi compiuti. Ad esempio hitrate@1 misura quanto spesso il primo risultato è corretto. **43**

HNSW – Hierarchical Navigable Small World: Struttura dati per la ricerca approssimata del vicino più prossimo, organizzata come un grafo gerarchico multilivello che permette di navigare efficientemente verso i vettori più simili senza confrontare l'intero insieme di dati, particolarmente adatta a scenari con inserimenti incrementali. **80**

IR – Information Retrieval: Insieme delle tecniche utilizzate per gestire la rappresentazione, la memorizzazione, l'organizzazione e l'accesso ad oggetti contenenti informazioni **3**

Ingestion: Indica il processo di caricamento e trasformazione dei documenti sorgente all'interno di un sistema di ricerca, che comprende l'estrazione dei dati, il calcolo degli embedding e la loro memorizzazione in una forma indicizzata e interrogabile.

2

Inner product: Metrica di similarità tra due vettori calcolata come somma dei prodotti delle rispettive componenti; a differenza della cosine similarity tiene conto anche della magnitudine dei vettori, per cui è equivalente ad essa solo quando i vettori confrontati sono già normalizzati. **80**

Iterative scan: Strategia di esecuzione che, in presenza di un filtro applicato dopo una ricerca approssimata su indice vettoriale, ripete la scansione dell'indice fino a recuperare il numero di risultati richiesto, compensando la possibile scarsità di risultati causata dal filtro [19]. **65**

LLM – Large Language Model: Modello di intelligenza artificiale addestrato su enormi quantità di testo per comprendere e generare linguaggio naturale, come GPT, Gemini o Mistral. Viene impiegato in compiti di analisi, estrazione di informazioni e generazione testuale. **3**

MRR – Mean Reciprocal Rank: È un indice statistico usato per valutare i sistemi di ricerca e recupero informazioni in base a quanto in alto posizionano la prima risposta corretta. Il reciproco del rank di una risposta ad una query è l'inverso della posizione (rank) della prima risposta corretta nella lista ordinata delle risposte. Il MRR è la media dei rank reciproci dei risultati per un insieme di query Q. **43**

Not found: Indica il numero di volte in cui una ricerca non produce alcun risultato.

43

Oversampling: Tecnica abbinata alle ricerche approssimate che recupera un numero di candidati superiore a quello richiesto dall'utente, per poi ricalcolarne il punteggio esatto e restituire solo i migliori k, compensando eventuali perdite di precisione. **80**

ParadeDB: Estensione per Postgres che integra un motore di ricerca full-text basato su BM25 direttamente nel database relazionale, sfruttando l'indice Tantivy (una libreria di ricerca scritta in Rust). Si propone come alternativa a Elasticsearch per i casi d'uso in cui si vuole evitare la sincronizzazione tra un sistema di persistenza primario e un motore di ricerca esterno, mantenendo le query in SQL standard e le garanzie ACID di Postgres. **5**

Persistenza poliglotta: Paradigma architetturale secondo cui un sistema adotta più tecnologie di persistenza dei dati distinte, ciascuna scelta in base alle caratteristiche del carico di lavoro o del modello dati che deve gestire, anziché affidarsi a un'unica soluzione generalista per tutti i casi d'uso. **1**

Pgvector: Estensione per Postgres, un database management system, che semplifica l'utilizzo dei vettori, consentendo di archivarli, cercarli e indicizzarli direttamente in un database relazionale. **2**

Phrase query: Modalità di ricerca full-text che richiede la corrispondenza esatta di una sequenza di parole nell'ordine specificato, anziché la semplice presenza dei singoli termini indipendentemente dalla loro posizione reciproca. **81**

PoC – Proof of concept: È un test pratico e mirato che serve a dimostrare se un'idea, un metodo o un software è realmente fattibile nel mondo reale. Fornisce una soluzione astratta al problema dell'interoperabilità tra interfacce differenti. **1**

Principio dei cassetti: Noto anche come *pigeonhole principle*, indica una classe di problemi in cui n oggetti vengono distribuiti in m contenitori, con n maggiore di m ; ne consegue necessariamente che almeno un contenitore conterrà più di un oggetto. **83**

Quantizzazione binaria: Tecnica di riduzione di precisione di un vettore che ne rappresenta ciascuna componente con un singolo bit anziché con un valore in virgola mobile, riducendo drasticamente lo

spazio occupato e il costo di confronto a scapito della precisione, tipicamente utilizzata per generare rapidamente un insieme di candidati da raffinare in una fase successiva. **101**

RAG – Retrieval Augmented Generation: Tecnica che permette agli LLM di reperire e incorporare informazioni da fonti di dati esterne. Questo permette di recuperare informazioni rilevanti da database, documenti caricati, fonti web, ecc...

Il vantaggio principale è l'attualità delle risposte fornite dall'LLM e la possibilità di non dover usare documenti privati in fase di addestramento. **3**

Ranking: Processo algoritmico con cui si assegna un punteggio di rilevanza a un insieme di documenti rispetto a una query, ordinandoli in una lista decrescente. **10**

Reranking: Indica il processo di combinazione dei risultati prodotti da diverse fonti o ricerche. Alcuni esempi sono il rescoring quando i punteggi delle diverse fonti sono direttamente comparabili, RRF o combinazione con modelli appositi quando non direttamente comparabili. **16**

Rescoring: Fase successiva al recupero di un insieme di candidati in cui il punteggio di rilevanza viene ricalcolato con maggiore precisione, tipicamente per correggere le approssimazioni introdotte da ottimizzazioni di ricerca. **80**

Retrieval answer rate: Indica la percentuale di volte in cui una ricerca produce un risultato corretto indipendentemente dalla posizione.

43

Retrieval latency: Indica il tempo di atteso medio trascorso tra l'avvio di una ricerca e la ricezione dei risultati dal punto di vista di chi invia la ricerca.

43

Retrieval wins: Indica il numero di volte in cui una ricerca produce un risultato indipendentemente dalla sua correttezza.

43

Ricerca full-text: È una tecnica informatica che permette di cercare parole o frasi all'interno di un intero documento o di un database, esaminando ogni singola parola archiviata e non solo parti limitate come i titoli o i metadati. **2**

Ricerca ibrida: Indica una tecnica di recupero delle informazioni che unisce la ricerca full-text e la ricerca semantica in un'unica richiesta. **2**

Ricerca linked: Modalità di ricerca che analizza ogni entità del database, con la possibilità di applicare un filtro, e combina i risultati secondo una configurazione che specifica come eseguire i join, vi è anche la possibilità di applicare un filtro al termine del join. **3**

Ricerca semantica: Anche detta ricerca vettoriale. Questo tipo di ricerca valuta la similarità basandosi su un criterio matematico di distanza tra due vettori di embedding. **2**

RRF – Reciprocal Rank Fusion: Algoritmo di aggregazione comunemente usato nell’ambito della ricerca ibrida, permette di combinare due o più elenchi di risultati con punteggi non direttamente paragonabili, utilizza solo la posizione di ogni elemento in ogni classifica al fine di calcolare il nuovo punteggio. **2**

Sharding: Tecnica di partizionamento orizzontale dei dati in più unità indipendenti (dette shard), distribuite su nodi diversi, per migliorare scalabilità e prestazioni di un sistema distribuito. **63**

Similarity search – Ricerca per similarità: Una funzione che cerca gli elementi della collezione più simili alla query secondo una misura di similarità e restituisce un ranking ordinato per grado di somiglianza decrescente. **66**

SQL – Structured Query Language: Linguaggio di programmazione standardizzato utilizzato per creare, gestire, modificare e interrogare database relazionali. **2**

Stemming: Tecnica linguistica che riduce una parola alla sua radice morfologica, in modo da far corrispondere varianti flesse dello stesso termine (ad esempio singolare e plurale) durante la ricerca [20]. **64**

Tokenizer: Componente che suddivide un testo in unità elementari (dette token), tipicamente parole o sotto-parole, come primo passo dell’elaborazione linguistica in un motore di ricerca [21]. **62**

Tsquery: Indica un tipo di dato nativo di PostgreSQL progettato specificamente per la ricerca testuale. Rappresenta il testo da cercare nel corpus documentale, viene usato nelle interrogazioni. **63**

Tsvector: Indica un tipo di dato nativo di PostgreSQL progettato specificamente per la ricerca testuale. Rappresenta un elemento del corpus documentale che viene cercato tramite una tsquery. **63**

Upsert: Operazione di scrittura su database che inserisce un nuovo record oppure, qualora esista già un record in conflitto con un vincolo di unicità, ne aggiorna i valori, evitando così di dover eseguire separatamente un controllo di esistenza seguito da un inserimento o un aggiornamento espliciti. **105**

Visitor: Design pattern comportamentale che permette di aggiungere nuove operazioni su una gerarchia di tipi senza modificarne le classi, incapsulando ciascuna operazione in un oggetto separato a cui ogni tipo della gerarchia delega l’esecuzione tramite un metodo dedicato. **95**

Bibliografia

- [1] Elastic, «Improving query efficiency». Consultato: 7 luglio 2026. [Online]. Disponibile su: <https://www.elastic.co/blog/faster-retrieval-of-top-hits-in-elasticsearch-with-block-max-wand>
- [2] Elastic, «Add and remove nodes in your cluster». Consultato: 7 luglio 2026. [Online]. Disponibile su: <https://www.elastic.co/guide/en/elasticsearch/reference/current/scalability.html>
- [3] PostgreSQL Global Development Group, «PostgreSQL: Documentation: 18: 12.6. Dictionaries». Consultato: 3 luglio 2026. [Online]. Disponibile su: <https://www.postgresql.org/docs/current/textsearch-dictionaries.html>
- [4] Elastic, «Analyzer reference». Consultato: 7 luglio 2026. [Online]. Disponibile su: <https://www.elastic.co/guide/en/elasticsearch/reference/current/analysis-analyzers.html>
- [5] Elastic, «Elastic Docs | Elastic». Consultato: 7 luglio 2026. [Online]. Disponibile su: <https://www.elastic.co/docs/reference/query-languages/query-dsl/query-dsl-minimum-should-match>
- [6] Elastic, «Reciprocal rank fusion | Elasticsearch Reference». Consultato: 7 luglio 2026. [Online]. Disponibile su: <https://www.elastic.co/docs/reference/elasticsearch/rest-apis/reciprocal-rank-fusion>
- [7] ParadeDB, «Reciprocal rank fusion | ParadeDB». Consultato: 10 luglio 2026. [Online]. Disponibile su: <https://www.paradedb.com/learn/search-concepts/reciprocal-rank-fusion#implementation-in-sql>
- [8] pgvector, «pgvector readme». Consultato: 10 luglio 2026. [Online]. Disponibile su: <https://github.com/pgvector/pgvector#filtering>
- [9] Elasticsearch, «Vector search filtering: how it works & how to use it effectively | Elasticsearch Labs». Consultato: 7 luglio 2026. [Online]. Disponibile su: <https://www.elastic.co/search-labs/blog/vector-search-filtering>
- [10] Grafana, «PostgreSQL data source | Grafana documentation». Consultato: 17 agosto 2026. [Online]. Disponibile su: <https://grafana.com/docs/grafana/latest/datasources/postgres/>
- [11] PostgreSQL Global Development Group, «PostgreSQL: Documentation: 18: Chapter 12. Full Text Search». Consultato: 3 luglio 2026. [Online]. Disponibile su: <https://www.postgresql.org/docs/current/textsearch.html>

- [12] TimescaleDB, «pgtextsearch readme». Consultato: 24 agosto 2026. [Online]. Disponibile su: https://github.com/timescale/pg_textsearch/blob/v1.3.1/README.md#limitations
- [13] Elastic, «Elastic Docs | Elastic». Consultato: 7 luglio 2026. [Online]. Disponibile su: <https://www.elastic.co/guide/index.html>
- [14] pgvector, «pgvector readme». Consultato: 10 luglio 2026. [Online]. Disponibile su: <https://github.com/pgvector/pgvector#indexing>
- [15] pgvector, «pgvector readme». Consultato: 10 luglio 2026. [Online]. Disponibile su: <https://github.com/pgvector/pgvector#binary-quantization>
- [16] pgvector, «pgvector readme». Consultato: 10 luglio 2026. [Online]. Disponibile su: <https://github.com/pgvector/pgvector#distances>
- [17] pgvector, «pgvector readme». Consultato: 10 luglio 2026. [Online]. Disponibile su: <https://github.com/pgvector/pgvector#exact-search>
- [18] Deepseo, «Reciprocal Rank fusion o RRF - DeepSEO Corso SEO Online». Consultato: 3 luglio 2026. [Online]. Disponibile su: <https://www.deepseo.it/reciprocal-rank-fusion-o-rrf/>
- [19] pgvector, «pgvector README». Consultato: 1 luglio 2026. [Online]. Disponibile su: <https://github.com/pgvector/pgvector#iterative-index-scans>
- [20] Elastic, «Stemming». Consultato: 7 luglio 2026. [Online]. Disponibile su: <https://www.elastic.co/guide/en/elasticsearch/reference/current/stemming.html>
- [21] Elastic, «Tokenizer reference». Consultato: 7 luglio 2026. [Online]. Disponibile su: <https://www.elastic.co/guide/en/elasticsearch/reference/current/analysis-tokenizers.html>